



UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA DE
SISTEMAS INFORMÁTICOS
GRADO EN INGENIERÍA DEL SOFTWARE

TRABAJO FIN DE GRADO

Diseño e implementación de una arquitectura de Business Intelligence para e-commerce basada en Data Warehouse y análisis avanzado de clientes

AUTOR: Rodrigo García Arroyo
TUTOR: Carlos Camacho Gómez

Madrid, 2026

Resumen

El crecimiento del comercio electrónico ha generado grandes volúmenes de datos que permiten analizar en profundidad el comportamiento de los clientes y mejorar la toma de decisiones empresariales. En este contexto, las soluciones de Business Intelligence y Data Science resultan fundamentales para transformar datos en información útil. El objetivo de este Trabajo Fin de Grado es diseñar e implementar una arquitectura de Business Intelligence aplicada a un entorno de e-commerce, utilizando como base el dataset Brazilian E Commerce Public Dataset by Olist. Se busca construir una solución completa que permita estructurar, analizar y visualizar los datos, así como obtener información relevante sobre el funcionamiento del negocio. Para ello, en primer lugar se ha diseñado un modelo dimensional basado en la metodología de Kimball, definiendo un esquema en estrella que organiza la información en tablas de hechos y dimensiones. Este modelo permite analizar el negocio desde distintas perspectivas, como productos, clientes o tiempo. A partir de este diseño, se han desarrollado procesos de integración y transformación de datos mediante técnicas ETL, que permiten preparar y cargar la información en el Data Warehouse de forma adecuada. Sobre esta base, se han construido distintos cuadros de mando interactivos que permiten analizar aspectos claves del negocio, facilitando la interpretación de los datos y proporcionando una visión global del sistema. Adicionalmente, se han incorporado técnicas de análisis de datos con el objetivo de profundizar en el estudio de comportamiento del cliente y los patrones de compra. Los resultados obtenidos evidencian el valor de estructurar y analizar los datos de forma adecuada para comprender el funcionamiento del negocio. La solución desarrollada permite integrar información procedente de distintas fuentes y transformarla en conocimiento útil, facilitando el análisis de los principales indicadores y la identificación de oportunidades de mejora. En conjunto, el proyecto pone de manifiesto la importancia de las soluciones de Business Intelligence en entornos de comercio electrónico, proporcionando una base sólida para la toma de decisiones y para el desarrollo de futuros análisis más avanzados.

Abstract

The growth of electronic commerce has generated large volumes of data that make it possible to analyze customer behavior in depth and improve business decision-making. In this context, Business Intelligence and Data Science solutions are essential for transforming data into useful information. The objective of this Final Degree Project is to design and implement a Business Intelligence architecture applied to an e-commerce environment, using the Brazilian E-Commerce Public Dataset by Olist as a basis. The aim is to build a complete solution that enables the structuring, analysis, and visualization of data, as well as the extraction of relevant insights into business performance. To achieve this, a dimensional model based on the Kimball methodology has been designed, defining a star schema that organizes information into fact and dimension tables. This model allows the business to be analyzed from different perspectives, such as products, customers, or time. Based on this design, data integration and transformation processes have been developed using ETL techniques, enabling the proper preparation and loading of data into the Data Warehouse. On this basis, several interactive dashboards have been developed to analyze key aspects of the business, facilitating data interpretation and providing an overall view of the system. Additionally, data analysis techniques have been incorporated to further explore customer behavior and purchasing patterns. The results obtained demonstrate the value of properly structuring and analyzing data to understand how the business operates. The developed solution enables the integration of information from different sources and its transformation into useful knowledge, facilitating the analysis of key performance indicators and the identification of improvement opportunities. Overall, the project highlights the importance of Business Intelligence solutions in e-commerce environments, providing a solid foundation for decision-making and for the development of more advanced future analyses.

Agradecimientos

En primer lugar, me gustaría agradecer a mi tutor, D. Carlos Camacho Gómez, por su orientación y apoyo durante el desarrollo del Trabajo Fin de Grado. Su dedicación y disponibilidad han sido fundamentales para poder llevar a cabo este proyecto. Continúo dando las gracias a todo el personal docente del Grado en Ingeniería del Software por contribuir de forma positiva a mi formación a lo largo de estos años, así como a mis compañeros, que han hecho de esta etapa universitaria una experiencia para toda la vida tanto a nivel personal como académico. Por último, quiero agradecer especialmente a mi familia por su apoyo incondicional durante todo el proceso. Gracias a ellos he podido afrontar este reto con esfuerzo y constancia, valores que han sido clave para alcanzar este objetivo.

Índice

1. Introducción	15
1.1. Contexto y motivación	15
1.2. Soluciones existentes y sus limitaciones	15
1.3. Objetivos	16
2. Contextos y fundamentos	18
2.1. Business Intelligence en entornos de datos	18
2.2. Modelos de almacenamiento de datos	20
2.2.1. Data Warehouse	20
2.2.2. Modelo dimensional	21
2.3. Herramientas de análisis y visualización	23
2.3.1. Microsoft Power BI	23
2.3.2. Tableau	24
2.3.3. Qlik	24
2.3.4. Looker	25
2.4. Frameworks y lenguajes de programación	25
2.4.1. Phyton	26
2.4.2. R	26
2.4.3. D3.js	27
2.4.4. SQL	27
2.5. Introducción al análisis de datos	28
2.6. Justificación de las herramientas elegidas	30
2.6.1. Python	30
2.6.2. MySQL y MySQL Workbench	30
2.6.3. PowerBI Desktop	31
3. Descripción del problema y casos de uso	32
3.1. Descripción del Dataset	32
3.1.1. olist_orders_dataset.csv	33
3.1.2. olist_order_items_dataset.csv	34
3.1.3. olist_customers_dataset.csv	34
3.1.4. olist_products_dataset.csv	35
3.1.5. olist_order_payments_dataset.csv	35
3.1.6. olist_order_reviews_dataset.csv	36
3.1.7. product_category_name_translation.csv	36
3.1.8. Archivos CSV descartados	36
3.2. Casos de uso	37
3.2.1. CU-01. Análisis del rendimiento de ventas	37
3.2.2. CU-02. Análisis del comportamiento de pago	37
3.2.3. CU-03. Análisis de la satisfacción del cliente	38
3.2.4. CU-04. Segmentación geográfica de clientes	38
3.2.5. CU-05. Segmentación de clientes por comportamiento	39
3.2.6. CU-06. Predicción de la satisfacción del cliente	39
4. Diseño de la solución	41
4.1. Diseño de la arquitectura del sistema	41
4.1.1. Data Sources	41
4.1.2. Data Area	42

4.1.3.	Data Access Tools	42
4.1.4.	Justificación de la ausencia de Área de Staging	43
4.2.	Diseño del modelo dimensional	43
4.2.1.	Metodología de diseño: Enfoque Kimball	43
4.2.2.	Tipo de esquema: Galaxy Schema	44
4.2.3.	Resumen de tablas del modelo	45
4.2.4.	Decisiones generales de diseño	46
4.2.5.	Uso de claves sustitutas (Surrogate Keys)	47
4.2.6.	Diseño tablas dimensión	47
4.2.7.	Diseño tablas hecho	49
4.3.	Diseño de los dashboards	50
4.3.1.	Boceto previo	50
4.3.2.	Principios de diseño aplicados	51
4.3.3.	Identidad visual	52
4.3.4.	Selección de visualizaciones	52
4.3.5.	Sistema de navegación	56
5.	Implementación de la solución desarrollada	57
5.1.	Integración y preparación de datos (ETL)	57
5.1.1.	load_dim_customer.py	57
5.1.2.	load_dim_product.py	59
5.1.3.	load_dim_date.py	62
5.1.4.	load_fact_sales.py	65
5.1.5.	load_fact_payment.py	69
5.1.6.	load_fact_review.py	72
5.2.	Construcción del Data Warehouse	75
5.2.1.	Forward Engineering desde MySQL Workbench	76
5.2.2.	Implementación de las tablas dimensión	76
5.2.3.	Implementación de las tablas hecho	78
5.3.	Modelos de análisis de datos	81
5.3.1.	Segmentación de clientes mediante K-means	81
5.3.2.	Predicción de la satisfacción del cliente	84
5.4.	Implementación de los dashboards	87
5.4.1.	Visión General	88
5.4.2.	Productos y Categorías	91
5.4.3.	Clientes y Geografía	94
5.4.4.	Análisis de pagos	97
5.4.5.	Segmentación de clientes	100
5.4.6.	Predicción de satisfacción	104
5.4.7.	Navegación e interactividad	108
6.	Resultados y conclusiones	112
6.1.	Resultados obtenidos	112
6.1.1.	Ventas y facturación	112
6.1.2.	Clientes y geografía	113
6.1.3.	Pagos	113
6.1.4.	Satisfacción del cliente	113
6.2.	Conclusiones	113
7.	Análisis de impacto	115

7.1. Impacto social	115
7.2. Impacto económico	115
7.3. Impacto medioambiental	116
7.4. Impacto ético	116
8. Líneas futuras	118
8.1. Ampliación del modelo de datos	118
8.2. Mejora del proceso ETL	118
8.3. Mejora de los modelos analíticos	119
8.4. Mejora de la capa de visualización	119
Referencias	120

Índice de figuras

1.	Evolución de la importancia del Business Intelligence a lo largo de los años.[11]	19
2.	Diagrama componentes Data Warehouse.[19]	21
3.	Ejemplo de esquema en estrella en un modelo dimensional. (Elaboración propia)	23
4.	Plataforma Power BI.[24]	24
5.	Plataforma Tableau.[26]	24
6.	Plataforma Qlik.[28]	25
7.	Plataforma Looker.[30]	25
8.	Plataforma Python.[32]	26
9.	Plataforma R.[34]	27
10.	Plataforma D3.[36]	27
11.	Plataforma SQL.[38]	28
12.	Relación entre los distintos ficheros del dataset Brazilian E-Commerce Public Dataset by Olist.[43]	33
13.	Arquitectura por capas del sistema desarrollado. (Elaboración propia)	41
14.	Modelo dimensional del sistema basado en un esquema en galaxia. (Elaboración propia)	45
15.	Boceto inicial de la estructura de los cuadros de mando. (Elaboración propia)	51
16.	Visualizaciones disponibles en Power BI para la construcción de cuadros de mando.[46]	56
17.	Configuración de conexión entre el proceso ETL y el Data Warehouse. (Elaboración propia)	57
18.	Fragmento de código correspondiente a la fase de extracción de <code>dim_customer</code> . (Elaboración propia)	58
19.	Fragmento de código correspondiente a la fase de transformación de <code>dim_customer</code> . (Elaboración propia)	58
20.	Fragmento de código correspondiente a la fase de carga de <code>dim_customer</code> . (Elaboración propia)	59
21.	Fragmento de código correspondiente a la fase de extracción de <code>dim_product</code> . (Elaboración propia)	60
22.	Fragmento de código correspondiente a la primera fase de transformación de <code>dim_product</code> . (Elaboración propia)	61
23.	Fragmento de código correspondiente a la segunda fase de transformación de <code>dim_product</code> . (Elaboración propia)	61
24.	Fragmento de código correspondiente a la fase de carga de <code>dim_product</code> . (Elaboración propia)	62
25.	Fragmento de código correspondiente a la fase de extracción de <code>dim_date</code> . (Elaboración propia)	63
26.	Fragmento de código correspondiente a la fase de transformación de <code>dim_date</code> . (Elaboración propia)	64
27.	Fragmento de código correspondiente a la fase de carga de <code>dim_date</code> . (Elaboración propia)	64
28.	Fragmento de código correspondiente a la fase de extracción de <code>fact_sales</code> . (Elaboración propia)	65
29.	Fragmento de código correspondiente a la primera fase de transformación de <code>fact_sales</code> . (Elaboración propia)	67
30.	Fragmento de código correspondiente a la segunda fase de transformación de <code>fact_sales</code> . (Elaboración propia)	68
31.	Fragmento de código correspondiente a la fase de carga de <code>fact_sales</code> . (Elaboración propia)	69

32.	Fragmento de código correspondiente a la fase de extracción de fact_payment . (Elaboración propia)	70
33.	Fragmento de código correspondiente a la fase de transformación de fact_payment . (Elaboración propia)	71
34.	Fragmento de código correspondiente a la fase de carga de fact_payment . (Elaboración propia)	72
35.	Fragmento de código correspondiente a la fase de extracción de fact_review . (Elaboración propia)	73
36.	Fragmento de código correspondiente a la fase de transformación de fact_review . (Elaboración propia)	74
37.	Fragmento de código correspondiente a la fase de carga de fact_review . (Elaboración propia)	75
38.	Sentencia SQL de creación del esquema del Data Warehouse. (Elaboración propia) . .	76
39.	Definición SQL de la tabla dim_customer . (Elaboración propia)	76
40.	Definición SQL de la tabla dim_product . (Elaboración propia)	77
41.	Definición SQL de la tabla dim_date . (Elaboración propia)	78
42.	Definición SQL de la tabla fact_sales parte uno. (Elaboración propia)	79
43.	Definición SQL de la tabla fact_sales parte dos. (Elaboración propia)	79
44.	Definición SQL de la tabla fact_payment . (Elaboración propia)	80
45.	Definición SQL de la tabla fact_review . (Elaboración propia)	81
46.	Consulta SQL utilizada para la extracción de datos del modelo de segmentación de clientes. (Elaboración propia)	82
47.	Fragmento de código correspondiente al preprocesamiento de datos y escalado de variables. (Elaboración propia)	82
48.	Fragmento de código correspondiente a la aplicación del algoritmo K-Means. (Elaboración propia)	83
49.	Preparación del conjunto de datos para el modelo de predicción de satisfacción. (Elaboración propia)	84
50.	Entrenamiento del modelo de regresión lineal. (Elaboración propia)	85
51.	Fragmento de código correspondiente al entrenamiento del modelo Random Forest. (Elaboración propia)	86
52.	Vista de modelo de Power BI con las relaciones del Data Warehouse y los modelos analíticos. (Elaboración propia)	88
53.	Página Visión General del informe desarrollado en Power BI. (Elaboración propia) . .	89
54.	Variables utilizadas en el gráfico comparativo de facturación anual. (Elaboración propia)	90
55.	Variables utilizadas en el gráfico de distribución por estado del pedido. (Elaboración propia)	91
56.	Página Productos y Categorías del informe desarrollado en Power BI. (Elaboración propia)	91
57.	Variables utilizadas en el gráfico de evolución temporal de la facturación. (Elaboración propia)	92
58.	Formato condicional aplicado a la tabla de rendimiento de productos. (Elaboración propia)	93
59.	Variables utilizadas en la tabla de rendimiento de productos. (Elaboración propia) . .	93
60.	Variables utilizadas en el gráfico de facturación por categoría de producto. (Elaboración propia)	94
61.	Página Clientes y Geografía del informe desarrollado en Power BI. (Elaboración propia)	94
62.	Variables utilizadas en el Treemap de facturación por estado. (Elaboración propia) . .	95
63.	Variables utilizadas en el gráfico circular de distribución de clientes por estado. (Elaboración propia)	96
64.	Variables utilizadas en la tabla de actividad de compra por cliente. (Elaboración propia)	96

65.	Formato condicional aplicado a la columna Tipo cliente. (Elaboración propia)	97
66.	Página Análisis de pagos del informe desarrollado en Power BI. (Elaboración propia) .	98
67.	Variables utilizadas en el gráfico de distribución del importe pagado por método. (Elaboración propia)	99
68.	Variables utilizadas en el gráfico de evolución temporal de pagos por método. (Elaboración propia)	99
69.	Variables utilizadas en el gráfico de distribución de pagos por número de cuotas. (Elaboración propia)	100
70.	Variables utilizadas en el gráfico de importe medio por método de pago. (Elaboración propia)	100
71.	Página Segmentación de clientes del informe desarrollado en Power BI. (Elaboración propia)	101
72.	Formato condicional aplicado a las variables de satisfacción y gasto. (Elaboración propia)	102
73.	Formato condicional aplicado a la variable de retraso medio. (Elaboración propia) . . .	103
74.	Variables utilizadas en el gráfico de distribución de segmentos de clientes. (Elaboración propia)	103
75.	Variables utilizadas en el gráfico de facturación por segmento. (Elaboración propia) . .	104
76.	Variables utilizadas en el gráfico de composición de satisfacción por segmento. (Elaboración propia)	104
77.	Página Predicción de satisfacción del informe desarrollado en Power BI. (Elaboración propia)	105
78.	Variables utilizadas en el gráfico de dispersión de predicción de satisfacción. (Elaboración propia)	106
79.	Variables utilizadas en el gráfico de importancia de variables del modelo Random Forest. (Elaboración propia)	107
80.	Variables utilizadas en el gráfico de error absoluto medio por valoración real. (Elaboración propia)	107
81.	Variables utilizadas en el gráfico de coeficientes de regresión lineal. (Elaboración propia)	108
82.	Barra de navegación superior implementada en la página Visión General. (Elaboración propia)	108
83.	Flechas de navegación implementadas en las páginas del informe. (Elaboración propia)	108
84.	Ejemplo de ventana de información implementada sobre visualización facturación 2018 VS 2017 de la página Visión General. (Elaboración propia)	109
85.	Grupo de elementos creado en el panel de selección. (Elaboración propia)	109
86.	Marcadores utilizados para mostrar y ocultar las ventanas informativas. (Elaboración propia)	110
87.	Configuración del botón de información mediante acciones de marcador. (Elaboración propia)	110
88.	Configuración del botón de cierre mediante acciones de marcador. (Elaboración propia)	111

Índice de tablas

1.	Resumen de tablas del modelo dimensional	46
2.	Tabla <code>dim_customer</code>	47
3.	Tabla <code>dim_product</code>	48
4.	Tabla <code>dim_date</code>	48
5.	Tabla <code>fact_sales</code>	49
6.	Tabla <code>fact_payment</code>	50
7.	Tabla <code>fact_review</code>	50
8.	Justificación de las visualizaciones del dashboard de Visión General	53
9.	Justificación de las visualizaciones del dashboard de Productos y Categorías	53
10.	Justificación de las visualizaciones del dashboard de Clientes y Geografía	53
11.	Justificación de las visualizaciones del dashboard de Análisis de pagos	54
12.	Justificación de las visualizaciones del dashboard de Segmentación de clientes	54
13.	Justificación de las visualizaciones del dashboard de Predicción de satisfacción	55
14.	Resumen del script <code>load_dim_customer.py</code>	57
15.	Resumen del script <code>load_dim_product.py</code>	59
16.	Resumen del script <code>load_dim_date.py</code>	63
17.	Resumen del script <code>load_fact_sales.py</code>	65
18.	Resumen del script <code>load_fact_payment.py</code>	70
19.	Resumen del script <code>load_fact_review.py</code>	73
20.	Características medias de los segmentos de clientes identificados mediante K-Means	83
21.	Coeficientes obtenidos por el modelo de regresión lineal	85
22.	Importancia de variables obtenida mediante el modelo Random Forest	86
23.	Comparativa de resultados entre los modelos predictivos	87
24.	Resumen de los principales resultados obtenidos	112

Acrónimos

BI Business Intelligence

DW Data Warehouse

ETL Extract, Transform and Load

KPI Key Performance Indicator

OLAP Online Analytical Processing

DS Data Science

DAX Data Analysis Expressions

LOD Level of Detail Expressions

CRM Customer Relationship Management

DOM Document Object Model

SVG Scalable Vector Graphics

HTML HyperText Markup Language

CSS Cascading Style Sheets

SQL Structured Query Language

SGBD Sistema de Gestión de Bases de Datos

API Application Programming Interface

CSV Comma-Separated Values

MAE Mean Absolute Error

ERP Enterprise Resource Planning

ODS Operational Data Store

SCD Slowly Changing Dimensions

DDL Data Definition Language

PK Primary Key

FK Foreign Key

SK Surrogate Key

PYMES Pequeñas y Medianas Empresas

RGPD Reglamento General de Protección de Datos

1. Introducción

1.1. Contexto y motivación

El comercio electrónico ha experimentado un crecimiento extraordinario durante la última década. Según datos de Statista, las ventas globales de comercio electrónico superaron los 5,8 billones de dólares en 2023 y se prevé que alcancen los 8 billones en 2027. En Latinoamérica, el crecimiento ha sido especialmente notable: Brasil se posiciona como el mayor mercado de e-commerce de la región, con decenas de millones de compradores digitales y un elevado volumen anual de transacciones comerciales. Este auge ha transformado radicalmente la forma en que las empresas operan, generando grandes volúmenes de datos transaccionales relacionados con pedidos, valoraciones, pagos, rutas de entrega o comportamiento de clientes[1].

Sin embargo, el volumen de datos por sí solo no genera valor. Una empresa que procesa miles de pedidos diarios acumula información que, sin un tratamiento adecuado, permanece desestructurada y difícil de interpretar. La capacidad de transformar estos datos en conocimiento útil se ha convertido en una ventaja competitiva fundamental. En este contexto, el Business Intelligence cobra especial relevancia, ya que permite estructurar, consolidar y visualizar la información de negocio con el objetivo de apoyar la toma de decisiones de forma ágil y fundamentada.

Por otro lado, la motivación de este proyecto surge del hecho de que las plataformas de comercio electrónico generan información distribuida entre múltiples sistemas, como ERP, CRM, plataformas logísticas o pasarelas de pago. Integrar y analizar dicha información de forma coherente constituye un desafío técnico y organizativo relevante. Además, muchas pequeñas y medianas empresas del sector no disponen de infraestructuras analíticas consolidadas, lo que dificulta responder con rapidez a preguntas clave relacionadas con la evolución de las ventas, el comportamiento de los clientes o la eficiencia logística [2].

Este proyecto nace de la necesidad de demostrar cómo es posible construir, utilizando tecnologías accesibles y ampliamente utilizadas en el ámbito del análisis de datos, una solución completa de Business Intelligence orientada al comercio electrónico, abarcando desde la adquisición y transformación de los datos hasta su visualización interactiva y análisis avanzado.

Un aspecto diferencial del proyecto es el uso de datos reales de negocio. El dataset de Olist recoge más de 100.000 pedidos realizados entre 2016 y 2018 en el mercado brasileño, incluyendo información real sobre clientes, productos, vendedores, métodos de pago, tiempos de entrega y valoraciones. Esto implica enfrentarse a problemas habituales en proyectos reales de datos, como inconsistencias, valores nulos, claves duplicadas, categorías sin traducción o fechas incorrectamente formateadas. Trabajar con este tipo de información, en lugar de utilizar datos sintéticos o previamente depurados, aporta una mayor cercanía a escenarios reales y hace que las decisiones de diseño y los procesos ETL desarrollados tengan una aplicabilidad directa en entornos profesionales [3].

1.2. Soluciones existentes y sus limitaciones

Ante el reto de analizar grandes volúmenes de datos en entornos de comercio electrónico, el mercado ofrece distintas aproximaciones, cada una con sus propias fortalezas y limitaciones.

Una primera vía son las plataformas de analítica integradas en los propios marketplaces, como los paneles de vendedor de Shopify[4]. Estas herramientas ofrecen métricas básicas relacionadas con ventas y rendimiento, pero permanecen limitadas al ecosistema de cada plataforma, dificultando el cruce de información con fuentes externas y restringiendo las posibilidades de personalización y análisis histórico avanzado. Resultan útiles para el seguimiento operativo diario, aunque insuficientes para proporcionar una visión estratégica global del negocio.

Una segunda aproximación son las herramientas de Business Intelligence generalistas, como Google Looker Studio o Power BI[5]. Estas soluciones destacan especialmente en el ámbito de la visualización y exploración de datos, pero requieren disponer previamente de información integrada, depurada y correctamente modelada. En consecuencia, gran parte del esfuerzo técnico asociado a procesos ETL, integración de fuentes y modelado analítico debe desarrollarse externamente a la propia herramienta. Finalmente, existen plataformas empresariales completas Microsoft Fabric[6], capaces de cubrir el ciclo completo de integración, almacenamiento y visualización de datos. Sin embargo, su coste económico, complejidad de implantación y dependencia de infraestructuras específicas dificultan su adopción en pequeñas y medianas empresas.

Es precisamente en este espacio intermedio, situado entre las herramientas demasiado simples y las soluciones empresariales de alta complejidad, donde se posiciona el presente proyecto.

1.3. Objetivos

Este Trabajo de Fin de Grado se centra en el diseño e implementación de una solución de Business Intelligence orientada al análisis de datos de comercio electrónico. Para ello, se parte del dataset Brazilian E-Commerce Public Dataset by Olist, a partir del cual se desarrolla una arquitectura analítica completa compuesta por procesos ETL, un Data Warehouse dimensional y distintos cuadros de mando interactivos desarrollados en Power BI. Además, el proyecto incorpora técnicas de análisis avanzado con el objetivo de ampliar las capacidades analíticas del sistema y obtener una visión más profunda del negocio.

Los principales objetivos de este trabajo fin de grado son:

- Diseñar un modelo dimensional basado en un esquema en galaxia (combinación de múltiples esquemas estrella) que permita estructurar la información para su análisis.
- Desarrollar procesos ETL que preparen la información para su explotación.
- Construir un Data Warehouse que sirva como base para el análisis.
- Crear dashboards interactivos en Power BI que faciliten la interpretación de datos.
- Analizar aspectos del negocio como los clientes, ventas o los métodos de pago, entre otros.
- Incorporar técnicas de Data Science que permitan un análisis más avanzado del negocio.

En cuanto al resto del documento, este se organiza de la siguiente manera. El capítulo 2 revisa el contexto y fundamentos teóricos sobre los que se apoya el proyecto, abordando conceptos relacionados con Business Intelligence, modelado dimensional y procesos ETL, entre otros. El capítulo 3 describe el

problema abordado y los requisitos del sistema, proporcionando contexto sobre el conjunto de datos y las necesidades analíticas identificadas.

Posteriormente, los capítulos 4 y 5 presentan el diseño y la implementación de la solución desarrollada, mostrando primero las decisiones de diseño adoptadas y posteriormente el proceso de construcción del sistema. En el capítulo 6 se presentan los principales resultados obtenidos, así como las conclusiones derivadas del análisis realizado. Por otro lado, el capítulo 7 evalúa el impacto del sistema desarrollado desde una perspectiva técnica y analítica, considerando su utilidad para apoyar la toma de decisiones y mejorar la comprensión del negocio. Finalmente, el último capítulo recoge posibles líneas futuras de mejora y ampliación del proyecto orientadas a incrementar las capacidades analíticas y funcionales del sistema.

2. Contextos y fundamentos

En este punto se presentan los principales conceptos y fundamentos teóricos para entender las tecnologías utilizadas en este proyecto y su correspondiente funcionamiento.

2.1. Business Intelligence en entornos de datos

El Business Intelligence (BI) agrupa un conjunto de procesos, tecnologías y distintas metodologías enfocadas a recopilar, integrar y analizar datos dentro de una organización. A través del uso de diversas técnicas y herramientas, el Business Intelligence permite organizar los millones de datos que se generan al día, analizarlos desde distintas perspectivas y presentarlos de manera fácilmente entendible. Esto ayuda a los usuarios a extraer conclusiones relevantes y a respaldar la toma de decisiones, ya sea en el ámbito operativo o estratégico.

El Business Intelligence tiene sus orígenes en la mitad del siglo XX, cuando empezaron a desarrollarse los primeros sistemas enfocados al análisis de datos. El término fue introducido por Hans Peter Luhn en los años 50 y, a lo largo del tiempo, el concepto fue evolucionando gracias a los avances tecnológicos. Fue a partir de los años 90 cuando el BI comenzó a ganar importancia en el mundo empresarial, especialmente con la aparición de nuevas herramientas que facilitaron en gran escala el acceso y análisis de datos [7].

En la primera década del siglo XXI, la democratización del acceso a los datos marcó un punto de inflexión en la evolución del Business Intelligence. Herramientas como Tableau o QlikView abrieron el análisis de datos a perfiles no técnicos mediante interfaces visuales e interactivas, reduciendo parcialmente la dependencia de los departamentos de IT. Paralelamente, el auge de internet y del comercio electrónico multiplicó el volumen de datos disponibles, impulsando el desarrollo de tecnologías Big Data, frameworks como Hadoop o Spark y las primeras plataformas de datos en la nube [8].

A partir de la década de 2010, el BI evolucionó hacia lo que se conoce como *Modern BI* o BI de auto-servicio. Plataformas como Power BI o Looker permitieron a los propios usuarios de negocio construir análisis y cuadros de mando sin depender completamente de desarrollos técnicos especializados. Al mismo tiempo, la consolidación del *cloud computing* favoreció la aparición de soluciones de Data Warehouse como Snowflake, Google BigQuery o Amazon Redshift, capaces de escalar el almacenamiento y procesamiento analítico sin necesidad de mantener infraestructura propia [9].

En la actualidad, el Business Intelligence continúa evolucionando hacia modelos de *Augmented BI* o inteligencia de negocio aumentada. La integración de técnicas de Inteligencia Artificial y *Machine Learning* permite automatizar la detección de patrones, generar alertas proactivas y desarrollar modelos predictivos integrados directamente en las herramientas analíticas. Además, el Procesamiento del Lenguaje Natural está modificando la forma en que los usuarios interactúan con los datos, incorporando interfaces conversacionales capaces de responder preguntas formuladas en lenguaje natural mediante visualizaciones automáticas.

Más recientemente, los sistemas de BI generativo, apoyados en modelos de lenguaje de gran escala (*LLMs*), comienzan a incorporar capacidades orientadas a la generación automática de explicaciones y narrativas sobre los datos, reduciendo la distancia existente entre el análisis técnico y la interpretación de negocio [10].

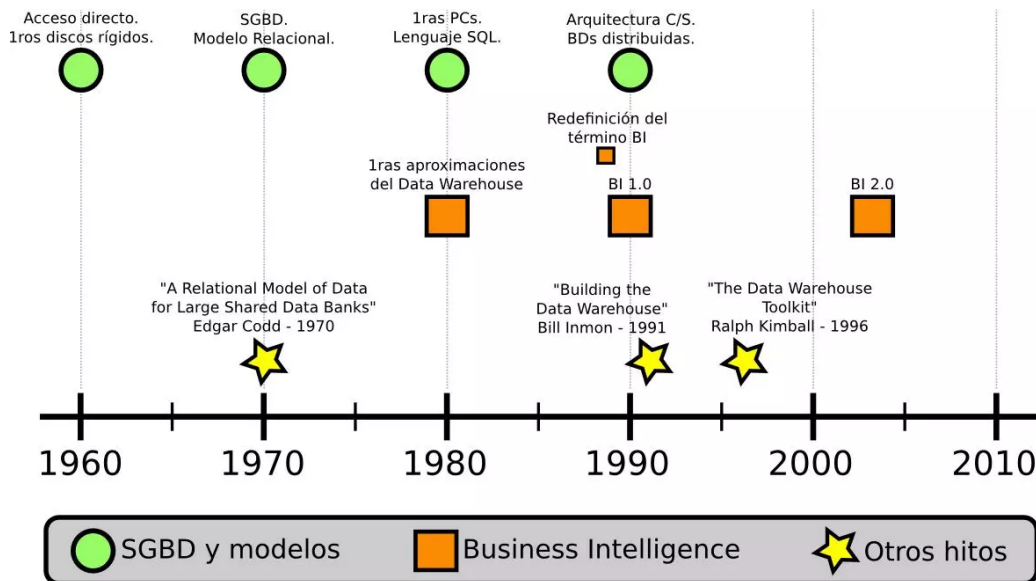


Figura 1: Evolución de la importancia del Business Intelligence a lo largo de los años.[11]

Por un lado, en el ámbito del análisis de datos, pueden distinguirse distintos enfoques en función del objetivo que se desea alcanzar [12]:

- **Análisis descriptivo.** Se centra en examinar datos históricos con el objetivo de comprender qué ha sucedido, identificando tendencias y patrones.
- **Análisis diagnóstico.** Permite profundizar en los datos para entender por qué han ocurrido determinados resultados.
- **Análisis predictivo.** Emplea modelos estadísticos y técnicas analíticas para anticipar posibles comportamientos futuros.
- **Análisis prescriptivo.** Sugiere acciones o decisiones basadas en los resultados obtenidos en los análisis anteriores.

Por otro lado, el funcionamiento general del Business Intelligence se basa en la capacidad de analizar los datos y representarlos visualmente para facilitar su comprensión. Mediante el uso de gráficos, tablas y cuadros de mando, los usuarios pueden interpretar la información de forma más rápida y detectar patrones relevantes.

En este contexto, se pueden definir algunos conceptos clave:

- **Dashboard.** Herramienta de visualización que permite centralizar, organizar y mostrar los principales datos, indicadores y métricas del negocio en una única vista [13].
- **KPI (Key Performance Indicator).** Indicador o medida cuantificable que utiliza una empresa para evaluar el rendimiento, la eficiencia o el éxito de un proceso dentro del negocio, facilitando la toma de decisiones basada en datos objetivos. En el contexto del comercio electrónico, los KPIs más habituales se agrupan en torno a cuatro áreas principales: ventas y facturación (volumen de

pedidos, ingresos totales), clientes (número de clientes nuevos frente a recurrentes, distribución geográfica), logística y operaciones (tiempo medio de entrega, pedidos entregados) y satisfacción (valoraciones de los clientes, reseñas) [14].

- **Insight.** Hallazgo o conclusión relevante extraída a partir del análisis de datos que aporta valor al negocio [15].

2.2. Modelos de almacenamiento de datos

Dentro del ámbito del Business Intelligence, con el objetivo de analizar los datos de forma eficiente, es imprescindible disponer de modelos de almacenamiento para organizar los datos estructuradamente facilitando su análisis. En este apartado se definen las principales soluciones utilizadas en este proyecto.

2.2.1. Data Warehouse

Uno de los elementos fundamentales dentro de este ámbito es el Data Warehouse, cuyo objetivo principal es proporcionar un entorno adecuado para el almacenamiento y análisis de grandes volúmenes de datos. A diferencia de los sistemas operacionales, diseñados para gestionar transacciones diarias, un Data Warehouse está orientado específicamente al análisis y consulta de información.

En términos generales, puede entenderse como un repositorio centralizado que integra datos procedentes de distintas fuentes, organizándolos para facilitar su explotación. En esta línea, Golfarelli y Rizzi lo definen como “un repositorio de datos históricos, integrados y consistentes” [16].

Una de las características más destacadas de un Data Warehouse es su orientación al análisis. Esto significa que los datos que se almacenan no se están modificando permanentemente, sino que se mantienen constantes para permitir consultas eficientes. Además de esto, la información se encuentra integrada y organizada coherentemente, lo que facilita su uso por herramientas analíticas. A lo largo del tiempo se han propuesto y diseñado distintas opciones para el diseño de un Data Warehouse. Por un lado, algunos proponen integrar todos los datos desde un principio, mientras que otros proponen un desarrollo incremental por áreas. Estos distintos enfoques permiten que cada organización adapte el desarrollo a sus intereses y necesidades [17]. En cuanto al punto de vista estructural, se compone de distintos elementos que permiten gestionar el flujo de datos desde que se originan hasta su explotación. Entre los más destacados se encuentran [18]:

- **ODS (Operational Data Store).** Es un sistema diseñado para integrar información de distintas fuentes en un entorno unificado. Su función principal es actuar de intermediario entre los sistemas operacionales y el Data Warehouse, facilitando el acceso a una versión integrada previa al almacenamiento definitivo.
- **Staging Area.** Es otro de los componentes intermediarios dentro de la arquitectura de un Data Warehouse. Su función principal es actuar como espacio temporal donde tienen lugar las tareas de extracción y transformación de los datos antes de su carga. Este sistema se diferencia del resto en que los datos almacenados no son persistentes, sino que suelen eliminarse una vez terminada la tarea asociada.

- **Datamarts.** Son subconjuntos de datos pertenecientes a un Data Warehouse orientados a un ámbito específico. En este caso en lugar de almacenar toda la información disponible, se enfoque solo en un bloque, por ejemplo, un departamento de ventas. Esto permite simplificar el acceso a la información y adaptarse únicamente a las necesidades específicas de esa área.
- **Metadatos.** Los metadatos son otros de los elementos fundamentales en un Data Warehouse. Básicamente proporcionan información sobre los datos almacenados con el objetivo de facilitar su comprensión. Se pueden entender como un mapa para gestionar el flujo de datos desde su fase inicial hasta los informes finales.

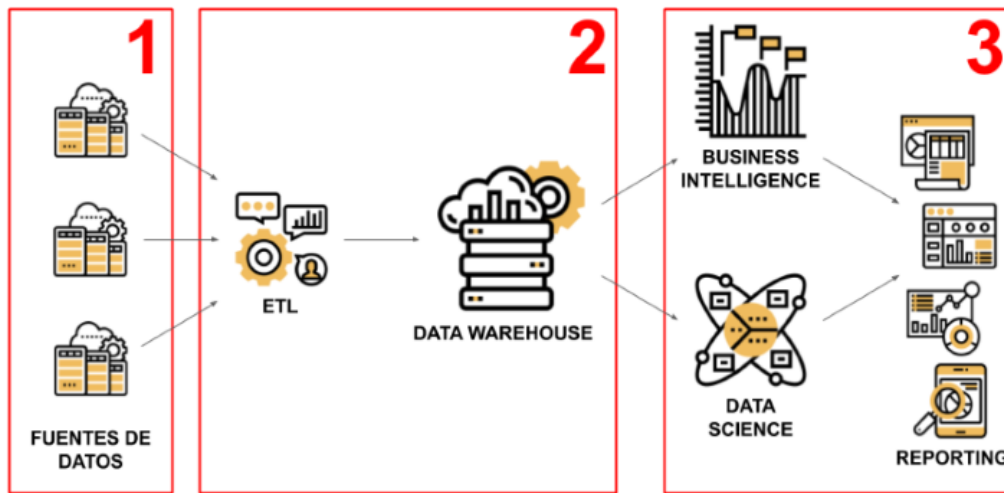


Figura 2: Diagrama componentes Data Warehouse.[19]

2.2.2. Modelo dimensional

El modelo dimensional es una técnica de diseños de bases de datos enfocada al análisis y consulta de grandes volúmenes de datos. Este esquema se diferencia del modelo tradicional en que no se centra en principalmente en eliminar la redundancia y garantizar la integridad de los datos, sino que apuesta por facilitar la comprensión por parte del usuario final y el rendimiento a la hora de ejecutar consultas complejas. Esta característica lo convierte en el estándar principal en el diseño de Data Warehouse y sistemas de Business Intelligence.

Este concepto fue normalizado por Ralph Kimball, cuya metodología conocida como “el enfoque bottom up” tiene como objetivo diseñar el almacén de datos a partir de subconjuntos de datos independientes orientados a un ámbito, es decir, los denominados Datamarts[20]. Kimball fija que todo modelo dimensional se diseña en torno a dos tipos de tablas: las tablas de hechos o fact tables y las tablas de dimensiones o dimensión tables.

Las tablas de hechos constituyen el núcleo del modelo dimensional. Almacenan los eventos del negocio que se desean analizar: ventas, pedidos... y están formadas principalmente por medidas cuantificables, denominadas métricas, que permiten facilitar el análisis. Cada registro en la tabla de hechos representa una ocurrencia de negocio específica en un nivel determinado de granularidad. La granularidad, por su parte, es uno de los conceptos más críticos y se encarga de determinar el grado de detalle con el que se

registra cada hecho y condiciona directamente la flexibilidad del modelo. Por otro lado, otro concepto importante que contienen las tablas de hechos son las claves foráneas, encargadas de referenciar a las tablas de dimensiones asociadas con el objetivo de contextualizar el análisis.

Las tablas de dimensiones, por su parte, aportan el contexto descriptivo en torno a cada hecho. Presentan atributos que permiten clasificar y dividir los datos, como por ejemplo el nombre de un producto o su región geográfica. En comparación con las tablas de hechos, estas suelen tener un menor tamaño, pero con una gran riqueza descriptiva en sus columnas [21]. Un aspecto de gran importancia en el diseño es el tratamiento de los cambios en las dimensiones a lo largo del tiempo, conocidos como Slowly Changing Dimensions (SCD). Kimball clasifica estos cambios en tres grandes tipos [22]:

- **Tipo 1 (Sobreescritura).** Es la estrategia más sencilla. Se basa en que cuando un atributo de una dimensión cambia, el valor se sustituye automáticamente, es decir, siempre se mantienen los datos actualizados y no se guarda ningún dato histórico. De aquí sale el principal inconveniente de este tipo, se pierde toda la trazabilidad histórica, pudiendo distorsionar el análisis retrospectivo.
- **Tipo 2 (Historial mediante nuevas filas).** Es el más utilizado en entornos de BI ya que ofrece una mayor riqueza analítica. Cuando se produce un cambio, no se modifica el registro, sino que se añade una nueva fila en la dimensión con el nuevo valor del atributo, dejando intacto el registro actual. Para la gestión de cada versión habitualmente se incluyen tres columnas adicionales: una fecha de inicio de validez, una fecha de fin de validez y un indicador booleano que indica si el valor corresponde al más actual. Debido a que cada versión genera un nuevo registro con clave primaria diferente, los hechos históricos permanecen vinculados a la versión vigente en el momento que ocurrieron, lo que permite por ejemplo calcular las ventas de un cliente cuando pertenecía a un determinado departamento y compararlas con las generadas en la actualidad. La principal desventaja de esta estrategia es el gran aumento de volumen que supone a lo largo del tiempo, así como un aumento en la complejidad de las consultas.
- **Tipo 3 (Atributo adicional para el valor anterior).** Esta estrategia mantiene un enfoque intermedio. En lugar de añadir filas, en una misma fila añade columnas adicionales para mantener el registro tanto del valor actual como del anterior. Esta estrategia puede resultar útil cuando la empresa necesita comparar el estado actual y anterior de forma habitual y directa. La limitación clara de este sistema es que solo permite guardar un número fijo de versiones anteriores, por lo que será insuficiente en un sistema donde los atributos varíen con una alta frecuencia.

La disposición física de estas tablas da lugar al esquema en estrella, que recibe nombre debido a que, a la hora de representarlo, las tablas de hechos ocupan el centro y las dimensiones se dirigen hacia afuera dando la sensación visual de una estrella. Esta estructura permite responder a preguntas analíticas de manera eficiente, por ejemplo, si tenemos una tabla de hechos FactVentas y una tabla de dimensión DimTiempo, podremos responder a preguntas como ¿Cuánto se vendió en un periodo de tiempo concreto? Es la técnica más típica y recomendada para implementar un modelo dimensional, ya que aporta un equilibrio entre simplicidad y eficiencia. Una variante del esquema en estrella es el esquema en copo de nieve, que consiste en descomponer los atributos de las dimensiones en otras tablas para reducir la redundancia de datos. Sin embargo, esto introduce una mayor complejidad en las

consultas además de afectar al rendimiento, por lo que se suele apostar más por el diagrama en estrella normalizado. Por otro lado encontramos el modelado en galaxia, el cual, a diferencia del modelado en estrella, incluye múltiples tablas de hechos que comparten dimensiones comunes.

En definitiva, el modelo dimensional y el esquema en estrella constituyen la base sobre la que se sustenta el Data Warehouse, facilitando todo el proceso desde el punto de vista de la comprensión como de la integración de futuras herramientas.

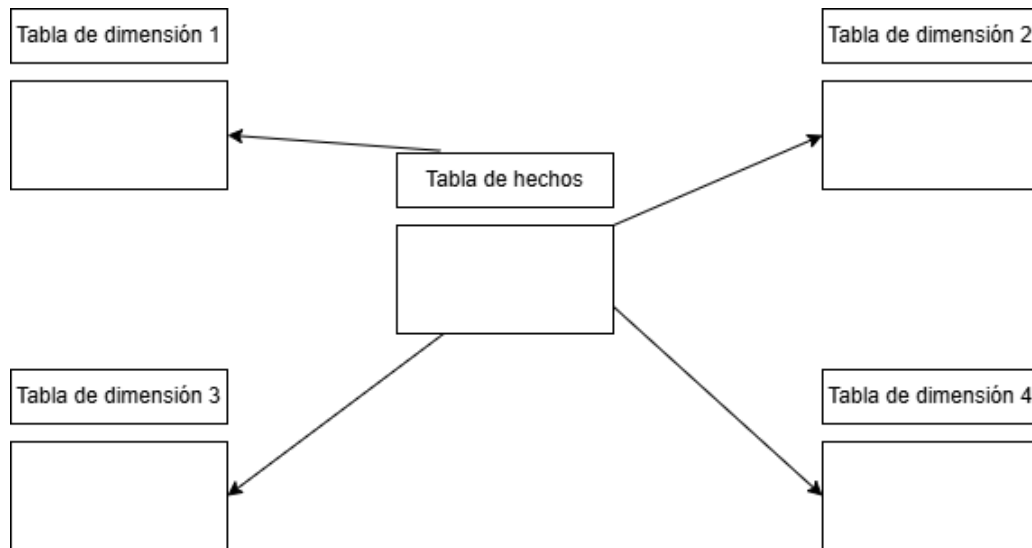


Figura 3: Ejemplo de esquema en estrella en un modelo dimensional. (Elaboración propia)

2.3. Herramientas de análisis y visualización

En el mundo del desarrollo de soluciones Business Intelligence encontramos un amplio rango de herramientas disponibles. Las herramientas de visualización constituyen la capa mas visible de las soluciones BI. Son las encargadas de transformar los datos procesados en representaciones gráficas que permiten al usuario final tratar con los datos.

2.3.1. Microsoft Power BI

Power BI es la plataforma de Business Intelligence desarrollada por Microsoft y una de las más destacadas a nivel mundial. Está compuesta por varios productos complementarios:

- **Power BI Desktop.** La aplicación de escritorio donde se modelan los datos y se construyen los informes.
- **Power BI Service.** El servicio en la nube que permite publicar, compartir y colaborar sobre los informes.
- **Power BI Mobile.** La aplicación para el uso en dispositivos móviles.

Su integración con el ecosistema Microsoft, incluyendo servicios como Azure, Excel o SQL Server, la convierte en una opción muy atractiva para las empresas que ya utilizan estos servicios. Destaca por

su potente motor de modelado basado en DAX, su conectividad con un amplio catálogo de datos y su gran capacidad para actualizar informes tanto en tiempo real como de forma programada [23].



Figura 4: Plataforma Power BI.[24]

2.3.2. Tableau

Tableau es una de las herramientas de visualización de datos mas reconocidas en el ámbito del BI, especialmente por la calidad y flexibilidad de sus representaciones gráficas. Permite construir visualizaciones de alta complejidad sin necesidad de programación, aunque también ofrece un lenguaje propio llamado LOD para análisis más avanzados. Esta herramienta fue adquirida por la empresa Salesforce en 2019 y, desde ese momento, ha reforzado su integración con soluciones CRM y plataformas cloud. La principal desventaja de esta herramienta frente a otras más destacadas como Power BI es su elevado coste de licenciamiento, siendo un gran inconveniente para proyectos de menor escala [25].



Figura 5: Plataforma Tableau.[26]

2.3.3. Qlik

Qlik es una empresa de software líder en el sector de BI que ofrece dos productos diferenciados:

- **QlikView.** Orientado al desarrollo de aplicaciones analíticas guiadas.
- **QlikSense.** Herramienta más moderna y enfocada al autoservicio analítico.

Ambos se basan en un motor asociativo propio que permite explorar relaciones de datos en cualquier dirección sin definir previamente jerarquías, a diferencias de los sistemas tradicionales. Esta capacidad es una de sus principales características que lo diferencian a nivel competitivo [27].



Figura 6: Plataforma Qlik.[28]

2.3.4. Looker

Looker es una plataforma de BI basada en web que fue adquirida por Google en 2019 y actualmente se encuentra integrada en Google Cloud. Su principal distintivo es el uso de LookML, un lenguaje de modelo que centraliza la definición de métricas y dimensiones en una capa reutilizable. Gracias a esto se facilita la consistencia de los datos en la organización y se reduce la dispersión de definiciones en distintos equipos. Se enfoca en entornos cloud-native y empresas que contienen equipos de datos maduros [29].



Figura 7: Plataforma Looker.[30]

2.4. Frameworks y lenguajes de programación

Además de las herramientas y representaciones visuales, el desarrollo de las soluciones BI requiere el uso de lenguajes de programación y frameworks para automatizar los procesos de extracción, transformación y carga de datos (ETL), realizar análisis avanzados y gestionar la infraestructura de datos subyacente.

2.4.1. Phyton

Python es el lenguaje de referencia en el ámbito del análisis e ingeniería de datos. Su sintaxis clara y extenso contenido de bibliotecas lo hacen la mejor alternativa para realizar tareas de procesamiento y transformación de datos. Entre las bibliotecas mas importantes en contextos de BI destacan:

- **Pandas.** Manipulación y análisis de datos tabulares.
- **NumPy.** Operaciones numéricas y matriciales.
- **SQLAlchemy.** Interacción con bases de datos relacionales mediante una capa de abstracción orientada a objetos.
- **Matplotlib/Seaborn.** Generación de visualizaciones estáticas.

Adicionalmente encontramos otras bibliotecas como Airflow que permiten orquestar flujos de trabajo de datos complejos, mientras que otras como data build tool facilita la transformación de datos directamente en el Data Warehouse [31].



Figura 8: Plataforma Python.[32]

2.4.2. R

R es un lenguaje estadístico de código abierto utilizado en entornos académicos y de investigación. Ofrece capacidades de análisis estadístico y diferentes paquetes para la visualización de datos, siendo ggplot2 uno de los más utilizados. Pese a no tener el mismo reconocimiento en el ámbito empresarial que otros lenguajes como Python, es una opción válida y con gran potencia para proyectos que contienen componentes estadísticos o predictivos [33].



Figura 9: Plataforma R.[34]

2.4.3. D3.js

D3.js (Data-Driven Documents), desarrollado por Mike Bostock, es la biblioteca de referencia para la visualización de datos en entornos web. Permite vincular datos a elementos del DOM y aplicarles transformaciones visuales mediante SVG, HTML Y CSS, ofreciendo así un control total sobre el gráfico resultante. Presenta una flexibilidad prácticamente ilimitada, lo que también implica la necesidad de una curva de aprendizaje pronunciada, ya que requiere una gran cantidad de conocimientos [35].



Figura 10: Plataforma D3.[36]

2.4.4. SQL

SQL es el lenguaje estándar para la definición, manipulación y consultas de datos en sistemas de bases de datos relacionales. Resulta fundamental en los proyectos de Data Warehouse, tanto para la creación

del esquema de tablas como para la construcción de las transformaciones ETL y las consultas sobre el modelo dimensional.

Dentro de los distintos gestores que implementan este lenguaje, podemos destacar MySQL, un SGBD de código abierto que destaca por su estabilidad, rendimiento y su amplia comunidad de soporte. Además, dispone de MySQL Workbench como entorno gráfico oficial, que permite la creación y diseño de esquemas, la ejecución de consultas y la administración del servidor, lo que simplifica en gran medida las tareas de modelado y mantenimientos de la base de datos [37].



Figura 11: Plataforma SQL.[38]

2.5. Introducción al análisis de datos

El crecimiento exponencial en el volumen de datos generados por las empresas ha impulsado el desarrollo de una serie de técnicas orientadas a extraer valor de dichos datos: el Data Science o ciencia de datos. A pesar de que el análisis de datos como concepto lleva existiendo mucho tiempo atrás, fue a finales del siglo XXI cuando el término Data Science comenzó a ganar popularidad, siendo conocido como una combinación de técnicas que combinan estadística, matemáticas, programación y conocimiento del negocio con el objetivo de apoyar la toma de decisiones basadas en la evidencia [39].

En cuanto a la relación de Data Science y Business Intelligence, a pesar de compartir su intención de convertir datos en información útil, se diferencian en su enfoque y el tipo de preguntas que afrontan. Por un lado, el BI tradicional se orienta hacia el análisis descriptivo, haciendo uso de herramientas que monitorizan el rendimiento del negocio mediante indicadores y cuadros de mando. Por otro lado, el Data Science amplía este alcance incorporando técnicas de mayor complejidad. Dentro de este ámbito, podemos distinguir varios tipos de análisis [40]:

- **Análisis diagnóstico.** Trata de explicar por qué ha ocurrido algo.
- **Análisis predictivo.** Estima que es probable que ocurra en el futuro aplicando modelos estadísticos y de aprendizaje automático.

- **Análisis prescriptivo.** Va un paso más allá que el resto de los análisis, sugiriendo qué acción debería tomarse para optimizar un resultado.
- **Análisis descriptivo.** Responde a la pregunta ¿qué ha pasado?. Resume datos históricos para entender el estado actual.

En la práctica, ambas disciplinas son complementarias y suelen utilizarse conjuntamente, siendo común que los equipos de Data Science integren sus modelos como una capa más sobre la infraestructura BI existente.

A partir de lo mencionado anteriormente, resulta fundamental comprender el ciclo de vida que siguen los datos en este tipo de proyectos.

En primer lugar, todo comienza con la recopilación de los datos, que se basa en identificar toda la información accediendo a las distintas fuentes: bases de datos, APIs, ficheros u otras fuentes. Tras tener toda la información recogida, debemos proceder a la limpieza de los datos, fase fundamental y en la que probablemente emprendamos más tiempo. Durante esta fase, se detectan posibles inconsistencias en las fuentes de datos, por ejemplo, valores nulos, errores en la sintaxis que pueden provocar fallos en un futuro, valores duplicados o simplemente información que no consideremos de importancia para nuestros futuros análisis, por lo que trataremos los datos hasta disponer de la información necesaria para nuestro proyecto.

Una vez tengamos los datos preparados, debemos pasar a la fase de análisis, etapa muy importante en los proyectos de Data Science, en la que a través de la observación de los datos decidiremos cuáles son las técnicas más efectivas y adecuadas para nuestro proyecto. Posteriormente pasaremos a la fase de modelado, en la que aplicaremos los algoritmos correspondientes: regresión, clasificación, clustering, entre otros, para extraer patrones o construir modelos predictivos. Por último, finalizaremos el proceso con la fase de despliegue en la que transformaremos los resultados en representaciones visuales como informes o gráficas con el objetivo de facilitar el entendimiento por parte del usuario final [39].

En paralelo a esta evolución en el análisis de datos, se ha producido una aparición y especialización de distintos roles vinculados a este ámbito, entre los que destacan [41]:

- **Data Engineer.** Es el responsable de diseñar y mantener la infraestructura de datos, por lo que su trabajo está vinculado a los pipelines ETL, almacenes de datos y sistemas de integración.
- **Data Analyst.** Se centra en la exploración y visualización de datos históricos para apoyar a la toma de decisiones, por lo que es el rol más cercano y vinculado al BI tradicional.
- **Data Scientist.** Combina conocimientos estadísticos y de programación para desarrollar modelos predictivos y técnicas de aprendizaje automático.
- **Machine Learning Engineer.** Se especializa en llevar los modelos desarrollados por el Data Scientist a entornos de producción, garantizando su escalabilidad y mantenimiento a lo largo del tiempo. Además de esto, también desarrolla sus propios modelos basados en datos.

En la realidad, estos roles no tienen una clara diferenciación, existiendo con frecuencia personas que asumen distintas responsabilidades de varios de ellos, especialmente en organizaciones que presentan un tamaño medio.

2.6. Justificación de las herramientas elegidas

El criterio de elección de herramientas para realizar este proyecto se ha basado en la experiencia previa utilizando ciertas tecnologías, así como en un análisis de las herramientas más adecuadas en los diferentes ámbitos tratados. A continuación se justifica la elección de cada herramienta en relación con su función dentro de la arquitectura de la solución.

2.6.1. Python

Python ha sido la herramienta central de este proyecto desde el punto de vista del desarrollo, teniendo una función esencial en dos aspectos clave. Por un lado, es el motor de los procesos ETL, encargado de extraer los datos, transformarlos según las necesidades del proyecto y posteriormente cargarlos para crear el Data Warehouse. Por otro lado, ha sido el entorno de implementación de los métodos de Data Science aplicados para complementar el análisis de los datos.

La elección de esta herramienta se justifica por varios motivos. En primer lugar, es una tecnología que he usado a lo largo de la carrera, teniendo una experiencia previa a la que, sumado el trabajo de investigación, me ha permitido adaptarme mejor al desarrollo. En segundo lugar, hay que destacar que Python presenta una enorme cantidad de bibliotecas, especialmente orientadas al tratamiento de datos, destacando Pandas para la manipulación de datos y mysql-connector-python para la integración con MySQL. En segundo lugar, valorar la versatilidad de la herramienta, la cual ha permitido efectuar en un mismo lenguaje tanto la ingeniería de datos como el análisis avanzado, evitando así la necesidad de incluir nuevas tecnologías en cada fase. Finalmente, Python es el lenguaje principal en el ámbito del aprendizaje automático y la ciencia de datos, por lo que su uso ha sido esencial para la implementación de algoritmos de clustering y predicción, que son parte del alcance de este trabajo.

2.6.2. MySQL y MySQL Workbench

MySQL ha sido el sistema gestor de bases de datos elegido para alojar el Data Warehouse del proyecto. La alta experiencia utilizando este sistema en distintas asignaturas como base de datos o base de datos avanzadas sumado a su naturaleza open source y su amplia contabilidad con Python lo convierte en una opción sólida para el almacenamiento y gestión de los datos transformados. Sobre MySQL se ha implementado el modelo dimensional en esquema galaxia, diseñado mediante MySQL Workbench, que integra las distintas tablas de hechos y dimensiones que conforman la base del proyecto.

La conexión entre Python y MySQL se ha realizado mediante la biblioteca mysql-connector-python, el conector oficial de MySQL para Python, que permite ejecutar desde los scripts ETL todas las operaciones necesarias sobre la base de datos: vaciado de tablas, inserciones masivas mediante executemany y consultas de dimensiones para la resolución de claves sustitutas.

Por otro lado, MySQL Workbench ha sido utilizado, además de para diseñar el modelo dimensional, para la administración de la base de datos resultante, permitiendo verificar y supervisar la estructura de las tablas. Por último, ha actuado como punto de conexión entre la capa de almacenamiento y la capa de visualización, facilitando la importación de los datos directamente en Power BI Desktop a través de la conexión con el servidor MySQL.

2.6.3. PowerBI Desktop

Power BI Desktop ha sido la herramienta elegida para la construcción de los cuadros de mando que conforman la capa de visualización de la solución. La elección de esta herramienta, en este caso, no se basa en la experiencia previa, lo que ha implicado la visualización de un curso para comprender las bases de la herramienta [42]. Su elección por tanto se ha basado en su capacidad para conectarse directamente a la base de datos de MySQL, importar el modelo dimensional definido y construir sobre este un modelo semántico que permita a los usuarios finales explorar los datos de forma intuitiva. A este hay que añadir la recomendación previa de compañeros para utilizar esta herramienta debido a considerarla intuitiva y fácil de manejar.

Entre sus características más relevantes para este proyecto destaca su motor de cálculo basado en DAX, el cual permite definir métricas sobre el modelo dimensional. Por otro lado, también cabe destacar su amplio catálogo de visualizaciones interactivas y su capacidad para integrar en un mismo informe datos provenientes de distintas tablas del modelo, respetando las relaciones previamente definidas entre hechos y dimensiones. Además, hay que destacar también que la herramienta es de uso gratuito en su versión de escritorio, lo que la hace accesible sin coste adicional para el desarrollo del proyecto.

3. Descripción del problema y casos de uso

En este apartado se presenta el contexto del problema abordado en el proyecto, los archivos en formato CSV con los datos empleados y los requisitos que debe satisfacer el sistema desarrollado.

En este contexto, el principal problema tratado radica en la ausencia de una estructura unificada que permita transformar los datos disponibles en información útil para la toma de decisiones por parte de los distintos actores del negocio, como gestores o vendedores. A pesar de que se dispone de gran cantidad de información relativa a pedidos, clientes, pagos y valoraciones, esta se dispone muy dispersa en tablas independientes y altamente normalizadas, lo que complica obtener una visión global de negocio.

Para dar respuesta a esta problemática, se plantea el diseño e implementación de una arquitectura de BI basada en un Data Warehouse, cuya implementación se desarrollará en apartados posteriores.

3.1. Descripción del Dataset

El sistema desarrollado en este trabajo utiliza el Brazilian E-Commerce Public Dataset by Olist, un conjunto de datos público disponible en la plataforma Kaggle [43]. Este dataset fue publicado por Olist, la mayor empresa de venta online en marketplaces brasileños, y contiene información real y anonimizada de aproximadamente 100.000 pedidos realizados entre los años 2016 y 2018 en múltiples plataformas de comercio electrónico de Brasil.

El dataset incluye nueve archivos CSV interrelacionados mediante identificadores comunes, entre los que destacan `order_id`, `customer_id` y `product_id`, y cubre todas las fases del ciclo de vida de un pedido de e-commerce : desde el momento de la compra hasta que es valorado por el cliente.

A continuación se describe cada uno de los archivos del dataset original, indicando para cada uno de ellos su contenido. Posteriormente se señala también los archivos que han sido descartados para el desarrollo del proyecto y la razón de esta decisión.

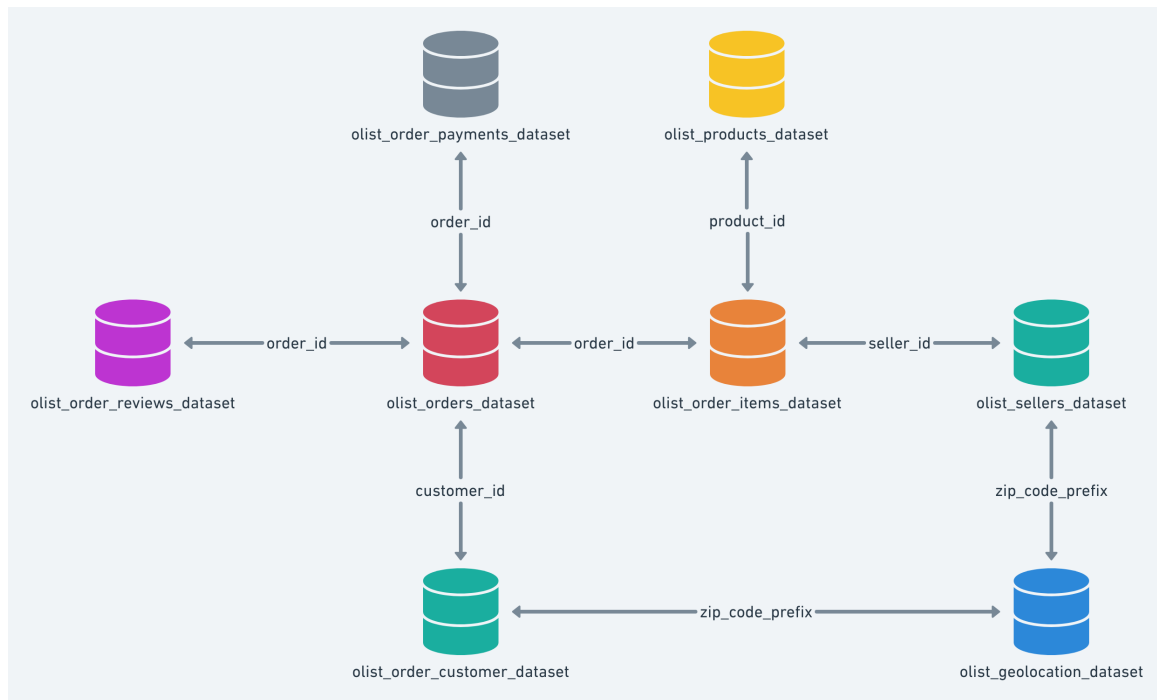


Figura 12: Relación entre los distintos ficheros del dataset Brazilian E-Commerce Public Dataset by Olist.[43]

3.1.1. olist_orders_dataset.csv

Este fichero contiene la información general de los pedidos realizados en la plataforma de comercio electrónico, así como las fechas asociadas a su ciclo de vida.

Su contenido viene dividido en las siguientes columnas:

- **order_id.** Identificador único del pedido.
- **customer_id.** Identificador del cliente asociado al pedido.
- **order_status.** Estado actual del pedido (delivered, shipped, canceled, invoiced, processing, unavailable y approved).
- **order_purchase_timestamp.** Fecha y hora en la que el pedido fue realizado.
- **order_approved_at.** Fecha y hora en la que el pedido fue aprobado por el sistema.
- **order_delivered_carrier_date.** Fecha y hora en la que el pedido fue entregado al personal de transporte.
- **order_delivered_customer_date.** Fecha y hora en la que el pedido fue entregado al cliente.
- **order_estimated_delivery_date.** Fecha estimada de entrega mostrada al cliente a la hora de la compra.

Este fichero, por tanto, es la base temporal y operativa para el análisis de ventas.

3.1.2. olist_order_items_dataset.csv

Este fichero describe los productos incluidos en cada pedido. Define la granularidad del hecho principal de ventas.

Su contenido viene dividido en las siguientes columnas:

- **order_id**. Identificador único del pedido al que pertenece el item.
- **order_item_id**. Identificador que permite distinguir múltiples productos en un mismo pedido.
- **product_id**. Identificador único del producto vendido.
- **seller_id**. Identificador único del vendedor que suministra el producto.
- **shipping_limit_date**. Fecha y hora límite para el envío del pedido al socio logístico por parte del vendedor.
- **price**. Precio del producto.
- **freight_value**. Coste de envío asociado al item.

Para calcular el precio final de un pedido puede haber confusiones, por lo que se debe tener en cuenta lo siguiente: Si tenemos un pedido con id 1, el cuál tiene asociado tres productos iguales con un precio de 20\$ y un coste de envío de 10\$, el valor total del pedido sería $20 \cdot 3 + 10 \cdot 3 = 90\$$.

3.1.3. olist_customers_dataset.csv

Este fichero contiene la información general de los clientes registrados en la plataforma de comercio electrónico.

Su contenido viene dividido en las siguientes columnas:

- **customer_id**. Identificador único del cliente asociado a un pedido. En este caso no es único, sino que un mismo cliente tendrá varios **customer_id** si realiza múltiples compras.
- **customer_unique_id**. Identificador único del cliente dentro de la plataforma.
- **customer_zip_code_prefix**. Prefijo del código postal del cliente.
- **customer_city**. Ciudad de residencia del cliente.
- **customer_state**. Región de residencia del cliente.

Este fichero es fundamental para identificar a cada usuario y asociarlo con su localización, facilitando los análisis y segmentaciones en función de las regiones o ciudades.

3.1.4. olist_products_dataset.csv

Este fichero contiene la información descriptiva de los productos registrados.

Su contenido viene dividido en las siguientes columnas:

- `product_id`. Identificador único del pedido.
- `product_category_name`. Nombre de la categoría del producto.
- `product_name_length`. Longitud del nombre del producto.
- `product_description_length`. Longitud de la descripción del producto.
- `product_photos_qty`. Número de fotografías asociadas al producto.
- `product_weight_g`. Peso del producto en gramo.
- `product_length_cm`. Longitud del producto en centímetros.
- `product_height_cm`. Altura del producto en centímetros.
- `product_width_cm`. Anchura del product en centímetros.

La principal característica de este archivo es que nos aporta la categoría del producto, lo que permite un futuro análisis en función de este dato.

3.1.5. olist_order_payments_dataset.csv

Este fichero contiene la información relativa a los pagos asociados a los pedidos realizados.

Su contenido viene dividido en las siguientes columnas:

- `order_id`. Identificador único del pedido al que corresponde el pago.
- `payment_sequential`. Número que identifica cada pago dentro de un mismo pedido. Es decir, un cliente puede pagar un pedido con más de un método de pago y, si lo hace, se incrementa el número.
- `payment_type`. Tipo de método de pago utilizado (credit card, boleto, voucher, entre otros).
- `payment_installments`. Número de cuotas en las que se ha dividido el pago.
- `payment_value`. Importe total del pago realizado.

Esta información es clave para el análisis financiero del negocio, permitiendo realizar distintos análisis como el comportamiento de pago de los clientes y los hábitos mas frecuentes.

3.1.6. `olist_order_reviews_dataset.csv`

Este fichero contiene la información sobre las valoraciones realizadas por los clientes en las encuestas de satisfacción. Cuando el cliente recibe el pedido o llega el día de la fecha estimada de recibirlo, el cliente recibe una encuesta por email donde pone su valoración.

Su contenido viene dividido en las siguientes columnas:

- `review_id`. Identificador único de la valoración.
- `order_id`. Identificador único del pedido al que está asociada la valoración.
- `review_score`. Puntuación otorgada por el cliente en una escala del 1 al 5.
- `review_comment_title`. Título del comentario de la valoración.
- `review_comment_message`. Contenido del comentario realizado por el cliente.
- `review_creation_date`. Fecha en la que se creó la valoración.
- `review_answer_timestamp`. Fecha y hora en la que se respondió la valoración.

Esta información es esencial para identificar patrones que influyen en la satisfacción del cliente y apoyar a la toma de decisiones para mejorar su experiencia.

3.1.7. `product_category_name_translation.csv`

Este fichero proporciona la traducción al inglés de la categoría del producto.

Su contenido viene dividido en las siguientes columnas:

- `product_category_name`. Nombre de la categoría en portugués.
- `product_category_name_english`. Traducción al inglés de la categoría.

Este fichero se utiliza para facilitar los posteriores análisis disponiendo de la información en inglés en lugar de en portugués, como viene dado por defecto.

3.1.8. Archivos CSV descartados

Durante la planificación del proyecto se ha decidido descartar dos archivos CSV de los 9 existentes en el dataset por distintas razones.

En primer lugar, `olist_sellers_dataset.csv` ha sido descartado debido a que la información relativa a la localización de los vendedores no ha sido considerada de interés en los análisis del proyecto y, además, la información del vendedor ya queda parcialmente representada a través del campo `seller_id` incluido en el archivo `olist_order_items_dataset.csv`, por lo que crear una dimensión adicional incrementaría la complejidad del modelo sin aportar un valor significativo.

Por otro lado, el otro archivo descartado ha sido `olist_geolocation_dataset.csv`, debido especialmente a su alta granularidad, ya que ofrece coordenadas geográficas a nivel de código postal. Además,

el archivo contiene más de un millón de registros, convirtiéndolo en un dato de escasa utilidad para los objetivos de este proyecto.

Estas decisiones permiten simplificar el modelo dimensional, adaptando el modelo a los principales objetivos de análisis del proyecto.

3.2. Casos de uso

El sistema desarrollado a partir de estos archivos está orientado a dar respuesta a un conjunto de necesidades de análisis de negocio identificadas. En este punto se describen los principales casos de uso que el sistema cubre, agrupados según el perfil de usuario al que van dirigidos.

Se identifican dos perfiles de usuario principales: el gestor de negocio, interesado en obtener una visión global del rendimiento de la plataforma sin necesidad de conocimientos técnicos, y el analista de datos, que requiere explorar la información con mayor profundidad para identificar patrones y oportunidades de mejora.

3.2.1. CU-01. Análisis del rendimiento de ventas

- **Actor principal.** Gestor de negocio.
- **Descripción.** El gestor necesita disponer de una visión consolidada del volumen de ventas generado por la plataforma en un periodo determinado. Esta visión debe permitir conocer el número total de pedidos, el importe total facturado y el coste de envío asociado, con posibilidad de filtrar la información por categoría de producto y región geográfica.
- **Necesidad que cubre.** En el contexto del problema planteado, los datos de ventas se encuentran distribuidos entre múltiples archivos, lo que dificulta su consulta directa. Este caso de uso justifica la necesidad de integrar la información de pedidos, productos y clientes en una única tabla de hechos, `fact_sales`, que permita responder de forma eficiente a preguntas como cuáles son las categorías más vendidas o cómo ha evolucionado la facturación a lo largo del tiempo.
- **Resultado esperado.** El gestor obtiene una visión clara y actualizada del rendimiento comercial del negocio, identificando los productos y mercados más rentables, así como posibles tendencias temporales en los patrones de compra.

3.2.2. CU-02. Análisis del comportamiento de pago

- **Actor principal.** Analista de datos.
- **Descripción.** El analista necesita estudiar en detalle cómo realizan los pagos los clientes. Esto incluye conocer qué métodos de pago son los más utilizados, cuántas cuotas emplean de media los clientes que financian sus compras y cuál es el importe medio por transacción. Además, el analista debe poder cruzar esta información con variables temporales para identificar posibles patrones estacionales en los comportamientos de pago.

- **Necesidad que cubre.** En el dataset original, los datos de pago se encuentran aislados de la información temporal del pedido, lo que dificulta realizar análisis evolutivos o comparativas temporales. Este caso de uso justifica la construcción de la tabla de hechos `fact_payment`, que integra cada registro de pago con su fecha de compra correspondiente mediante la clave `date_sk`, permitiendo desarrollar análisis temporales que no serían posibles utilizando los datos originales sin transformar.
- **Resultado esperado.** El analista obtiene una caracterización detallada del comportamiento de pago de los clientes, identificando los métodos de pago preferidos, los patrones de financiación y posibles diferencias en el importe medio según el tipo de pago utilizado.

3.2.3. CU-03. Análisis de la satisfacción del cliente

- **Actor principal.** Gestor de negocio.
- **Descripción.** El gestor necesita monitorizar continuamente la satisfacción de los clientes con los pedidos recibidos. Para ello, debe poder consultar las puntuaciones medias de las valoraciones registradas, su evolución temporal y su distribución por categoría de producto. Además, debe poder identificar qué proporción de valoraciones incorpora comentarios, ya que esto puede reflejar un mayor grado de insatisfacción o una experiencia de compra especialmente relevante para el cliente.
- **Necesidad que cubre.** En el dataset original, las valoraciones de los clientes se almacenan en un archivo independiente, sin conexión directa con la dimensión temporal ni con los productos asociados al pedido. Este caso de uso justifica la construcción de la tabla de hechos `fact_review`, que incorpora una clave temporal y un indicador binario derivado del proceso ETL para señalar la presencia de comentarios escritos. Gracias a esta integración, es posible realizar análisis más avanzados que la simple consulta de puntuaciones individuales.
- **Resultado esperado.** El gestor obtiene una visión clara y estructurada del nivel de satisfacción de los clientes, pudiendo identificar tendencias temporales, categorías con peores valoraciones y factores operativos que influyen en la experiencia de compra, como el retraso en la entrega o el coste de envío del pedido.

3.2.4. CU-04. Segmentación geográfica de clientes

- **Actor principal.** Gestor de negocio.
- **Descripción.** El gestor necesita conocer cómo se distribuyen geográficamente los clientes, tanto en términos de volumen como de comportamiento de compra. Esto incluye identificar los estados y ciudades con mayor número de clientes registrados, el volumen de pedidos generado por cada región y el importe medio de compra por zona geográfica. Esta información resulta esencial para orientar decisiones logísticas, campañas de captación y estrategias de fidelización diferenciadas según el mercado.

- **Necesidad que cubre.** En el dataset original, la información geográfica está vinculada al identificador de cliente, que no es único para cada cliente real, lo que dificulta su análisis directo. Este caso de uso justifica la deduplicación realizada durante el proceso ETL de la dimensión `dim_customer`, donde se consolida un único registro por cliente real, garantizando que los análisis geográficos no queden distorsionados por clientes con múltiples identificadores.
- **Resultado esperado.** El gestor obtiene una visión geográfica del negocio que le permite identificar los mercados con mayor actividad, detectar regiones con potencial de crecimiento y comparar el comportamiento de compra entre distintas zonas del país.

3.2.5. CU-05. Segmentación de clientes por comportamiento

- **Actor principal.** Analista de datos.
- **Descripción.** El analista necesita agrupar a los clientes en segmentos homogéneos a partir de su historial de compras, con el objetivo de identificar perfiles diferenciados que aporten valor al negocio. Para ello, se aplican técnicas de clustering sobre variables derivadas del comportamiento de compra, como el importe total gastado, el número de pedidos, el retraso medio de entrega o la valoración media de las reseñas. El resultado es una clasificación de los clientes en grupos con características similares que permite orientar estrategias comerciales específicas para cada perfil.
- **Necesidad que cubre.** Este caso de uso va más allá de la consulta directa de datos y requiere la aplicación de técnicas de análisis avanzado sobre la información contenida en el Data Warehouse. La disponibilidad de un modelo dimensional correctamente estructurado constituye un prerequisite para construir las variables agregadas necesarias para el análisis, lo que justifica la importancia de disponer de una arquitectura de BI sólida como base para los modelos analíticos.
- **Resultado esperado.** El analista obtiene una segmentación de clientes en grupos interpretables, acompañada de una descripción de las características principales de cada segmento. Esta información permite al gestor diseñar estrategias de retención, fidelización y captación adaptadas a cada perfil de cliente.

3.2.6. CU-06. Predicción de la satisfacción del cliente

- **Actor principal.** Analista de datos.
- **Descripción.** El analista necesita anticipar la valoración que un cliente asignará a un pedido a partir de características conocidas previamente, como el número de pedidos realizados, el gasto total, el coste medio de envío o el retraso medio de entrega. Para ello, se entrenan modelos predictivos utilizando como variable objetivo la puntuación media registrada en las reseñas de los clientes, con el objetivo de identificar qué factores influyen en mayor medida sobre la satisfacción y estimar el riesgo de obtener valoraciones negativas.
- **Necesidad que cubre.** Este caso de uso requiere combinar información procedente de distintas tablas del modelo dimensional, cruzando datos de ventas, pagos y reseñas para construir las

variables necesarias para el entrenamiento de los modelos. La integración de todas estas fuentes dentro de un único Data Warehouse es lo que permite desarrollar análisis predictivos consistentes, poniendo de manifiesto el valor de la arquitectura implementada más allá de la simple visualización de datos.

- **Resultado esperado.** El analista obtiene modelos predictivos capaces de estimar la satisfacción del cliente antes de que la valoración se produzca, identificando además los factores operativos con mayor impacto en la experiencia de compra. Esta información proporciona a los gestores una herramienta de apoyo a la toma de decisiones orientada a mejorar la calidad del servicio y reducir posibles situaciones de insatisfacción.

4. Diseño de la solución

4.1. Diseño de la arquitectura del sistema

El sistema de Business Intelligence desarrollado ha decidido organizarse en tres capas funcionales, las cuales pueden ser observada en la imagen. Estas capas son, en primer lugar, la capa que contiene los archivos csv del sistema origen, denominada Data Source. En segundo lugar, se encuentra el área de datos, denominada Data Area, que contiene la base de datos almacenada en torno al modelo dimensional. Por último, tenemos la capa de acceso y visualización, la cual contiene las representaciones visuales del análisis realizado de los datos. Cabe destacar también que, a diferencia de como suele representarse en las estructuras clásicas, en este diseño se ha prescindido de un data staging área, decisión la cual se desarrolla al final de este apartado.

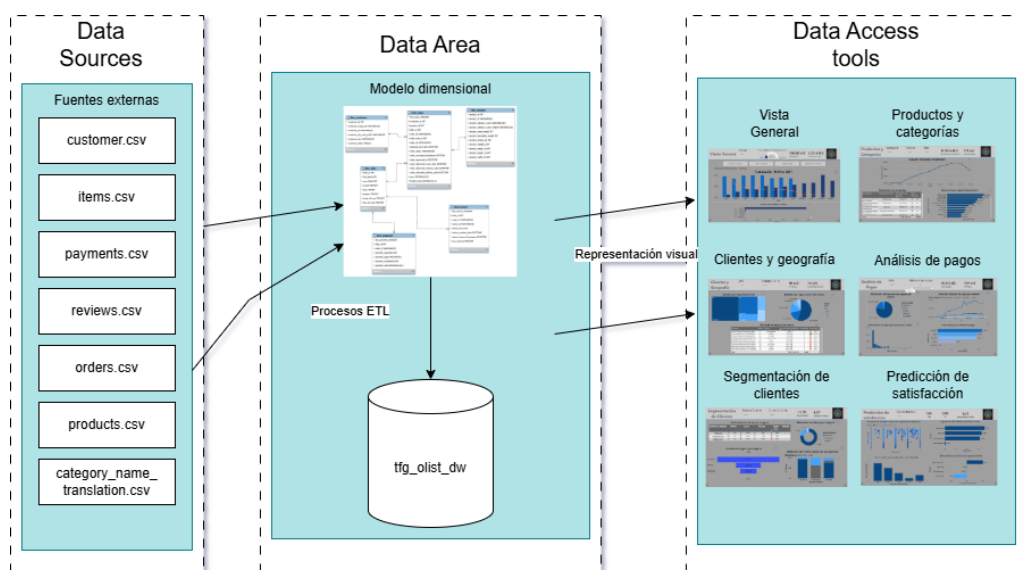


Figura 13: Arquitectura por capas del sistema desarrollado. (Elaboración propia)

4.1.1. Data Sources

Es el primer área del sistema, la base de el sistema desarrollado, que contiene toda la información del sistema origen. Está compuesto por siete archivos CSV que componen el Brazilian E-Commerce Public Dataset de Olist, obtenidos de la plataforma Kaggle. Estos archivos contienen el ciclo de vida de los pedidos realizados entre 2016 y 2018 en mercado de comercio electrónico brasileño, incluyendo todo tipo de información al respecto.

Al tratarse de un dataset de público acceso y estático, las fuentes de datos no requieren de ningún mecanismo de conexión o extracción, sino que se encuentran almacenados en el sistema de ficheros local y se accede a ellos directamente desde los scripts de los procesos ETL. A partir de estas características como la naturaleza estática del sistema y la ausencia de cargas es por lo que se decide la ausencia de implementar un área de staging, cómo se explicará más adelante.

4.1.2. Data Area

El Data Area o área de datos es el núcleo de este sistema, completamente imprescindible, y está formado por dos elementos clave con una gran relación entre ellos: el proceso ETL, implementado a través de Python y el Data Warehouse, que contiene el modelo dimensional diseñado, construido sobre MySQL. Estos elementos colaboran para transformar los datos previamente mencionados en un sistema adaptado para el análisis y la toma de decisiones.

En primer lugar, el proceso ETL o extracción, transformación y carga se ha implementado mediante 6 scripts independientes, es decir, uno por cada tabla existente en el modelo dimensional. Cada script, cada uno orientado a sus necesidades, se ha diseñado con la misma estructura dividida en tres fases. En la fase de extracción, los archivos csv almacenados localmente se leen mediante librerías pandas, que proporciona las herramientas necesarias para el tratamiento de grandes volúmenes de datos. En la fase de transformación se aplican las operaciones necesarias para el archivo, como limpieza de espacios en blanco, eliminación de duplicados o gestión de nulos entre otros, lo que será explicado con más detalle en el apartado de implementación. Finalmente, en la fase de carga, los datos transformados se insertan al Data Warehouse mediante operaciones de inserción masiva, con una previa eliminación de los registros anteriores para evitar posibles problemas.

En segundo lugar, el Data Warehouse se ha diseñado sobre MySQL bajo el nombre `tfg_olist_dw`, siguiendo un modelo dimensional de tipo esquema en galaxia que integra tres procesos de negocio diferenciados: ventas, pagos y reseñas. Este modelo está compuesto por tres tablas de hechos, que corresponden a los procesos de negocio (`fact_sales`, `fact_payment` y `fact_review`) y tres dimensiones compartidas (`dim_customer`, `dim_product` y `dim_date`). Las tablas de hechos se relacionan con las dimensiones a través de claves sustitutas, que se han decidido utilizar para desacoplar el sistema de los identificadores originales del dataset y mejorar la consistencia del modelo analítico. Asimismo, las distintas tablas de hechos se encuentran relacionadas de forma indirecta mediante el identificador de pedido (`order_id`), lo que permite conectar los distintos procesos de negocio sin necesidad de duplicar dimensiones en cada una de ellas, manteniendo la coherencia del modelo.

4.1.3. Data Access Tools

Para este proyecto se ha decidido que la capa de acceso y visualización se diseñe utilizando Microsoft Power BI Desktop, que actúa como interfaz analítica sobre el Data Warehouse. Power BI se conecta a la base de datos de MySQL a través de su conector nativo, cargando toda la información para poder realizar su análisis y representarlo visualmente. La representación visual se organiza en cinco páginas, cada una orientada a un proceso de negocio o perspectiva de análisis diferente. Estas páginas se han definido como: Visión general, Productos y categorías, Clientes y geografía, Análisis de pago y Segmentación de clientes.

Cada página contiene información a través de KPIs y gráficos de distintas características. Además, dispone de filtros interactivos que afectan a las visualizaciones, permitiendo al usuario analizar el sistema desde distintas perspectivas sin necesidad de conocimientos técnicos, al mismo tiempo que permite a los usuarios más técnicos realizar análisis con mayor profundidad. Finalmente, la navegación entre las distintas páginas se facilita mediante botones visibles en todas las vistas, y cada gráfico

contiene un botón de información para poder obtener una información mas detallada sobre lo que está siendo analizado, dotando así al informe de una experiencia de uso coherente y fluida.

4.1.4. Justificación de la ausencia de Área de Staging

En arquitecturas de Data Warehouse empresariales es muy común incorporar un área de staging intermedia entre las fuentes de datos originales y el Data Warehouse definitivo. Esta zona permite almacenar los datos brutos sin transformar antes de realizar los procesos ETL, lo que permite, entre otras cosas, la recuperación de datos en casos de fallo.

A la hora de analizar el diseño de este proyecto, se ha optado por la ausencia de esta capa por las siguientes razones:

- **Fuente de datos estática y única.** El dataset de Olist utilizado es un conjunto de archivos CSV cerrado, que no va a experimentar actualizaciones. En un entorno con cargas diarias, el staging sería necesario para aislar los nuevos datos entrantes de los que ya están procesados.
- **ETL en memoria como staging funcional.** A pesar de no tener el peso de un área de staging, los DataFrames de `pandas` actúan como una capa parecida en memoria. Almacenan temporalmente los datos crudos extraídos de los archivos antes de aplicar los procesos ETL. Por tanto, en el proyecto este staging en memoria es funcionalmente equivalente al staging persistido en base de datos.
- **Simplicidad arquitectónica justificada.** En el contexto de este proyecto con una fuente de origen única y con carga completa, añadir una zona de staging habría incrementado la complejidad del sistema, al requerir una base de datos extra y scripts de sincronización, sin aportar un valor adicional. La arquitectura resultante de tres capas es más sencilla, fácil de mantener y suficiente para el objetivo de este proyecto.

A pesar de estas razones cabe destacar que, en futuras actualizaciones, esta arquitectura podría ser adapta para permitir el análisis de nuevos datos y cargas incrementales sin necesidad de rediseñar todo el proyecto.

4.2. Diseño del modelo dimensional

En este apartado se explican las decisiones de diseño en el modelado dimensional.

4.2.1. Metodología de diseño: Enfoque Kimball

El diseño del modelo dimensional se ha realizado siguiendo la metodología propuesta por Ralph Kimball, basada en un enfoque bottom-up y orientada al análisis del negocio. Esta metodología, como se ha mencionado anteriormente, se basa en el uso de modelos dimensionales compuestos por tablas de hechos y dimensiones, con el objetivo de facilitar la consulta y explotación de los datos [20].

Entre los principios claves definidos por esta metodología, podemos destacar los siguientes que han sido utilizados en el proyecto:

- La definición explícita de la granularidad de cada tabla de hechos antes de su diseño.
- El uso del esquema en estrella como base conceptual para facilitar las consultas analíticas. Cabe destacar que el modelo en estrella es la base, mientras que la solución proporcionada esta compuesta por varios modelos en estrella al existir mas de una tabla de hechos.
- La separación existente entre hechos, eventos medibles y cuantificables, como los pagos o las ventas, y dimensiones, que aportan contexto descriptivo a estos eventos, como pueden ser las fechas o los productos.
- La utilización de claves sustitutas en todas las dimensiones para desacoplar del sistema origen, cuyo uso será argumentado en un punto posterior.

Este enfoque y estas decisiones de diseño han permitido construir un modelo flexible, fácilmente escalable y preparado para su análisis.

4.2.2. Tipo de esquema: Galaxy Schema

Aunque como ya ha sido mencionado el proyecto ha sido desarrollado tomando como base el modelo en estrella propuesto por Kimball, la existencia de tres procesos de negocio diferenciados para el análisis, que son ventas, pagos y reseñas, ha producido que se adapte a un modelo con múltiples tablas de hechos que comparten dimensiones comunes. Este tipo de esquema recibe el nombre de esquema en galaxia (Galaxy Schema) o constelación de hechos.

La utilización de este enfoque permite analizar cada proceso de negocio de forma independiente, al mismo tiempo que mantiene una visión global mediante el uso de dimensiones compartidas. De esta manera se evita la duplicidad de la información y se garantiza la coherencia en el análisis, facilitando la comparación entre indicadores procedentes de distintas tablas de hechos.

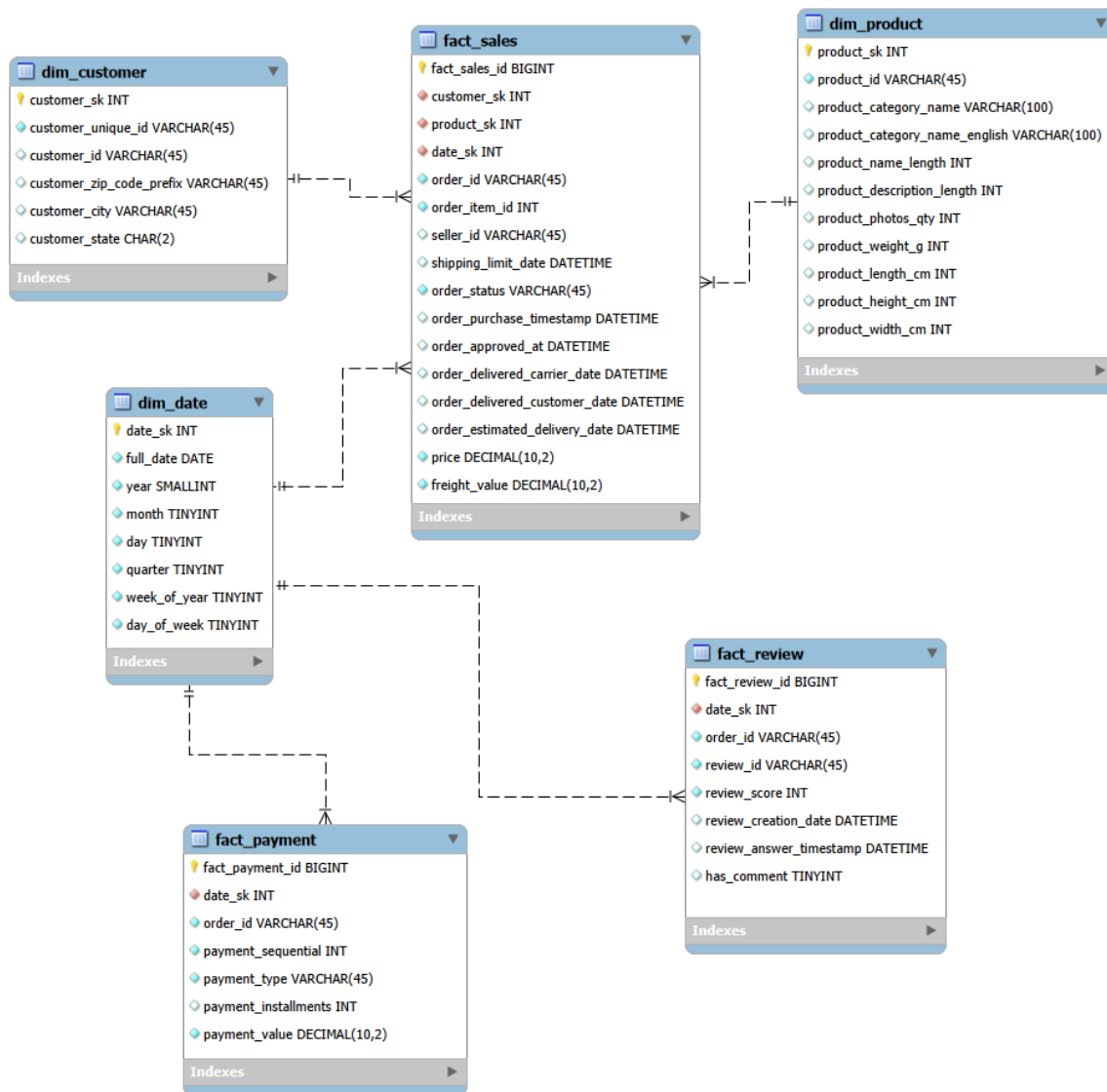


Figura 14: Modelo dimensional del sistema basado en un esquema en galaxia. (Elaboración propia)

4.2.3. Resumen de tablas del modelo

El modelo dimensional está compuesto por seis tablas: tres dimensiones y 3 hechos. En la siguiente tabla se resume la información correspondiente a cada una:

Tabla	Tipo	Granularidad	Descripción
<code>dim_customer</code>	Dimensión	-	Tabla de clientes, contiene un registro por cliente único (<code>customer_unique_id</code>).
<code>dim_product</code>	Dimensión	-	Tabla de productos, contiene un registro por producto del catálogo.
<code>dim_date</code>	Dimensión	-	Tabla de fechas, contiene un registro por día del periodo 2016–2018.
<code>fact_sales</code>	Hecho	Línea de pedido	Tabla de ventas, contiene un registro por producto dentro de un pedido.
<code>fact_payment</code>	Hecho	Pago de pedido	Tabla de pagos, contiene un registro por pago secuencial de un pedido.
<code>fact_review</code>	Hecho	Reseña	Tabla de reseñas, contiene un registro por valoración de un pedido.

Cuadro 1: Resumen de tablas del modelo dimensional

4.2.4. Decisiones generales de diseño

Durante el diseño del modelo se han adoptado las siguientes decisiones clave:

- Se consideran ventas válidas únicamente aquellos pedidos cuyo estado se encuentra en **delivered**, garantizando que los análisis de facturación se basen únicamente en pagos completados.
- La fecha principal de análisis es la fecha de compra (`order_purchase_timestamp`), a partir de la cual deriva la clave de fecha utilizada en las tablas de hechos. Esto permite que todos los análisis se realicen en torno a una misma dimensión temporal.
- La granularidad del hecho principal se ha definido como una línea de pedido, lo que significa que hay un registro por cada producto dentro de un pedido, permitiendo de esta manera calcular métricas a distintos niveles como cliente, producto o pedido.
- El modelo se desacopla de los identificadores del sistema origen mediante el uso de claves sustitutas en todas las dimensiones. A pesar de esto, las claves originales del dataset se almacenan como atributos para mantener la trazabilidad.
- En cuanto a la disposición del modelo, podemos observar que las tablas tienen una conexión indirecta a través del atributo `order_id`, permitiendo relacionar los procesos de ventas, pagos y valoraciones sin necesidad de duplicar dimensiones en cada una. Esta decisión de diseño permite

que se mantenga la independencia de cada hecho al mismo tiempo que se reduce la redundancia, garantizando la coherencia analítica. Además, cabe destacar que todas las tablas comparten la dimensión temporal, facilitando el análisis desde la perspectiva temporal.

4.2.5. Uso de claves sustitutas (Surrogate Keys)

Aunque el proyecto integra datos procedentes de una única fuente, el dataset de Olist, se ha optado por el uso de claves sustitutas (SK) como claves primarias en las dimensiones.

Esta decisión de diseño se justifica por las siguientes razones:

- Permiten desacoplar el modelo analítico del sistema origen, evitando las dependencias directas a los identificadores originales del dataset.
- Facilitan la homogeneidad del diseño, empleando el mismo patrón de clave primaria en todas las tablas de dimensiones.
- Mejoran la claridad conceptual del Data Warehouse, diferenciando explícitamente entre identificadores de negocio y claves internas del modelo.
- Permiten gestionar de forma adecuada los cambios en las dimensiones a lo largo del tiempo (Slowly Changing Dimensions), manteniendo el histórico de los datos sin afectar a la integridad del modelo.
- Preparan el modelo para una posible integración futura de nuevas fuentes de datos sin necesidad de rediseñar las relaciones existentes.

4.2.6. Diseño tablas dimensión

Atributo	Tipo	Restricciones	Descripción
customer_sk	INT	PK, NOT NULL, AI	Clave sustituta de la dimensión. Identifica de forma única cada registro del cliente dentro del Data Warehouse.
customer_unique_id	VARCHAR(45)	NOT NULL, UNIQUE	Identificador único del cliente en el sistema origen. Se conserva para trazabilidad.
customer_id	VARCHAR(45)	NOT NULL	Identificador del cliente asociado al pedido en el sistema origen.
customer_zip_code_prefix	VARCHAR(45)	NULL	Código postal del cliente.
customer_city	VARCHAR(45)	NULL	Ciudad del cliente.
customer_state	CHAR(2)	NULL	Estado del cliente.

Cuadro 2: Tabla dim_customer

Atributo	Tipo	Restricciones	Descripción
product_sk	INT	PK, NOT NULL, AI	Clave sustituta de la dimensión. Identifica de forma única cada producto en el modelo analítico.
product_id	VARCHAR(45)	NOT NULL, UNIQUE	Identificador del producto en el sistema origen. Se conserva para trazabilidad.
product_category_name	VARCHAR(100)	NULL	Categoría original del producto.
product_category_name_english	VARCHAR(100)	NULL	Categoría del producto traducida al inglés. Facilita el análisis.
product_name_length	INT	NULL	Longitud del nombre del producto.
product_description_length	INT	NULL	Longitud de la descripción del producto.
product_photos_qty	INT	NULL	Número de fotografías asociadas al producto.
product_weight_g	INT	NULL	Peso del producto en gramos.
product_length_cm	INT	NULL	Longitud del producto en centímetros.
product_height_cm	INT	NULL	Altura del producto en centímetros.
product_width_cm	INT	NULL	Anchura del producto en centímetros.

Cuadro 3: Tabla dim_product

Atributo	Tipo	Restricciones	Descripción
date_sk	INT	PK, NOT NULL	Clave sustituta de la dimensión temporal. Se genera en formato numérico para facilitar uniones y agregaciones.
full_date	DATE	UNIQUE, NOT NULL	Fecha completa asociada a la clave temporal.
year	SMALLINT	NOT NULL	Año de la fecha.
month	TINYINT	NOT NULL	Mes de la fecha.
day	TINYINT	NOT NULL	Día del mes.
quarter	TINYINT	NOT NULL	Trimestre del año.
week_of_year	TINYINT	NOT NULL	Semana del año.
day_of_week	TINYINT	NOT NULL	Día de la semana.

Cuadro 4: Tabla dim_date

4.2.7. Diseño tablas hecho

Atributo	Tipo	Restricciones	Descripción
fact_sales_id	BIGINT	PK, NOT NULL, UNIQUE, AI	Clave sustituta de la tabla de ventas. Identifica de forma única cada línea de venta.
customer_sk	INT	FK, NOT NULL	Clave foránea hacia dim_customer . Permite analizar ventas por cliente y localización.
product_sk	INT	FK, NOT NULL	Clave foránea hacia dim_product . Permite analizar ventas por producto y categoría.
date_sk	INT	FK, NOT NULL	Clave foránea hacia dim_date . Permite un análisis temporal de las ventas.
order_id	VARCHAR(45)	NOT NULL	Identificador del pedido.
order_item_id	INT	NOT NULL	Identificador del artículo dentro del pedido.
seller_id	VARCHAR(45)	NULL	Identificador del vendedor.
shipping_limit_date	DATETIME	NULL	Fecha límite del envío del producto.
order_status	VARCHAR(45)	NOT NULL	Estado del pedido.
order_purchase_timestamp	DATETIME	NOT NULL	Fecha y hora de compra del pedido.
order_approved_at	DATETIME	NULL	Fecha de aprobación del pedido. Puede ser nula si no consta en origen.
order_delivered_carrier_date	DATETIME	NULL	Fecha de entrega del pedido al transportista.
order_delivered_customer_date	DATETIME	NULL	Fecha de entrega al cliente.
order_estimated_delivery_date	DATETIME	NULL	Fecha estimada de entrega al cliente.
price	DECIMAL(10,2)	NOT NULL	Importe del producto vendido.
freight_value	DECIMAL(10,2)	NOT NULL	Coste de envío del producto.

Cuadro 5: Tabla **fact_sales**

Atributo	Tipo	Restricciones	Descripción
fact_payment_id	BIGINT	PK, NOT NULL, AI	Clave sustituta de la tabla de hechos de pagos. Identifica de forma única cada registro de pago.
date_sk	INT	FK, NOT NULL	Clave foránea hacia dim_date . Permite analizar los pagos desde un punto de vista temporal.
order_id	VARCHAR(45)	NOT NULL	Identificador del pedido asociado al pago.
payment_sequential	INT	NOT NULL	Secuencia del pago dentro de un mismo pedido.
payment_type	VARCHAR(45)	NOT NULL	Método de pago utilizado por el cliente.
payment_installments	INT	NOT NULL	Número de cuotas del pago.
payment_value	DECIMAL(10,2)	NOT NULL	Importe del pago.

Cuadro 6: Tabla **fact_payment**

Atributo	Tipo	Restricciones	Descripción
fact_review_id	BIGINT	PK, NOT NULL, AI	Clave sustituta de la tabla de hechos de valoraciones. Identifica de forma única cada valoración.
date_sk	INT	FK, NOT NULL	Clave foránea hacia dim_date . Permite analizar las valoraciones desde un punto de vista temporal.
order_id	VARCHAR(45)	NOT NULL	Identificador del pedido valorado.
review_id	VARCHAR(45)	NOT NULL	Identificador de la valoración en el sistema origen.
review_score	INT	NOT NULL	Puntuación otorgada por el cliente del 1 al 5.
review_creation_date	DATETIME	NOT NULL	Fecha de creación de la valoración.
review_answer_timestamp	DATETIME	NULL	Fecha de respuesta de la valoración.
has_comment	TINYINT	NOT NULL	Indicador 0/1 que refleja si la valoración incluye comentario.

Cuadro 7: Tabla **fact_review**

4.3. Diseño de los dashboards

En este apartado se describe el diseño previo a la implementación de los dashboards en Power BI. Se recogen las decisiones que se han tomado respecto a la estructura general del informe, relativas a factores como la disposición de los elementos en cada página, los colores utilizados y el sistema de navegación de páginas, entre otros.

4.3.1. Boceto previo

Con carácter previo a la implementación de las páginas, se elaboró un boceto general que recoge la disposición estructural de los elementos comunes a todas las páginas. Cabe destacar que es un

boceto previo por lo que cada página posteriormente se adapta a sus necesidades. Este boceto sirvió como referencia durante la fase de implementación, permitiendo tomar decisiones de diseño de forma anticipada y garantizando la coherencia en el resultado final.

El boceto define una plantilla común aplicable a las seis páginas del informe, estableciendo las siguientes zonas funcionales: una franja superior destinada a los indicadores claves de negocio (KPIs) y los filtros interactivos, una zona central para las representaciones visuales de los análisis, en la que cada página se adapta con el número de gráficos o representaciones que se considera necesario, y una serie de elementos interactivos para facilitar la navegación y comprensión del sistema. Esta estructura permite al usuario que pueda orientarse con facilidad, al encontrar siempre los elementos en la misma posición.



Figura 15: Boceto inicial de la estructura de los cuadros de mando. (Elaboración propia)

4.3.2. Principios de diseño aplicados

El diseño de los distintos dashboards desarrollados se ha guiado por los siguientes principios, aplicados en todas las páginas [44]:

- **Claridad y legibilidad.** En primer lugar, los dashboards se separan por categorías, de manera que los usuarios pueden guiarse con claridad sobre las distintas pantallas. Además, cada visualización en los informes transmite un único mensaje claro, prefiriendo múltiples visualizaciones simples a una sola visualización más compleja de interpretar.
- **Jerarquía visual.** Se aplican los principios de jerarquía visual en los que la parte superior de un dashboard capta la atención principal de los usuarios. Por tanto, los indicadores principales (KPIs de negocio) se sitúan en la parte superior, con tipografía de mayor tamaño y peso visual, al igual que el nombre de la página y los filtros disponibles, permitiendo que el usuario identifique de inmediato los valores clave antes de analizar en detalle las visualizaciones inferiores.

- **Consistencia.** Se aplica el mismo diseño en todas las páginas, teniendo en cuenta factores como la paleta de colores, la tipografía o la disposición de los elementos. Esto reduce la carga para el usuario, que no necesita reaprender la interfaz cada vez que cambia la pantalla.
- **Interactividad controlada.** En cada página se incorporan únicamente filtros relevantes para el contenido, no exponiendo a los usuarios a opciones que no modifiquen las visualizaciones existentes. Cabe destacar que, principalmente en el caso de la representación de las técnicas de análisis avanzado, se presentan algunos gráficos fijos que representan datos no variables. Los filtros, como se ha indicado, se encuentran en la parte superior para que su peso visual se vea potenciado.
- **Orientación al usuario de negocio.** El informe está diseñado para ser entendido por todo tipo de usuario, tanto técnico como no técnico. Los títulos son descriptivos y en lenguaje natural, los valores vienen detallados, entre otras cosas. Además de esto, se incorporan iconos de información en cada gráfico para permitir acceder a información más detallada.

4.3.3. Identidad visual

La identidad visual del informe se definió en la fase de diseño previo a la implementación, y se mantiene en todo el informe.

En primer lugar, respecto a la paleta de colores, se han utilizado diferentes tipos dependiendo de los objetivos. En cuanto a los colores de los gráficos, se basa en una escala de diferentes tonos de azules, ya que es el color asociado convencionalmente a los entornos de negocio y análisis de datos, ofreciendo buen contraste sobre el fondo de la imagen y permitiendo distinguir representaciones de datos mediante variaciones de saturación e intensidad.

Por otro parte, para el fondo se designa una variación del color gris, la parte superior una parte mas clara y, la inferior, un poco mas oscura, lo que potencia el contraste frente a los gráficos de color azul. Además, estos colores reducen la fatiga visual en sesiones de análisis prolongadas y aportan un aspecto profesional.

Por último, se reservan los colores rojo, amarillo y verde como indicadores, ya que son unos valores visuales comúnmente utilizados y que el usuario asocia fácilmente con su significado. También se utiliza el naranja, como caso puntual, para representar la diferencia de facturación entre dos gráficos [45].

En cuanto a la tipografía, se utiliza una misma familia Segoe UI, variando este tipo junto Segoe UI Semibold, utilizando variaciones de tamaño y peso para establecer la jerarquía visual. De esta manera, se designa la tipografía semibold para títulos de gráficos, filtros y KPIs, mientras que el uso normal se utiliza para valores con menor peso visual como valores o leyendas.

4.3.4. Selección de visualizaciones

Una de las decisiones de diseño más relevantes en este ámbito es la elección del tipo de visualización adecuado para cada dato y objetivo. Una elección inadecuada puede dificultar la interpretación por parte del usuario o incluso llevarle a conclusiones erróneas. La siguiente tabla recoge los tipos de visualización utilizados para cada informe y su correspondiente justificación:

Visión General		
Información elegida	Visualización elegida	Justificación
Comparativo temporal entre dos años	Gráfico de columnas agrupadas y de líneas	Permite comparar volúmenes absolutos y la tendencia relativa en un mismo gráfico.
Estado del pedido	Gráfico de barras agrupadas	Permite comparar de forma clara la distribución de pedidos según su estado, identificando rápidamente las categorías predominantes.

Cuadro 8: Justificación de las visualizaciones del dashboard de Visión General

Productos y Categorías		
Información elegida	Visualización elegida	Justificación
Evolución temporal de la facturación	Gráfico de líneas	Se selecciona un gráfico de líneas ya que permite representar de manera clave la evolución de una variable a lo largo del tiempo, facilitando la detección de tendencias y posibles alteraciones en el comportamiento.
Rendimiento de productos	Tabla con indicadores visuales	Se emplea una tabla debido a que permite incluir múltiples atributos asociados a un producto simultáneamente. Los indicadores visuales permiten identificar rápidamente la tendencia de los productos.
Facturación por categoría de producto	Gráfico de barras agrupadas	Se selecciona un gráfico de barras agrupadas porque permite calcular de forma clara la facturación entre categorías, facilitando la identificación de las categorías con mayor facturación. Además mejora la legibilidad de etiquetas contextuales largas.

Cuadro 9: Justificación de las visualizaciones del dashboard de Productos y Categorías

Clientes y Geografía		
Información elegida	Visualización elegida	Justificación
Facturación por estado	Treemap	Se selecciona un mapa de árbol ya que resulta innovador, alejándose de los típicos gráficos. También aporta una visión rápida de las regiones con mayor peso económico.
Distribución de clientes por estado	Gráfico circular	Se utiliza un gráfico circular para representar la proporción de clientes por estado, permitiendo comparar el peso relativo de cada categoría sobre el total.
Actividad de compra por cliente	Tabla con indicadores visuales	Se emplea una tabla con indicadores visuales para mostrar simultáneamente diferentes atributos asociados a cada cliente, facilitando la identificación de patrones. Los patrones visuales permiten detectar si un cliente es recurrente o no, factor importante para decisiones de negocio.

Cuadro 10: Justificación de las visualizaciones del dashboard de Clientes y Geografía

Análisis de pagos		
Información elegida	Visualización elegida	Justificación
Distribución del importe total pagado por método	Gráfico circular	Se selecciona un gráfico circular debido a que permite visualizar la proporción que representa cada método de pago sobre el importe total procesado.
Evolución temporal de pagos por método	Gráfico de líneas	Se emplea un gráfico de líneas múltiples para representar la evolución temporal de distintos métodos y comparar su comportamiento.
Distribución de pagos por número de cuotas	Gráfico de columnas apiladas	Se utiliza un gráfico de columnas apiladas que permite ver la frecuencia de aparición de los diferentes números de cuotas, identificando concentraciones y patrones de comportamiento.
Ticket medio por método de pago	Gráfico de barras agrupadas	Se utiliza un gráfico de barras agrupadas para comparar el valor medio del ticket de los diferentes métodos de pago e identificar diferencias significativas.

Cuadro 11: Justificación de las visualizaciones del dashboard de Análisis de pagos

Segmentación de clientes		
Información elegida	Visualización elegida	Justificación
Perfil medio de clientes por categoría	Tabla con indicadores visuales	Se emplea una tabla con indicadores visuales para mostrar visualmente distintas métricas asociadas a cada segmento en el proceso de clustering, permitiendo una comparación entre sectores.
Distribución de clientes por categoría	Gráfico de anillos	Se selecciona un gráfico de anillos ya que aporta una alternativa al gráfico circular y representa la proporción relativa de clientes pertenecientes a cada segmento de forma clara y compacta.
Contribución al gasto por categoría	Embudo	En este caso se utiliza un gráfico de embudo para representar la contribución relativa de cada segmento al gasto total, permitiendo visualizar de forma intuitiva la reducción progresiva entre categorías.
Distribución del nivel de satisfacción por segmento	Gráfico de columnas 100 % apiladas	Se utiliza un gráfico de columnas 100 % apiladas para comparar la composición porcentual de los niveles de satisfacción dentro de cada segmento.

Cuadro 12: Justificación de las visualizaciones del dashboard de Segmentación de clientes

Predicción de satisfacción		
Información elegida	Visualización elegida	Justificación
Precisión del modelo: valoración real frente a estimada	Gráfico de dispersión	Se utiliza un gráfico de dispersión para analizar la relación entre los valores reales y las predicciones del modelo, lo que permite evaluar visualmente su precisión.
Influencia de variables en la satisfacción (Random Forest)	Gráfico de barras apiladas	Se selecciona un gráfico de barras apiladas para representar la importancia relativa de cada variable en el modelo Random Forest, facilitando la identificación de los factores con mayor influencia.
Efecto de las variables en la regresión lineal	Gráfica de barras agrupadas	Se utiliza un gráfico de barras agrupadas para representar el sentido y magnitud de los coeficientes del modelo, diferenciando claramente impactos positivos y negativos.
Error medio según valoración real del cliente	Gráfico de columnas agrupadas	Se selecciona un gráfico de columnas agrupadas para comparar el error medio del modelo según el nivel de valoración real, facilitando la identificación de rangos donde la predicción presenta mayor desviación.

Cuadro 13: Justificación de las visualizaciones del dashboard de Predicción de satisfacción

Además de todas estas visualizaciones implementadas en los dashboards, también se incorporan otros elementos complementarios ya mencionados para facilitar la interpretación y mejorar la interacción.

En cuanto a los KPIs, se representan principalmente mediante tarjetas simples, las cuales muestran de forma directa el valor asociado a una métrica relevante, como puede ser la facturación total. Asimismo, en alguna ocasión también se ha optado por la idea de utilizar un medidor, que permite representar visualmente el valor alcanzado respecto a un rango objetivo o valor de referencia.

Por otro lado, para proporcionar capacidad de exploración y análisis dinámico se utilizan los filtros, representado con el nombre segmentación de datos. Su implementación puede adoptar distintos formatos, entre los que se han elegido los menús desplegables, utilizados para seleccionar valores concretos de una categoría, y los filtros de tipo entre, empleados para valores numéricos, como el número de cuotas. Estos últimos funcionan mediante un marcador deslizante que se mueve entre dos valores límite, permitiendo definir de forma visual el intervalo de valores sobre el que se realiza el análisis.

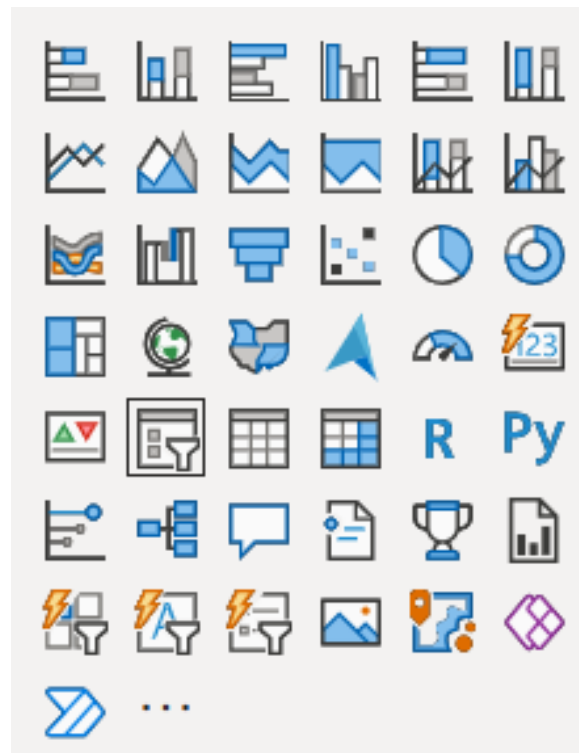


Figura 16: Visualizaciones disponibles en Power BI para la construcción de cuadros de mando.[46]

4.3.5. Sistema de navegación

El diseño de la navegación entre páginas se diseñó en la fase previa a la implementación, con el objetivo de que el usuario pueda moverse entre las distintas páginas del informe sin complejidad. Para ello se diseñó en primer lugar, en la página inicial, una barra de navegación superior con botones con el nombre de cada página que actúan como accesos directos. Además de este sistema, las posteriores páginas contienen un sistema basado en flechas, una situada en la esquina inferior izquierda y otra en la esquina inferior derecha, lo que permite al usuario navegar por el sistema con facilidad.

Esta decisión responde en primer lugar al principio de navegación no lineal, permitiendo acceder desde la primera página a todas sin necesidad de seguir un orden predeterminado. Por otra parte, también permite a los usuarios una navegación lineal adoptando el orden que se ha considerado adecuado en el diseño del informe. De esta manera, el usuario puede adoptar la técnica que mas le convenga sin necesidad de seguir unas pautas fijas.

5. Implementación de la solución desarrollada

En este punto, tras presentar la fase de diseño, se describe la implementación técnica de las distintas partes del sistema de Business Intelligence. Se aborda todo el proceso de implementación, desde el desarrollo de los scripts ETL sobre los datos del sistema origen hasta la representación visual de los datos.

5.1. Integración y preparación de datos (ETL)

Este apartado describe la implementación del proceso ETL que alimenta el Data Warehouse con los datos del dataset de Olist. El proceso se implementa en Python mediante scripts independientes, uno por cada tabla del modelo. En este apartado se va a explicar el desarrollo de cada script, pero cabe destacar un patrón común entre todos, la configuración. La contraseña de acceso a MySQL no se almacena en el código fuente sino que se lee en tiempo de ejecución desde la variable de entorno del sistema operativo `MYSQL_PASSWORD`, garantizando que las credenciales no queden expuestas en el repositorio del proyecto.

```
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": os.getenv("MYSQL_PASSWORD"),
    "database": "tfg_olist_dw",
    "port": 3306
}
```

Figura 17: Configuración de conexión entre el proceso ETL y el Data Warehouse. (Elaboración propia)

5.1.1. `load_dim_customer.py`

Este script carga la dimensión de clientes. El principal reto es que el dataset original contiene múltiples filas para un mismo cliente, ya que cada pedido genera un registro independiente con el mismo `customer_unique_id` pero distinto `customer_id`, que refleja el indicador de sesión de compra.

Tabla <code>load_dim_customer.py</code>	
Tabla destino	<code>dim_customer</code>
CSV de entrada	<code>olist_customers_dataset.csv</code>
Dependencias	Ninguna
Registros aprox.	96096 clientes únicos (99441 originales)

Cuadro 14: Resumen del script `load_dim_customer.py`

En primer lugar, en la fase de extracción se carga el archivo `olist_customers_dataset.csv` en un único DataFrame. Como se ha mencionado anteriormente, este archivo contiene una fila por cada pedido realizado y no por cada cliente único, por lo que un mismo cliente puede aparecer múltiples veces dentro del conjunto de datos.

```
CUSTOMERS_CSV = "olist_customers_dataset.csv"
df = pd.read_csv(CUSTOMERS_CSV)
```

Figura 18: Fragmento de código correspondiente a la fase de extracción de `dim_customer`. (Elaboración propia)

En cuanto a la transformación, el script aplica tres operaciones:

- **Limpieza de strings.** Los campos `customer_id`, `customer_unique_id`, `customer_city` y `customer_state` se convierten a tipo string y se eliminan los espacios en blanco extremos mediante `.astype(str).str.strip()` evitando inconsistencias y posibles duplicados invisibles durante el proceso posterior de deduplicación.
- **Deduplicación.** Se aplica `drop_duplicates()` sobre el atributo `customer_unique_id`, conservando únicamente la primera aparición de cada cliente real. Esta operación reduce el DataFrame original manteniendo un único registro por cliente y eliminando filas adicionales asociadas a pedidos posteriores realizados por el mismo usuario.
- **Selección de columnas.** Se conservan únicamente los atributos relevantes que formarán parte de la dimensión, seleccionando los campos que posteriormente serán cargados en la tabla `dim_customer`.

```
# =====
# LIMPIEZA BÁSICA
# =====

for col in ["customer_id", "customer_unique_id", "customer_city", "customer_state"]:
    df[col] = df[col].astype(str).str.strip()

# =====
# DEDUPLICAR PARA DIMENSIÓN
# =====

df_dim = df.drop_duplicates(subset=["customer_unique_id"], keep="first").copy()

# =====
# SELECCIONAR COLUMNAS QUE CARGAMOS EN dim_customer
# =====
df_dim = df_dim[[
    "customer_unique_id",
    "customer_id",
    "customer_zip_code_prefix",
    "customer_city",
    "customer_state"
]]
```

Figura 19: Fragmento de código correspondiente a la fase de transformación de `dim_customer`. (Elaboración propia)

Finalmente, durante el proceso de carga se establece conexión con la base de datos del Data Warehouse y se ejecuta una operación `DELETE` sobre la tabla destino para evitar inconsistencias derivadas de cargas anteriores. Tras ello, los registros se insertan mediante `executemany()`, optimizando la carga masiva de datos, y finalmente se cierran las conexiones utilizadas.

```

# =====
# CONECTAR A MYSQL
# =====
conn = mysql.connector.connect(**DB_CONFIG)
cur = conn.cursor()

# =====
# VACIAR TABLA (RECARGA COMPLETA)
# =====
cur.execute("DELETE FROM dim_customer;")
conn.commit()

# =====
# 8) INSERT MASIVO
# =====
insert_sql = """
    INSERT INTO dim_customer
    (customer_unique_id, customer_id, customer_zip_code_prefix, customer_city, customer_state)
    VALUES (%s, %s, %s, %s, %s)
"""

data = list(df_dim.itertuples(index=False, name=None))

cur.executemany(insert_sql, data)
conn.commit()

# =====
# 9) CERRAR CONEXIONES
# =====
cur.close()
conn.close()

```

Figura 20: Fragmento de código correspondiente a la fase de carga de `dim_customer`. (Elaboración propia)

5.1.2. `load_dim_product.py`

Este script es el más complejo de los tres de dimensiones. Durante este proceso lee dos fuentes independientes, corrige errores tipográficos del dataset original y carga todos los datos de forma adecuada para su futuro tratamiento.

Tabla <code>load_dim_product.py</code>	
Tabla destino	<code>dim_product</code>
CSV de entrada	<code>olist_products_dataset.csv</code> , <code>product_category_name_translation.csv</code>
Dependencias	Ninguna
Registros aprox.	32951 productos

Cuadro 15: Resumen del script `load_dim_product.py`

En la fase de extracción, como se ha mencionado anteriormente, se leen en paralelo los archivos `olist_products_dataset.csv` y `product_category_name_translation.csv`. El primer archivo contiene la información asociada a los productos, incluyendo la categoría definida en su idioma original, portugués. Por otro lado, el segundo proporciona la traducción de dichas categorías al inglés.

```
PRODUCTS_CSV = "olist_products_dataset.csv"
CATEGORIES_CSV = "product_category_name_translation.csv"
products = pd.read_csv(PRODUCTS_CSV)
categories = pd.read_csv(CATEGORIES_CSV)
```

Figura 21: Fragmento de código correspondiente a la fase de extracción de `dim_product`. (Elaboración propia)

En cuanto a la transformación, se realizan las siguientes operaciones de manera secuencial:

- **Corrección de errores tipográficos.** El dataset original contiene dos columnas con la palabra *length* incorrectamente escrita como *lenght*. Este problema se corrige mediante `rename()` antes de cualquier otra operación, garantizando una nomenclatura consistente y correcta en el Data Warehouse.
- **Limpieza de campos clave.** Se aplica sobre los atributos `product_id` y `product_category_name` del DataFrame de productos, así como sobre las columnas del DataFrame de traducciones. Estos atributos se convierten a tipo string y se eliminan espacios en blanco extremos para evitar inconsistencias durante las operaciones posteriores.
- **Join con el archivo de traducción.** Se realiza un `LEFT JOIN` entre los productos y las categorías traducidas utilizando el atributo `product_category_name`. El uso de esta operación garantiza que todos los productos permanezcan en la dimensión aunque no exista una traducción asociada, en cuyo caso el atributo correspondiente quedará con valor `NULL`.
- **Deduplicación.** Se garantiza la existencia de un único registro por cada `product_id`.
- **Selección de columnas.** Se conservan únicamente los atributos necesarios para el análisis posterior: `product_id`, `product_category_name`, `product_category_name_english`, `product_name_length`, `product_description_length`, `product_photos_qty`, `product_weight_g`, `product_length_cm`, `product_height_cm` y `product_width_cm`.
- **Gestión de valores nulos.** Los valores `NaN` se transforman a `None` para permitir su correcta interpretación como `NULL` durante la inserción en MySQL. Esta conversión permite mantener registros incompletos sin necesidad de eliminarlos del proceso de carga.

```
# =====
# 2) CORREGIR COLUMNAS MAL ESCRITAS
# =====
products = products.rename(columns={
    "product_name_lenght": "product_name_length",
    "product_description_lenght": "product_description_length"
})

# =====
# 3) LIMPIEZA DE CAMPOS CLAVE
# =====
products["product_id"] = products["product_id"].astype("string").str.strip()
products["product_category_name"] = products["product_category_name"].astype("string").str.strip()

categories["product_category_name"] = categories["product_category_name"].astype("string").str.strip()
categories["product_category_name_english"] = categories["product_category_name_english"].astype("string").str.strip()
# =====
# 4) TRANSFORM → JOIN PARA AÑADIR TRADUCCIÓN
# =====
df = products.merge(
    categories,
    on="product_category_name",
    how="left"
)
# =====
# 5) ELIMINAR DUPLICADOS
# =====
df_dim = df.drop_duplicates(subset=["product_id"], keep="first").copy()
```

Figura 22: Fragmento de código correspondiente a la primera fase de transformación de `dim_product`. (Elaboración propia)

```
# =====
# 6) SELECCIONAR COLUMNAS DEL DATA WAREHOUSE
# =====
df_dim = df_dim[[
    "product_id",
    "product_category_name",
    "product_category_name_english",
    "product_name_length",
    "product_description_length",
    "product_photos_qty",
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm"
]]
# =====
# 7) CONVERTIR NaN A None
# =====
df_dim = df_dim.astype(object).where(df_dim.notna(), None)
```

Figura 23: Fragmento de código correspondiente a la segunda fase de transformación de `dim_product`. (Elaboración propia)

Finalmente, durante el proceso de carga se establece conexión con la base de datos del Data Warehouse y se ejecuta una operación **DELETE** sobre la tabla destino para evitar inconsistencias derivadas de cargas anteriores. Tras ello, los registros se insertan mediante `executemany()`, optimizando la carga masiva de datos, y finalmente se cierran las conexiones utilizadas.

```

# =====
# 9) CONEXIÓN A MYSQL
# =====
conn = mysql.connector.connect(**DB_CONFIG)
cur = conn.cursor()
# =====
# 10) RECARGA COMPLETA DE LA DIMENSIÓN
# =====
# Se eliminan los datos anteriores antes de insertar los nuevos
cur.execute("DELETE FROM dim_product;")
conn.commit()
# =====
# 11) SENTENCIA SQL DE INSERCIÓN
# =====
insert_sql = """
    INSERT INTO dim_product
    (
        product_id,
        product_category_name,
        product_category_name_english,
        product_name_length,
        product_description_length,
        product_photos_qty,
        product_weight_g,
        product_length_cm,
        product_height_cm,
        product_width_cm
    )
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
"""
# =====
# 12) CONVERTIR DATAFRAME EN TUPLAS PARA INSERT MASIVO
# =====
data = list(df_dim.itertuples(index=False, name=None))
cur.executemany(insert_sql, data)
conn.commit()
# =====
# 13) CERRAR CONEXIÓN
# =====
cur.close()
conn.close()

```

Figura 24: Fragmento de código correspondiente a la fase de carga de `dim_product`. (Elaboración propia)

5.1.3. load_dim_date.py

Este script es el único que no carga datos directamente desde un archivo CSV hacia una dimensión del Data Warehouse, sino que genera la dimensión temporal de forma sintética. Para ello, utiliza el archivo `olist_orders_dataset.csv` únicamente con el objetivo de identificar el rango temporal mínimo y máximo existente en los pedidos, permitiendo construir automáticamente la secuencia completa de fechas comprendidas dentro de dicho intervalo.

Tabla load_dim_date.py	
Tabla destino	dim_date
CSV de entrada	olist_orders_dataset.csv (solo utilizado para obtener el rango temporal)
Dependencias	Ninguna
Registros aprox.	735 días

Cuadro 16: Resumen del script load_dim_date.py

En este caso, la extracción se realiza a partir del CSV de los pedidos, cargando el dataset para extraer el rango de fechas.

```
ORDERS_CSV = "olist_orders_dataset.csv"
orders = pd.read_csv(ORDERS_CSV)
```

Figura 25: Fragmento de código correspondiente a la fase de extracción de `dim_date`. (Elaboración propia)

Por otro lado, la transformación se compone de distintas fases. En primer lugar, el atributo `order_purchase_timestamp` se convierte a tipo `datetime`, obteniendo posteriormente las fechas mínima y máxima presentes en el conjunto de datos. Una vez determinado el intervalo temporal, se genera un `DataFrame` con una fila por cada día comprendido dentro del rango calculado. A partir de cada fecha se derivan los distintos atributos temporales que forman la dimensión. El campo `date_sk` adopta un formato numérico `YYYYMMDD` (por ejemplo, `20210204` para representar el 4 de febrero de 2021), actuando como clave primaria dentro de la dimensión temporal y utilizándose posteriormente como clave foránea en las tablas de hechos. Además, el atributo `day_of_week` se calcula sumando una unidad al valor devuelto por `pandas`, el cual numera los días desde cero. De esta forma se obtiene una escala comprendida entre 1 y 7, donde el valor 1 corresponde al lunes y el valor 7 al domingo.

Finalmente, el atributo `full_date` se convierte desde `datetime` a tipo `date`, garantizando un formato adecuado antes de realizar la carga en el Data Warehouse.


```

# =====
# 2) CONVERTIR A DATETIME
# =====
orders["order_purchase_timestamp"] = pd.to_datetime(
    orders["order_purchase_timestamp"], errors="coerce"
)
# =====
# 3) RANGO DE FECHAS
# =====
min_date = orders["order_purchase_timestamp"].min().date()
max_date = orders["order_purchase_timestamp"].max().date()
# =====
# 4) GENERAR CALENDARIO
# =====

dim_date = pd.DataFrame({
    "full_date": pd.date_range(start=min_date, end=max_date, freq="D")
})
dim_date["date_sk"] = dim_date["full_date"].dt.strftime("%Y%m%d").astype(int)
dim_date["year"] = dim_date["full_date"].dt.year
dim_date["month"] = dim_date["full_date"].dt.month
dim_date["day"] = dim_date["full_date"].dt.day
dim_date["quarter"] = dim_date["full_date"].dt.quarter
dim_date["week_of_year"] = dim_date["full_date"].dt.isocalendar().week.astype(int)
dim_date["day_of_week"] = dim_date["full_date"].dt.dayofweek + 1
dim_date["full_date"] = dim_date["full_date"].dt.date

```

Figura 26: Fragmento de código correspondiente a la fase de transformación de `dim_date`. (Elaboración propia)

Para concluir el script, como en los casos anteriores, se establece la conexión, se vacía la tabla y se cargan los atributos seleccionados.

```

# =====
# 5) CONEXIÓN MYSQL
# =====
# Se establece conexión con la base de datos del Data Warehouse
conn = mysql.connector.connect(**DB_CONFIG)
cur = conn.cursor()
# =====
# 6) RECARGA COMPLETA
# =====
cur.execute("DELETE FROM dim_date;")
conn.commit()
# =====
# 7) INSERT MASIVO
# =====
insert_sql = """
INSERT INTO dim_date
(date_sk, full_date, year, month, day, quarter, week_of_year, day_of_week)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
"""

data = list(dim_date[
    ["date_sk", "full_date", "year", "month", "day", "quarter", "week_of_year", "day_of_week"]
].itertuples(index=False, name=None))

cur.executemany(insert_sql, data)
conn.commit()
# =====
# 9) CERRAR CONEXIÓN
# =====
cur.close()
conn.close()

```

Figura 27: Fragmento de código correspondiente a la fase de carga de `dim_date`. (Elaboración propia)

5.1.4. load_fact_sales.py

Este es el script más complejo del proceso ETL. Combina tres archivos CSV, realiza múltiples joins y es el único que necesita consultar el Data Warehouse durante la fase de transformación.

Tabla load_fact_sales.py	
Tabla destino	fact_sales
CSV de entrada	olist_orders_dataset.csv, olist_order_items_dataset.csv, olist_customers_dataset.csv
Dependencias	Requiere dim_customer y dim_product cargadas en MySQL.
Registros aprox.	112650 líneas de pedido

Cuadro 17: Resumen del script load_fact_sales.py

En la fase de extracción se leen simultáneamente los tres archivos CSV necesarios. Estos archivos son olist_order_items_dataset.csv como fuente principal, manteniendo una fila por línea de pedido; olist_orders_dataset.csv, utilizado para obtener la información temporal y el estado asociado a cada pedido; y olist_customers_dataset.csv, empleado para relacionar el identificador customer_id con customer_unique_id, permitiendo asociar cada pedido con un cliente único.

```
ORDER_ITEMS_CSV = "olist_order_items_dataset.csv"
ORDERS_CSV = "olist_orders_dataset.csv"
CUSTOMERS_CSV = "olist_customers_dataset.csv"
order_items = pd.read_csv(ORDER_ITEMS_CSV)
orders = pd.read_csv(ORDERS_CSV)
customers = pd.read_csv(CUSTOMERS_CSV)
```

Figura 28: Fragmento de código correspondiente a la fase de extracción de fact_sales. (Elaboración propia)

En cuanto a la fase de transformación, el script aplica varias operaciones en secuencia:

- **Conversión de fechas.** Se convierten a `datetime` todas las columnas de fecha del dataset de pedidos y la columna `shipping_limit_date` de `order_items`.
- **Join order_items + orders.** Se realiza un `LEFT JOIN` sobre `order_id` para añadir a cada línea de producto el estado del pedido, la fecha de compra y las distintas fechas del ciclo de vida.
- **Join con customers.** Se realiza un `LEFT JOIN` sobre `customer_id` para añadir `customer_unique_id`, necesario para la resolución posterior de la clave sustituta de cliente.
- **Resolución de claves sustitutas.** Es una operación compleja en este script. Se abre una conexión a MySQL y se leen las dimensiones ya cargadas para obtener los mapeos entre claves

naturales y sustitutas. A partir de este paso, se realizan dos joins para reemplazar las claves naturales por las claves sustitutas.

- **Generación de `date_sk`.** Se formatea `order_purchase_timestamp` como entero `YYYYMMDD` para obtener la clave foránea hacia `dim_date`.
- **Selección de columnas.** Se seleccionan las columnas que van a formar parte de la tabla de ventas, las cuales son: `customer_sk`, `product_sk`, `date_sk`, `order_id`, `order_item_id`, `seller_id`, `shipping_limit_date`, `order_status`, `order_purchase_timestamp`, `order_approved_at`, `order_delivered_date`, `order_delivered_customer_date`, `order_estimated_delivery_date`, `price` y `freight_value`.
- **Gestión de nulos.** Se convierten los valores `NaN` a `None` para evitar problemas en la base de datos, ya que varios campos, como la fecha de entrega, pueden ser nulos para pedidos no completados.

```

# 2) CONVERTIR COLUMNAS DE FECHA A DATETIME
# =====
# Se transforman todas las fechas relevantes del pedido
order_date_cols = [
    "order_purchase_timestamp",
    "order_approved_at",
    "order_delivered_carrier_date",
    "order_delivered_customer_date",
    "order_estimated_delivery_date",
]
for c in order_date_cols:
    orders[c] = pd.to_datetime(orders[c], errors="coerce")

order_items["shipping_limit_date"] = pd.to_datetime(
    order_items["shipping_limit_date"], errors="coerce"
)
# =====
# 3) JOIN order_items + orders
# =====
df = order_items.merge(
    orders[
        [
            "order_id",
            "customer_id",
            "order_status",
            "order_purchase_timestamp",
            "order_approved_at",
            "order_delivered_carrier_date",
            "order_delivered_customer_date",
            "order_estimated_delivery_date",
        ]
    ],
    on="order_id",
    how="left",
)
# =====
# 4) JOIN CON CLIENTES
# =====
df = df.merge(
    customers[["customer_id", "customer_unique_id"]],
    on="customer_id",
    how="left",
)
# =====
# 5) OBTENER CLAVES SUSTITUTAS DE DIMENSIONES
# =====
conn = mysql.connector.connect(**DB_CONFIG)

dim_customer = pd.read_sql(
    "SELECT customer_sk, customer_unique_id FROM dim_customer",
    conn
)
dim_product = pd.read_sql(
    "SELECT product_sk, product_id FROM dim_product",
    conn
)

```

Figura 29: Fragmento de código correspondiente a la primera fase de transformación de `fact_sales`. (Elaboración propia)

```

# =====
# 6) JOIN CON DIMENSIONES
# =====
df = df.merge(dim_customer, on="customer_unique_id", how="left")
df = df.merge(dim_product, on="product_id", how="left")
# =====
# 7) CREAR date_sk
# =====
df["date_sk"] = pd.to_numeric(
    df["order_purchase_timestamp"].dt.strftime("%Y%m%d"),
    errors="coerce"
)
# =====
# 8) CONSTRUIR FACT TABLE
# =====
fact = df[
    [
        "customer_sk",
        "product_sk",
        "date_sk",
        "order_id",
        "order_item_id",
        "seller_id",
        "shipping_limit_date",
        "order_status",
        "order_purchase_timestamp",
        "order_approved_at",
        "order_delivered_carrier_date",
        "order_delivered_customer_date",
        "order_estimated_delivery_date",
        "price",
        "freight_value",
    ]
].copy()
# =====
# 9) CONVERTIR NaN A None
# =====
fact = fact.astype(object).where(fact.notna(), None)

```

Figura 30: Fragmento de código correspondiente a la segunda fase de transformación de `fact_sales`. (Elaboración propia)

Finalmente, los registros se cargan en la tabla de hechos mediante el mismo procedimiento de inserción utilizado anteriormente. Antes de realizar la carga se ejecuta una operación `DELETE` sobre la tabla destino para evitar inconsistencias derivadas de cargas previas y, una vez completada la inserción, se cierran las conexiones utilizadas.

En este caso, cabe destacar que la conexión MySQL abierta durante la fase de transformación para consultar las dimensiones ya cargadas se reutiliza posteriormente durante la fase de carga, evitando la apertura de una segunda conexión y reduciendo el coste asociado a operaciones adicionales de conexión.

```

# =====
# 10) VACIAR TABLA (RECARGA COMPLETA)
# =====
cur = conn.cursor()
cur.execute("DELETE FROM fact_sales;")
conn.commit()
# =====
# 11) INSERT MASIVO
# =====
insert_sql = """
    INSERT INTO fact_sales
    (
        customer_sk,
        product_sk,
        date_sk,
        order_id,
        order_item_id,
        seller_id,
        shipping_limit_date,
        order_status,
        order_purchase_timestamp,
        order_approved_at,
        order_delivered_carrier_date,
        order_delivered_customer_date,
        order_estimated_delivery_date,
        price,
        freight_value
    )
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
"""

data = list(fact.itertuples(index=False, name=None))
cur.executemany(insert_sql, data)
conn.commit()
# =====
# 12) CERRAR CONEXIONES
# =====
cur.close()
conn.close()

```

Figura 31: Fragmento de código correspondiente a la fase de carga de **fact_sales**. (Elaboración propia)

5.1.5. load_fact_payment.py

Este script se utiliza para cargar la tabla de hechos de pagos. Su granularidad es el pago secuencial, ya que un pedido puede haberse abonado mediante múltiples métodos de pago, por lo que puede existir más de un registro por **order_id**. Cabe destacar que el dataset de pagos no contiene información de fecha, por lo que es necesario un join con el dataset de pedidos para obtenerla.

Tabla load_fact_payment.py	
Tabla destino	fact_payment
CSV de entrada	olist_orders_dataset.csv, olist_order_payments_dataset.csv
Dependencias	Requiere dim_date cargada en MySQL.
Registros aprox.	103886 pagos

Cuadro 18: Resumen del script load_fact_payment.py

En este script, la fase de extracción lee dos archivos CSV: `olist_order_payments_dataset.csv` como fuente principal y `olist_orders_dataset.csv` para obtener el atributo de fecha `order_purchase_timestamp`.

```
PAYMENTS_CSV = "olist_order_payments_dataset.csv"
ORDERS_CSV = "olist_orders_dataset.csv"
payments = pd.read_csv(PAYMENTS_CSV)
orders = pd.read_csv(ORDERS_CSV)
```

Figura 32: Fragmento de código correspondiente a la fase de extracción de `fact_payment`. (Elaboración propia)

En cuanto a la fase de transformación, se realizan las siguientes operaciones para adaptar los datos al diseño esperado:

- **Conversión de fecha.** Se convierte `order_purchase_timestamp` del dataset de pedidos a formato `datetime` para poder calcular `date_sk`.
- **JOIN pagos + pedidos.** Se realiza un `LEFT JOIN` sobre `order_id` para añadir la fecha de compra a cada registro de pago. Como se ha mencionado anteriormente, se selecciona únicamente `order_purchase_timestamp` del dataset de pedidos, descartando el resto de los atributos.
- **Generación de date_sk.** Se crea y formatea la clave de fecha a partir del atributo, definiendo el formato como entero `YYYYMMDD`.
- **Selección de columnas.** Se seleccionan los campos necesarios para la creación de `fact_payment`, manteniendo la granularidad de una fila por pago dentro de un pedido.
- **Gestión de nulos.** Se convierten los valores `NaN` a `None`, ya que principalmente el campo `payment_installments` puede contener nulos en determinados casos.

```

# =====
# 2) CONVERTIR FECHA DE COMPRA A DATETIME
# =====
orders["order_purchase_timestamp"] = pd.to_datetime(
    orders["order_purchase_timestamp"], errors="coerce"
)
# =====
# 3) JOIN PAYMENTS + ORDERS
# =====
df = payments.merge(
    orders[["order_id", "order_purchase_timestamp"]],
    on="order_id",
    how="left"
)
# =====
# 4) CREAR date_sk (YYYYMMDD)
# =====
df["date_sk"] = pd.to_numeric(
    df["order_purchase_timestamp"].dt.strftime("%Y%m%d"),
    errors="coerce"
)
# =====
# 5) CONSTRUIR FACT TABLE
# =====
fact = df[[
    "date_sk",
    "order_id",
    "payment_sequential",
    "payment_type",
    "payment_installments",
    "payment_value"
]].copy()
# =====
# 6) CONVERTIR NaN A None
# =====
# MySQL necesita NULL, no NaN
fact = fact.astype(object).where(fact.notna(), None)

```

Figura 33: Fragmento de código correspondiente a la fase de transformación de `fact_payment`. (Elaboración propia)

Finalmente se realiza la carga a MySQL de los seis atributos de la tabla y se cierra la conexión.


```
# =====  
# 7) CONECTAR A MYSQL  
# =====  
conn = mysql.connector.connect(**DB_CONFIG)  
cur = conn.cursor()  
# =====  
# 8) VACIAR TABLA (RECARGA COMPLETA)  
# =====  
cur.execute("DELETE FROM fact_payment;")  
conn.commit()  
# =====  
# 9) INSERT MASIVO  
# =====  
insert_sql = """  
    INSERT INTO fact_payment  
    (  
        date_sk,  
        order_id,  
        payment_sequential,  
        payment_type,  
        payment_installments,  
        payment_value  
    )  
    VALUES (%s, %s, %s, %s, %s, %s)  
""">  
  
data = list(fact.itertuples(index=False, name=None))  
cur.executemany(insert_sql, data)  
conn.commit()  
# =====  
# 10) CERRAR CONEXIONES  
# =====  
cur.close()  
conn.close()
```

Figura 34: Fragmento de código correspondiente a la fase de carga de `fact_payment`. (Elaboración propia)

5.1.6. `load_fact_review.py`

Este script carga la tabla de hechos de reseñas. Es el script más simple y autónomo de los tres, ya que toda la información necesaria está contenida en un único archivo CSV sin necesidad de joins ni consultas extra.

Tabla load_fact_review.py	
Tabla destino	fact_review
CSV de entrada	olist_orders_reviews_dataset.csv
Dependencias	Requiere dim_date cargada en MySQL.
Registros aprox.	99224 reseñas

Cuadro 19: Resumen del script load_fact_review.py

En la fase de extracción se lee únicamente el archivo `olist_orders_reviews_dataset.csv`, ya que contiene todos los atributos necesarios para la construcción de esta tabla de hechos.

```
REVIEWS_CSV = "olist_order_reviews_dataset.csv"
reviews = pd.read_csv(REVIEWS_CSV)
```

Figura 35: Fragmento de código correspondiente a la fase de extracción de `fact_review`. (Elaboración propia)

En cuanto a la fase de transformación, el script realiza las siguientes cinco operaciones de forma secuencial:

- **Conversión de fechas.** Se convierten a `datetime` los dos campos de fecha del dataset, `review_creation_date` (fecha en la que el cliente escribió la reseña) y `review_answer_timestamp` (fecha en la que se respondió a la reseña). Ambas fechas se conservan para permitir el análisis del tiempo de respuesta.
- **Generación de `date_sk`.** Se genera la clave de fecha a partir de `review_creation_date`, que es la fecha de referencia de la reseña, formateada como entero `YYYYMMDD`. Cabe destacar que en este caso la fecha se genera a partir de este atributo porque el hecho analizado no es la compra, sino la valoración realizada por el cliente. De esta forma, el análisis temporal de las reviews se vincula al momento en el que se emitió la valoración, sin mantener relación con la fecha de compra.
- **Crear indicador `has_comment`.** Se genera un indicador binario que tiene el valor de 1 si el cliente escribió un comentario y 0 si es lo contrario. Se diseña a partir del atributo `review_comment_message`.
- **Selección de columnas.** Se seleccionan los siete atributos que forman parte de la tabla `fact_review`.
- **Gestión de nulos.** Se convierten los valores nulos de `NaN` a `None` para poder insertarlos correctamente en MySQL.

```

# =====
# 2) CONVERTIR COLUMNAS DE FECHA A DATETIME
# =====
reviews["review_creation_date"] = pd.to_datetime(
    reviews["review_creation_date"], errors="coerce"
)

reviews["review_answer_timestamp"] = pd.to_datetime(
    reviews["review_answer_timestamp"], errors="coerce"
)

# =====
# 3) CREAR date_sk
# =====
reviews["date_sk"] = pd.to_numeric(
    reviews["review_creation_date"].dt.strftime("%Y%m%d"),
    errors="coerce"
)

# =====
# 4) CREAR INDICADOR has_comment
# =====
reviews["has_comment"] = reviews["review_comment_message"].notna().astype(int)

# =====
# 5) CONSTRUIR FACT TABLE
# =====
fact = reviews[
    [
        "date_sk",
        "order_id",
        "review_id",
        "review_score",
        "review_creation_date",
        "review_answer_timestamp",
        "has_comment"
    ]
].copy()

# =====
# 6) CONVERTIR NaN A None
# =====
fact = fact.astype(object).where(fact.notna(), None)

```

Figura 36: Fragmento de código correspondiente a la fase de transformación de `fact_review`. (Elaboración propia)

Finalmente, los registros se cargan en la tabla de hechos mediante el mismo procedimiento utilizado en los scripts anteriores. Antes de la inserción se ejecuta una operación `DELETE` sobre la tabla destino para evitar inconsistencias derivadas de cargas previas. Posteriormente, los registros se insertan mediante `executemany()`, optimizando la carga masiva de datos, y finalmente se cierran las conexiones utilizadas.

```
# =====
# 7) CONECTAR CON MYSQL
# =====
conn = mysql.connector.connect(**DB_CONFIG)
cur = conn.cursor()
# =====
# 8) VACIAR TABLA (RECARGA COMPLETA)
# =====
cur.execute("DELETE FROM fact_review;")
conn.commit()

# =====
# 9) INSERT MASIVO
# =====
insert_sql = """
    INSERT INTO fact_review
    (
        date_sk,
        order_id,
        review_id,
        review_score,
        review_creation_date,
        review_answer_timestamp,
        has_comment
    )
    VALUES (%s, %s, %s, %s, %s, %s, %s)
"""

data = list(fact.itertuples(index=False, name=None))
cur.executemany(insert_sql, data)
conn.commit()
# =====
# 10) CERRAR CONEXIONES
# =====
cur.close()
conn.close()
```

Figura 37: Fragmento de código correspondiente a la fase de carga de `fact_review`. (Elaboración propia)

5.2. Construcción del Data Warehouse

Este apartado describe la implementación física del Data Warehouse sobre MySQL, centrada en la creación de la base de datos final derivada del modelo dimensional, incluyendo el proceso de generación del esquema de base de datos, la estructura de las tablas creadas y las pruebas de validación realizadas para verificar la integridad y el rendimiento del sistema.

5.2.1. Forward Engineering desde MySQL Workbench

La implementación física del Data Warehouse se ha llevado a cabo mediante el proceso de Forward Engineering de MySQL. Este proceso consiste en traducir automáticamente el modelo dimensional diseñado en el capítulo 4 a un script SQL de definición de datos (DDL), que incluye la creación del esquema, las tablas con sus tipos de dato, las restricciones de integridad y los índices necesarios.

El proceso se ejecuta desde MySQL Workbench a través del menú Database, seleccionando Forward Engineer y seleccionando el modelo dimensional previamente diseñado como origen. La herramienta genera el script SQL completo, que se ejecuta sobre la instancia de MySQL para materializar el esquema `tfg_olist_dw` con todas sus tablas. Este enfoque garantiza la coherencia entre el modelo lógico diseñado y la implementación física resultante, eliminando el riesgo de discrepancias manuales entre ambos.

El script generado comienza con la creación del esquema de base de datos con codificación UTF-8.

```
-----
-- Schema tfg_olist_dw
-----
CREATE SCHEMA IF NOT EXISTS `tfg_olist_dw` DEFAULT CHARACTER SET utf8 ;
USE `tfg_olist_dw` ;
```

Figura 38: Sentencia SQL de creación del esquema del Data Warehouse. (Elaboración propia)

5.2.2. Implementación de las tablas dimensión

De las tres tablas dimensión del modelo, `dim_date` es la única que no se crea con clave primaria autoincremental, ya que su clave primaria es el entero `YYYYMMDD`, pero las tres utilizan índices únicos sobre sus claves naturales para garantizar la unicidad de los registros independientemente del proceso ETL.

En primer lugar, `dim_customer` implementa un índice único sobre `customer_unique_id` para impedir la inserción de clientes duplicados. Además de las características de atributos, ya comentadas previamente en la fase de diseño, los atributos de localización se definen como nullable dado que no todos los registros del dataset disponen de información completa.

```
-----
-- Table `tfg_olist_dw`.`dim_customer`
-----
CREATE TABLE IF NOT EXISTS `tfg_olist_dw`.`dim_customer` (
  `customer_sk` INT NOT NULL AUTO_INCREMENT,
  `customer_unique_id` VARCHAR(45) NOT NULL,
  `customer_id` VARCHAR(45) NULL,
  `customer_zip_code_prefix` VARCHAR(45) NULL,
  `customer_city` VARCHAR(45) NULL,
  `customer_state` CHAR(2) NULL,
  PRIMARY KEY (`customer_sk`),
  UNIQUE INDEX `customer_unique_id_UNIQUE` (`customer_unique_id` ASC) VISIBLE)
ENGINE = InnoDB;
```

Figura 39: Definición SQL de la tabla `dim_customer`. (Elaboración propia)

Por otro lado, en `dim_product`, los atributos de dimensiones físicas y longitudes se definen como nullable por la misma razón anterior. Se define un índice único sobre `product_id` para garantizar unicidad de registros y mejorar el rendimiento de las operaciones de búsqueda y validación.

```

-----
-- Table `tfg_olist_dw`.`dim_product`
-----
CREATE TABLE IF NOT EXISTS `tfg_olist_dw`.`dim_product` (
  `product_sk` INT NOT NULL AUTO_INCREMENT,
  `product_id` VARCHAR(45) NOT NULL,
  `product_category_name` VARCHAR(100) NULL,
  `product_category_name_english` VARCHAR(100) NULL,
  `product_name_length` INT NULL,
  `product_description_length` INT NULL,
  `product_photos_qty` INT NULL,
  `product_weight_g` INT NULL,
  `product_length_cm` INT NULL,
  `product_height_cm` INT NULL,
  `product_width_cm` INT NULL,
  PRIMARY KEY (`product_sk`),
  UNIQUE INDEX `product_id_UNIQUE` (`product_id` ASC) VISIBLE)
ENGINE = InnoDB;

```

Figura 40: Definición SQL de la tabla `dim_product`. (Elaboración propia)

Por último, `dim_date` es la única dimensión en la que la clave primaria no es autoincremental, como se ha mencionado. Todos los atributos son NOT NULL, ya que la dimensión se genera sintéticamente y no depende de datos externos con posibles nulos. Se añade un índice único sobre `full_date`.

```
-----  
-- Table `tfg_olist_dw`.`dim_date`  
-----  
CREATE TABLE IF NOT EXISTS `tfg_olist_dw`.`dim_date` (  
  `date_sk` INT NOT NULL,  
  `full_date` DATE NOT NULL,  
  `year` SMALLINT NOT NULL,  
  `month` TINYINT NOT NULL,  
  `day` TINYINT NOT NULL,  
  `quarter` TINYINT NOT NULL,  
  `week_of_year` TINYINT NOT NULL,  
  `day_of_week` TINYINT NOT NULL,  
  PRIMARY KEY (`date_sk`),  
  UNIQUE INDEX `full_date_UNIQUE` (`full_date` ASC) VISIBLE)  
ENGINE = InnoDB;
```

Figura 41: Definición SQL de la tabla `dim_date`. (Elaboración propia)

5.2.3. Implementación de las tablas hecho

Las tablas de hechos se implementan con claves foráneas hacia las dimensiones correspondientes con política `ON DELETE NO ACTION` y `ON UPDATE NO ACTION`, garantizando la integridad referencial sin eliminaciones en cascada que pudieran comprometer el histórico de datos. Se crean índices secundarios sobre todas las columnas de clave foránea para optimizar el rendimiento de las consultas analíticas de tipo `JOIN`.

En primer lugar, se encuentra `fact_sales`, la tabla central del modelo. Referencia las tres dimensiones mediante claves foráneas indexadas. Los campos de fechas del ciclo de vida del pedido se definen como nullable ya que no todos los pedidos completan todas las etapas.

```

-----
-- Table `tfg_olist_dw`.`fact_sales`
-----
CREATE TABLE IF NOT EXISTS `tfg_olist_dw`.`fact_sales` (
  `fact_sales_id` BIGINT NOT NULL AUTO_INCREMENT,
  `customer_sk` INT NOT NULL,
  `product_sk` INT NOT NULL,
  `date_sk` INT NOT NULL,
  `order_id` VARCHAR(45) NOT NULL,
  `order_item_id` INT NOT NULL,
  `seller_id` VARCHAR(45) NULL,
  `shipping_limit_date` DATETIME NULL,
  `order_status` VARCHAR(45) NOT NULL,
  `order_purchase_timestamp` DATETIME NULL,
  `order_approved_at` DATETIME NULL,
  `order_delivered_carrier_date` DATETIME NULL,
  `order_delivered_customer_date` DATETIME NULL,
  `order_estimated_delivery_date` DATETIME NULL,
  `price` DECIMAL(10,2) NOT NULL,
  `freight_value` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`fact_sales_id`),

```

Figura 42: Definición SQL de la tabla `fact_sales` parte uno. (Elaboración propia)

```

UNIQUE INDEX `idfact_sales_UNIQUE` (`fact_sales_id` ASC) VISIBLE,
INDEX `fk_fact_sales_dim_customer_idx` (`customer_sk` ASC) VISIBLE,
INDEX `fk_fact_sales_dim_product1_idx` (`product_sk` ASC) VISIBLE,
INDEX `fk_fact_sales_dim_date1_idx` (`date_sk` ASC) VISIBLE,
CONSTRAINT `fk_fact_sales_dim_customer`
  FOREIGN KEY (`customer_sk`)
    REFERENCES `tfg_olist_dw`.`dim_customer` (`customer_sk`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
CONSTRAINT `fk_fact_sales_dim_product1`
  FOREIGN KEY (`product_sk`)
    REFERENCES `tfg_olist_dw`.`dim_product` (`product_sk`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
CONSTRAINT `fk_fact_sales_dim_date1`
  FOREIGN KEY (`date_sk`)
    REFERENCES `tfg_olist_dw`.`dim_date` (`date_sk`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

```

Figura 43: Definición SQL de la tabla `fact_sales` parte dos. (Elaboración propia)

Por otro lado, **fact_payment** es más sencilla, ya que solo referencia la dimensión de fecha mediante clave foránea. El índice creado sobre **date_sk** mejora el rendimiento de las consultas que filtran o agrupan los pagos por fecha. Esto resulta especialmente útil en los análisis temporales realizados posteriormente.

```

-----
-- Table `tfg_olist_dw`.`fact_payment`
-----
CREATE TABLE IF NOT EXISTS `tfg_olist_dw`.`fact_payment` (
  `fact_payment_id` BIGINT NOT NULL AUTO_INCREMENT,
  `date_sk` INT NOT NULL,
  `order_id` VARCHAR(45) NOT NULL,
  `payment_sequential` INT NOT NULL,
  `payment_type` VARCHAR(45) NOT NULL,
  `payment_installments` INT NULL,
  `payment_value` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`fact_payment_id`),
  INDEX `fk_fact_payment_dim_date1_idx` (`date_sk` ASC) VISIBLE,
  CONSTRAINT `fk_fact_payment_dim_date1`
    FOREIGN KEY (`date_sk`)
      REFERENCES `tfg_olist_dw`.`dim_date` (`date_sk`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

```

Figura 44: Definición SQL de la tabla **fact_payment**. (Elaboración propia)

Por último, **fact_review**, que solo referencia la dimensión fecha, de la misma manera que la tabla de pagos. Gracias a esta relación, es posible analizar la evolución temporal de la satisfacción de los clientes, identificar tendencias y comparar valoraciones de distintos periodos.

```

-----
-- Table `tfg_olist_dw`.`fact_review`
-----

CREATE TABLE IF NOT EXISTS `tfg_olist_dw`.`fact_review` (
  `fact_review_id` BIGINT NOT NULL AUTO_INCREMENT,
  `date_sk` INT NOT NULL,
  `order_id` VARCHAR(45) NOT NULL,
  `review_id` VARCHAR(45) NOT NULL,
  `review_score` INT NOT NULL,
  `review_creation_date` DATETIME NULL,
  `review_answer_timestamp` DATETIME NULL,
  `has_comment` TINYINT NULL,
  PRIMARY KEY (`fact_review_id`),
  INDEX `fk_fact_review_dim_date1_idx` (`date_sk` ASC) VISIBLE,
  CONSTRAINT `fk_fact_review_dim_date1`
    FOREIGN KEY (`date_sk`)
      REFERENCES `tfg_olist_dw`.`dim_date` (`date_sk`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

```

Figura 45: Definición SQL de la tabla `fact_review`. (Elaboración propia)

5.3. Modelos de análisis de datos

En este apartado se describen los modelos de análisis de datos aplicados en el desarrollo del proyecto con el objetivo de dotar al sistema de una mayor capacidad analítica. Para ello, se han aplicado dos técnicas complementarias: clustering no supervisado para la segmentación de clientes y modelos de aprendizaje supervisado para la predicción de la satisfacción del cliente.

Ambos modelos se implementan en Python mediante la librería `scikit-learn` y toman como base los datos almacenados en el Data Warehouse.

5.3.1. Segmentación de clientes mediante K-means

El objetivo de este análisis es agrupar los clientes en segmentos homogéneos según factores como su comportamiento de compra, su gasto, el coste de envío asociado, el retraso en la entrega y su nivel de satisfacción. El algoritmo empleado es K-means, una técnica de clustering no supervisado que agrupa los registros en k clústeres minimizando la varianza intra-clúster.

En primer lugar, tiene lugar la fase de extracción. Los datos de entrada del modelo se obtienen directamente desde el Data Warehouse mediante una consulta SQL que agrega la información de las tres tablas de hechos por cliente único. La consulta combina `fact_sales`, `dim_customer` y `fact_review` para calcular, para cada cliente, su número de pedidos, gasto total, gasto medio de envío, retraso medio de entrega y puntuación media de las reseñas.

```
query = """
SELECT
    c.customer_unique_id,
    COUNT(DISTINCT s.order_id) AS num_orders,
    SUM(s.price) AS total_spent,
    AVG(s.freight_value) AS avg_freight,
    AVG(DATEDIFF(s.order_delivered_customer_date, s.order_estimated_delivery_date)) AS avg_delay,
    AVG(r.review_score) AS avg_review
FROM fact_sales s
JOIN dim_customer c ON s.customer_sk = c.customer_sk
LEFT JOIN fact_review r ON s.order_id = r.order_id
GROUP BY c.customer_unique_id
"""

df = pd.read_sql(query, conn)
```

Figura 46: Consulta SQL utilizada para la extracción de datos del modelo de segmentación de clientes. (Elaboración propia)

Posteriormente tiene lugar la fase de preprocesamiento, en la que se aplica un proceso de limpieza básico mediante `fillna(0)` para sustituir los valores nulos, principalmente en clientes sin reseñas, por cero. A continuación, se seleccionan las cinco variables de entrada del modelo y se escalan mediante `StandardScaler`, transformando cada variable para que tenga media 0 y desviación típica 1. Este escalado es imprescindible en K-Means, ya que el algoritmo calcula distancias entre registros y, si una variable presenta valores significativamente superiores a los de las demás, tendría una influencia excesiva en la formación de los grupos.

```
# 2) LIMPIEZA
df = df.fillna(0)

# 3) VARIABLES PARA EL CLUSTERING
features = df[[
    "num_orders",
    "total_spent",
    "avg_freight",
    "avg_delay",
    "avg_review"
]]

# 4) ESCALADO
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)
```

Figura 47: Fragmento de código correspondiente al preprocesamiento de datos y escalado de variables. (Elaboración propia)

Por último, se aplica K-Means con $k = 3$ clústeres y `random_state = 42` para garantizar la reproducibilidad del resultado. La elección de $k = 3$ responde a una decisión de diseño orientada a obtener segmentos interpretables y accionables desde el punto de vista de negocio, diferenciando de esta manera

entre clientes con alta satisfacción, clientes insatisfechos y clientes premium.

```
# 5) APLICAR K-MEANS
kmeans = KMeans(n_clusters=3, random_state=42)
df["cluster"] = kmeans.fit_predict(X_scaled)

# 6) MOSTRAR RESULTADOS EN CONSOLA
print("\nPrimeras filas con cluster asignado:")
print(df.head())

print("\nNúmero de clientes por cluster:")
print(df["cluster"].value_counts().sort_index())

print("\nPerfil medio de cada cluster:")
print(
    df.groupby("cluster")[[
        "num_orders",
        "total_spent",
        "avg_freight",
        "avg_delay",
        "avg_review"
    ]].mean()
)

# 7) GUARDAR RESULTADOS EN CSV
df.to_csv("customer_clusters.csv", index=False)
print("\n CSV generado: customer_clusters.csv")

conn.close()
```

Figura 48: Fragmento de código correspondiente a la aplicación del algoritmo K-Means. (Elaboración propia)

Como resultado del clustering, el algoritmo identifica tres segmentos con perfiles claramente diferenciados. La siguiente tabla muestra el perfil medio de cada clúster junto con el nombre asignado en función de sus características.

Clúster	Nombre	Nº Clientes	Pedidos medio	Gasto total medio	Gasto envío medio	Retraso medio	Review medio
0	Satisfechos	74.082 (77,6 %)	1,03	110,95	17,91	-13,4	4,65
1	Insatisfechos	17.036 (17,9 %)	1,02	126,27	18,98	-3,45	1,58
2	Premium	4.302 (4,5 %)	1,12	762,85	65,00	-12,81	4,06

Cuadro 20: Características medias de los segmentos de clientes identificados mediante K-Means

- **Clúster 0, Satisfechos.** Constituye el mayor porcentaje de la base de clientes. Son compradores ocasionales con gasto moderado, entregas significativamente adelantadas respecto a la fecha estimada y valoraciones muy positivas.

- **Clúster 1, Insatisfechos.** Está formado por clientes con un gasto similar al grupo anterior, pero con fechas de entrega mucho menos adelantadas y valoraciones muy bajas. El menor adelanto en la entrega parece correlacionar con una peor experiencia percibida por el cliente.
- **Clúster 2, Premium.** Son clientes de alto valor, con un gasto total medio casi siete veces superior al de los otros grupos y un coste de envío más elevado, posiblemente asociado a productos de mayores dimensiones y peso. Sus valoraciones son positivas, aunque ligeramente inferiores a las del grupo de clientes satisfechos, por lo que, a pesar de gastar más, no presentan un nivel de satisfacción superior.

Los resultados del clustering se exportan al archivo `customer_cluster.csv`, que sirve como entrada para los modelos predictivos descritos en el siguiente apartado y como fuente de datos para la página de Segmentación de Clientes del informe desarrollado en Power BI.

5.3.2. Predicción de la satisfacción del cliente

Con el objetivo de aumentar la profundidad del análisis mediante técnicas de Data Science, se ha decidido, además de incluir el análisis de clustering, implementar un modelo de predicción de la satisfacción del cliente. La variable objetivo es `avg_review`, que representa la valoración media del cliente, mientras que las variables explicativas son `num_orders`, `total_spent`, `avg_freight` y `avg_delay`. El dataset de entrada es el archivo `customer_clusters.csv` generado en el apartado anterior, que contiene un registro por cliente con sus métricas agregadas.

Para la preparación de los datos, el dataset se divide en un conjunto de entrenamiento (80 %) y un conjunto de prueba (20 %) mediante `train_test_split()` con `random_state = 42` para garantizar la reproducibilidad de los resultados. Las métricas de evaluación empleadas son el error absoluto medio (MAE) y el coeficiente de determinación (R^2).

```
# 1) Cargar datos
df = pd.read_csv("customer_clusters.csv")

# 2) Variables de entrada y variable objetivo
X = df[["num_orders", "total_spent", "avg_freight", "avg_delay"]]
y = df["avg_review"]

# 3) Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Figura 49: Preparación del conjunto de datos para el modelo de predicción de satisfacción. (Elaboración propia)

Para el análisis se implementan dos modelos, regresión lineal y Random Forest, y se realiza una comparativa entre ambos.

En primer lugar, se implementa un modelo de regresión lineal debido a su simplicidad e interpretabilidad. Este modelo permite analizar la dirección y magnitud de la relación existente entre cada variable

explicativa y la satisfacción del cliente mediante el estudio de sus coeficientes.

```
# =====
# MODELO 1: REGRESIÓN LINEAL
# =====
modelo_lr = LinearRegression()
modelo_lr.fit(X_train, y_train)
y_pred_lr = modelo_lr.predict(X_test)

mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

# Tabla real vs predicho - Regresión Lineal
resultados_lr = pd.DataFrame({
    "Review real": y_test.reset_index(drop=True),
    "Review predicha": pd.Series(y_pred_lr).round(2)
})
resultados_lr["Diferencia"] = (
    resultados_lr["Review real"] - resultados_lr["Review predicha"]
).round(2)
```

Figura 50: Entrenamiento del modelo de regresión lineal. (Elaboración propia)

Los resultados obtenidos muestran un MAE de 1,0137, lo que indica que el modelo comete un error medio de aproximadamente un punto en la escala de satisfacción. Por otro lado, el coeficiente de determinación de 0,0935 refleja una capacidad explicativa limitada, ya que el modelo solo explica el 9,35 % de la variabilidad de la satisfacción del cliente. Esto sugiere la existencia de factores no recogidos en el dataset que influyen en la valoración, como la calidad del producto o la experiencia de compra. El análisis de los coeficientes permite extraer conclusiones relevantes sobre el impacto de cada variable, las cuales se recogen en la siguiente tabla.

Variable	Coefficiente	Interpretación
Número de pedidos	+0,0496	Relación positiva. Los clientes con más pedidos tienden a otorgar mejores valoraciones.
Gasto total	-0,0002	Influencia prácticamente nula sobre la satisfacción.
Coste medio de envío	-0,0034	Relación negativa leve. Un mayor coste de envío se asocia con una menor satisfacción.
Retraso medio	-0,0399	Mayor impacto negativo observado. Los retrasos en la entrega reducen significativamente la satisfacción del cliente.
Intercepto	3,6517	Valor base estimado de la satisfacción cuando todas las variables explicativas toman valor cero.

Cuadro 21: Coeficientes obtenidos por el modelo de regresión lineal

Por tanto, el retraso en la entrega es la variable con mayor impacto negativo sobre la satisfacción, lo que confirma que los clientes penalizan especialmente los pedidos que llegan tarde.

Por otro lado, con el fin de mejorar la capacidad predictiva, se implementa adicionalmente un modelo

de Random Forest, que permite capturar relaciones no lineales entre las variables mediante un conjunto de árboles de decisión entrenados sobre submuestras aleatorias del dataset.

```
# =====
# MODELO 2: RANDOM FOREST
# =====
modelo_rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
modelo_rf.fit(X_train, y_train)
y_pred_rf = modelo_rf.predict(X_test)

mae_rf = mean_absolute_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)
```

Figura 51: Fragmento de código correspondiente al entrenamiento del modelo Random Forest. (Elaboración propia)

El modelo obtiene un MAE de 0,9813, inferior al de la regresión lineal, lo que indica una mayor precisión en la predicción de valores individuales. Sin embargo, el coeficiente de determinación R^2 de 0,0771 es inferior al obtenido por la regresión lineal, lo que sugiere que, aunque el modelo comete menos errores en promedio, su capacidad para explicar la variabilidad de la satisfacción es menor.

El análisis de la importancia de variables del modelo Random Forest ofrece una perspectiva complementaria sobre los factores que más contribuyen a la predicción.

Variable	Importancia	% sobre el total
Retraso medio	0,3551	35,5 %
Gasto total	0,3281	32,8 %
Coste medio de envío	0,3106	31,1 %
Número de pedidos	0,0061	0,6 %

Cuadro 22: Importancia de variables obtenida mediante el modelo Random Forest

A diferencia de la regresión lineal, el modelo Random Forest identifica el retraso, el gasto y el coste de envío como variables con una importancia similar y elevada, mientras que el número de pedidos contribuye de forma prácticamente nula. Este resultado matiza las conclusiones obtenidas mediante la regresión lineal, donde el gasto total aparecía como una variable irrelevante, sugiriendo que su relación con la satisfacción podría ser no lineal y que el modelo basado en árboles es capaz de capturarla de forma más adecuada.

En cuanto a la comparativa entre ambos modelos, la siguiente tabla resume las métricas de evaluación obtenidas sobre el conjunto de prueba.

Modelo	MAE	R^2	Interpretación
Regresión lineal	1,0137	0,0935	Error medio cercano a 1. Baja capacidad explicativa global, pero coeficientes fácilmente interpretables.
Random Forest	0,9813	0,0771	Menor error medio individual. Presenta un R^2 inferior, lo que indica una menor capacidad explicativa global.

Cuadro 23: Comparativa de resultados entre los modelos predictivos

Ambos modelos presentan métricas modestas en términos de R^2 , lo que evidencia que la satisfacción del cliente es una variable compleja influida por factores no recogidos en el dataset, como la calidad del producto, la atención al cliente o la experiencia de compra. No obstante, los resultados confirman que los factores operativos, especialmente el retraso en la entrega, tienen un impacto significativo y consistente en ambos modelos.

En términos de elección del modelo, la regresión lineal ofrece una mayor interpretabilidad gracias a sus coeficientes, lo que la hace más adecuada para extraer conclusiones de negocio. Por su parte, el modelo Random Forest mejora ligeramente la precisión individual de las predicciones, pero a cambio de perder transparencia en el razonamiento del modelo. Para el objetivo de este proyecto, la regresión lineal resulta más útil como herramienta de análisis, mientras que el Random Forest aporta valor como mecanismo de contraste y validación de las conclusiones obtenidas.

Como resultado del proceso de entrenamiento y evaluación de los modelos predictivos, se generan varios archivos CSV que posteriormente son utilizados como fuente de datos adicional en Power BI para la construcción de la página de análisis predictivo.

- **Predicciones_modelos.csv.** Contiene las predicciones generadas por ambos modelos sobre el conjunto de prueba, con una fila por cada cliente evaluado. Incluye las columnas **Review real**, **Review predicha**, **Diferencia** y **Modelo**, permitiendo comparar en una misma tabla los resultados de la Regresión Lineal y del Random Forest.
- **Comparativa_modelos.csv.** Tabla resumen con una fila por modelo que recoge las métricas globales de evaluación, concretamente el MAE y el R^2 . Este archivo constituye la fuente de datos utilizada por las tarjetas KPI de la página.
- **Importancia_variables.csv.** Contiene la importancia relativa de cada variable explicativa calculada por el modelo Random Forest, incluyendo las columnas **Variable** e **Importancia**, ordenadas de mayor a menor contribución.
- **Coeficientes_regresion_lineal.csv.** Contiene los coeficientes obtenidos por el modelo de regresión lineal para cada variable explicativa, incluyendo las columnas **Variable** y **Coeficiente**. Los valores positivos indican un efecto favorable sobre la satisfacción del cliente, mientras que los negativos representan un impacto desfavorable.

5.4. Implementación de los dashboards

Este apartado describe la implementación del informe en Power BI, que constituye la capa de explotación analítica del Data Warehouse desarrollado. El informe se conecta a la base de datos MySQL

mediante el conector nativo de Power BI, accesible desde la opción *Obtener datos* de la pestaña *Inicio*. Una vez configurada la conexión, se importan las tablas del Data Warehouse para su posterior modelado.

Tras la carga de los datos, se verifican y completan las relaciones en la vista de modelo de Power BI, garantizando que las relaciones entre tablas de hechos y dimensiones se corresponden con las claves foráneas definidas durante el diseño del Data Warehouse. Asimismo, se incorporan las relaciones necesarias para integrar los resultados generados por los modelos de análisis de datos. Sobre este modelo relacional se construyen las medidas DAX necesarias para cada página y se desarrollan las distintas visualizaciones siguiendo el diseño propuesto en el apartado 4.3.

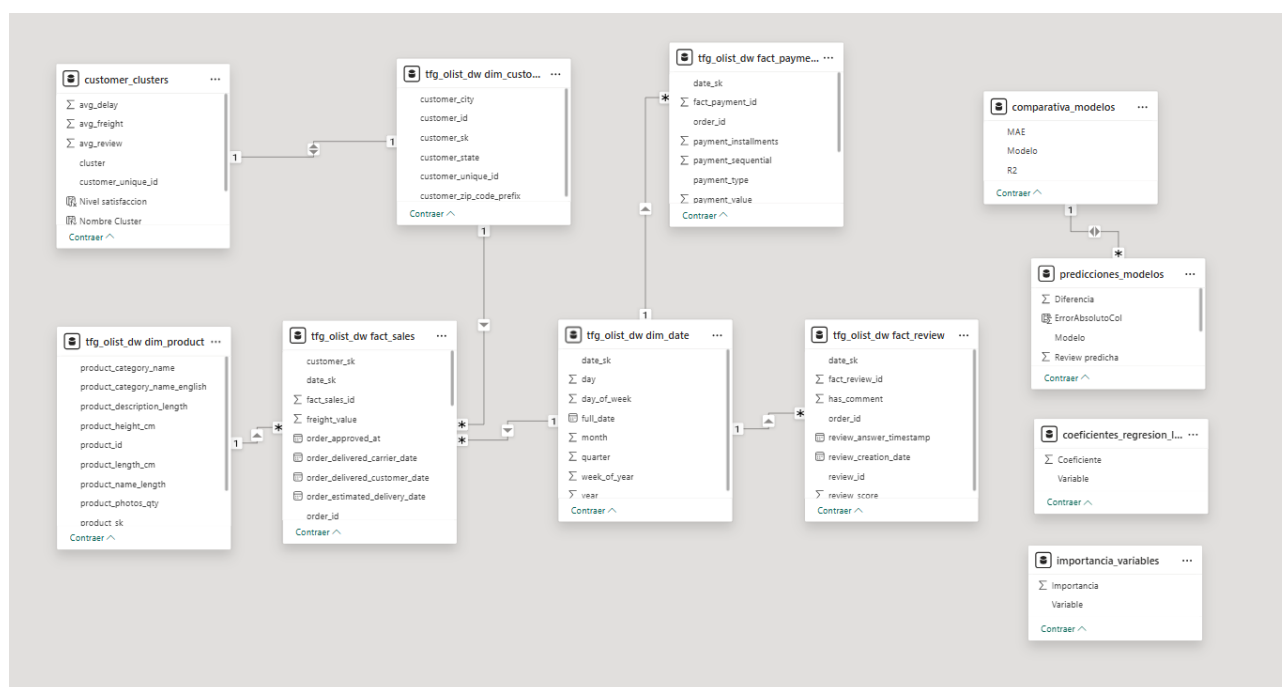


Figura 52: Vista de modelo de Power BI con las relaciones del Data Warehouse y los modelos analíticos. (Elaboración propia)

El informe final se estructura en seis páginas temáticas interconectadas mediante un sistema de navegación que permite al usuario desplazarse entre ellas de forma intuitiva. Cada página incorpora filtros interactivos que modifican dinámicamente todas las visualizaciones asociadas, facilitando la exploración de la información desde diferentes perspectivas. A continuación, se describe el proceso de implementación de cada una de ellas.

5.4.1. Visión General

La página *Visión General* constituye la pantalla principal del informe y tiene como objetivo proporcionar una visión ejecutiva del estado global del negocio. Para ello, reúne los indicadores más relevantes a nivel agregado y permite al usuario obtener una primera aproximación al volumen de actividad y a la evolución general de la empresa antes de profundizar en áreas específicas de análisis.



Figura 53: Página Visión General del informe desarrollado en Power BI. (Elaboración propia)

En primer lugar, se implementan los principales indicadores KPI y filtros de la página:

- **Tarjeta – Nº Pedidos.** Muestra el número total de pedidos distintos para el periodo seleccionado. La medida utiliza la función DAX `DISTINCTCOUNT` sobre el atributo `order_id`, evitando así el doble conteo derivado de la granularidad de línea de pedido existente en la tabla `fact_sales`.
- **Tarjeta – Coste Envío Total.** Representa la suma del coste de envío de todas las líneas de pedido incluidas en el contexto de filtrado actual. La medida se implementa mediante la función DAX `SUM` aplicada sobre el atributo `freight_value`.
- **Medidor – % de facturación destinado a envío.** Indicador de tipo velocímetro que muestra el porcentaje que representa el coste de envío sobre la facturación total. Para su cálculo se utiliza una medida basada en la función DAX `DIVIDE`, lo que permite gestionar correctamente posibles divisiones entre cero.
- **Filtro – Estado del pedido.** Segmentador que permite filtrar dinámicamente todas las visualizaciones de la página según el estado de los pedidos.

En cuanto a las visualizaciones analíticas, destacan dos gráficos principales.

Por un lado, se implementa un gráfico de comparación de facturación entre el año seleccionado y el ejercicio anterior. Esta visualización permite analizar simultáneamente la evolución mensual de la facturación y la variación porcentual respecto al año previo. Para ello se desarrollan tres medidas DAX: *Facturación año actual*, *Facturación año anterior* y *Diferencia %*. Las dos primeras utilizan la función `CALCULATE` junto con `REMOVEFILTERS` para obtener la facturación correspondiente a cada ejercicio independientemente del contexto de filtrado activo. A partir de ambas medidas se calcula posteriormente

la variación porcentual mediante la función `DIVIDE`, evitando errores en aquellos periodos donde no existe facturación en el año de referencia.

The image shows a configuration panel for a dashboard chart. It is divided into three sections: 'Eje X' (X-axis), 'Eje Y de columna' (Column Y-axis), and 'Eje Y de línea' (Line Y-axis). Each section contains a list of variables with expand/collapse and delete icons.

Eje X	
full_date	▼ ✕
Mes	✕

Eje Y de columna	
Facturación 2018	▼ ✕
Facturación 2017	▼ ✕

Eje Y de línea	
Diferencia %	▼ ✕

Figura 54: Variables utilizadas en el gráfico comparativo de facturación anual. (Elaboración propia)

Por otro lado, se desarrolla un gráfico de distribución por estado del pedido, cuyo objetivo es representar el número de pedidos asociados a cada fase de su ciclo de vida. Esta visualización permite evaluar rápidamente la situación operativa de la plataforma y detectar posibles concentraciones de pedidos en estados concretos. Para su construcción se utiliza la medida *Nº Pedidos* junto con el atributo `order_status` procedente del modelo dimensional.

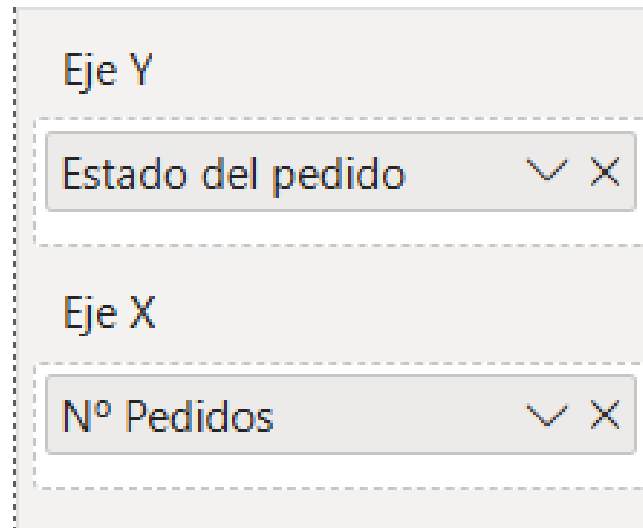


Figura 55: Variables utilizadas en el gráfico de distribución por estado del pedido. (Elaboración propia)

5.4.2. Productos y Categorías

La página de Productos y Categorías tiene como objetivo analizar el rendimiento del catálogo de productos de la plataforma, identificando los artículos y categorías con mayor facturación e ítems vendidos, así como la evolución temporal de los ingresos a lo largo del periodo analizado. Esta página responde a preguntas de negocio orientadas a la gestión del catálogo y a la planificación comercial.

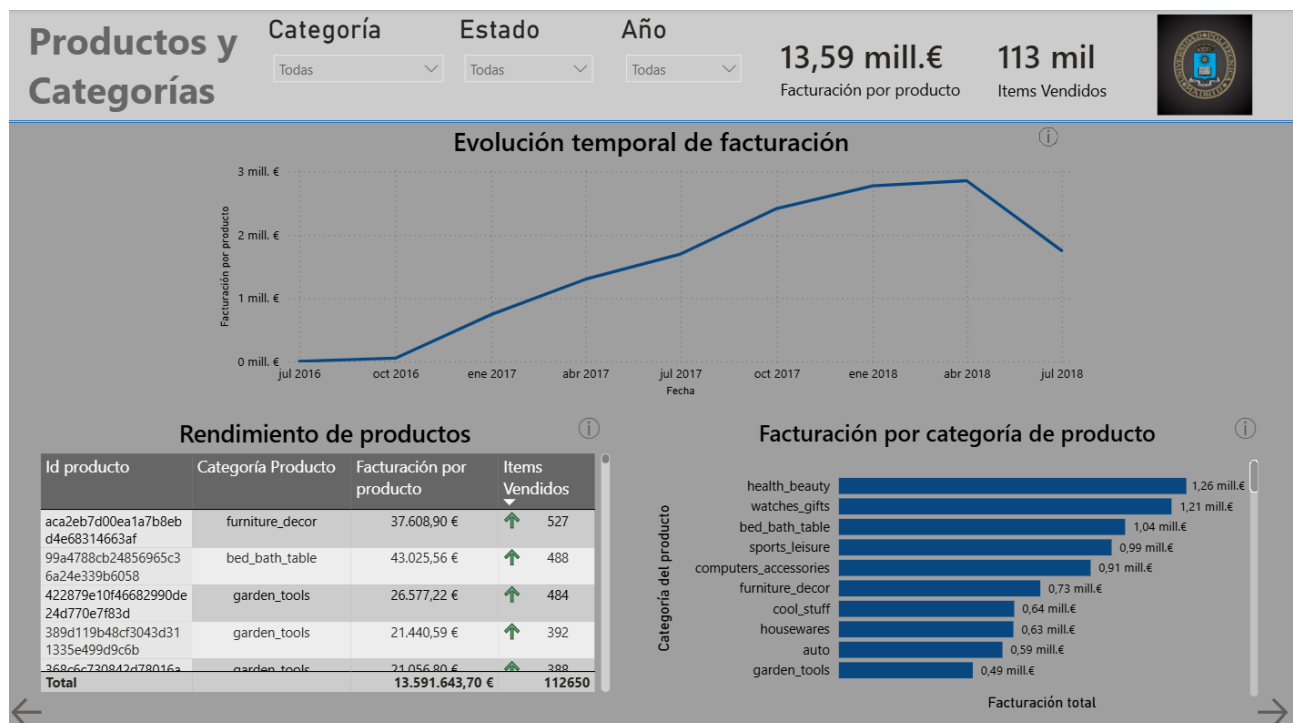


Figura 56: Página Productos y Categorías del informe desarrollado en Power BI. (Elaboración propia)

En primer lugar, se implementan los principales indicadores KPI y filtros de la página:

- **Tarjeta – Facturación por producto.** Representa la suma total del precio de venta de todas las líneas de pedido incluidas en el contexto de filtrado activo. Se implementa mediante la función DAX SUM sobre el atributo `price` de la tabla `fact_sales`.
- **Tarjeta – Ítems vendidos.** Muestra el número total de líneas de pedido, es decir, el número de unidades vendidas. Se implementa mediante la función DAX COUNTROWS sobre la tabla `fact_sales`. A diferencia de la medida *Nº Pedidos*, no realiza deduplicación sobre `order_id`, ya que su objetivo es contabilizar unidades vendidas y no pedidos distintos.
- **Filtros – Categoría, Estado y Año.** Tres segmentadores situados en la parte superior de la página que permiten filtrar dinámicamente todas las visualizaciones. El filtro de categoría utiliza el atributo `product_category_name_english` de `dim_product`, el filtro de estado utiliza `customer_state` de `dim_customer` y el filtro de año utiliza el atributo `year` de `dim_date`.

En cuanto a las visualizaciones analíticas, la página se organiza en tres gráficos complementarios que permiten analizar el catálogo desde distintas perspectivas.

En primer lugar, se encuentra el gráfico de evolución temporal de la facturación, situado en la parte superior de la página. Esta visualización representa la facturación acumulada a lo largo del tiempo y permite identificar tendencias de crecimiento, periodos de mayor actividad y posibles patrones estacionales.



Figura 57: Variables utilizadas en el gráfico de evolución temporal de la facturación. (Elaboración propia)

El eje X se construye a partir del atributo `full_date` de la dimensión fecha, mientras que el eje Y utiliza la medida *Facturación por producto*. Gracias a esta configuración es posible observar la evolución de los ingresos durante todo el periodo analizado.

En segundo lugar, se desarrolla la tabla de rendimiento de productos, situada en la zona inferior izquierda. Esta tabla muestra para cada producto su identificador, categoría, facturación acumulada e

ítems vendidos. Con ello se facilita la identificación de los productos con mejor rendimiento económico. La tabla incorpora además formato condicional sobre la columna de ítems vendidos, utilizando indicadores visuales que permiten identificar rápidamente aquellos productos cuyo volumen de ventas se sitúa por encima o por debajo de la media.

Iconos - Items Vendidos ×

Estilo de formato: Reglas ▼

Aplicar a: Solo valores ▼

¿En qué campo debemos basar esto?: Items Vendidos ▼

Diseño de los iconos: A la izquierda de los datos ▼

Alineación de los iconos: Superior ▼

Estilo: ▼ (Red arrow down, Yellow arrow right, Green arrow up)

Reglas: [1] Inversión del orden de los iconos + Nueva regla

Si el valor	>=	0	Número	y	<	101	Número	entonces	▼ (Red arrow down)	↑ ↓ ×
Si el valor	>=	101	Número	y	<	301	Número	entonces	▼ (Yellow arrow right)	↑ ↓ ×
Si el valor	>=	301	Número	y	<=	600	Número	entonces	▼ (Green arrow up)	↑ ↓ ×

[Más información sobre el formato condicional](#)

Aceptar Cancelar

Figura 58: Formato condicional aplicado a la tabla de rendimiento de productos. (Elaboración propia)

Por defecto, la tabla se encuentra ordenada de forma descendente según la facturación total, mostrando en las primeras posiciones los productos que generan mayores ingresos.

Columnas

- Id producto ▼ ×
- Categoría Producto ▼ ×
- Facturación por produ... ▼ ×
- Items Vendidos ▼ ×

Figura 59: Variables utilizadas en la tabla de rendimiento de productos. (Elaboración propia)

Por último, se implementa el gráfico de facturación por categoría de producto, situado en la parte inferior derecha de la página. Esta visualización muestra la facturación agregada para cada categoría de producto y permite identificar cuáles son las categorías con mayor contribución a los ingresos totales.

de la plataforma.

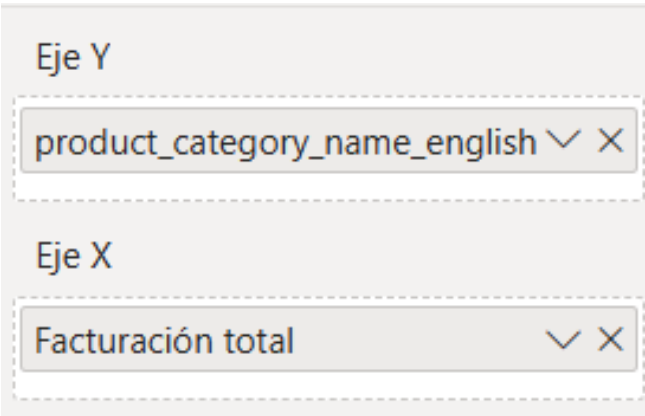


Figura 60: Variables utilizadas en el gráfico de facturación por categoría de producto. (Elaboración propia)

La combinación de esta visualización con el filtro de categorías permite explorar en detalle el comportamiento de segmentos específicos del catálogo y analizar su contribución relativa al negocio.

5.4.3. Clientes y Geografía

La página de Clientes y Geografía tiene como objetivo analizar la distribución geográfica de los clientes y su actividad de compra. Permite identificar los estados brasileños con mayor concentración de clientes y facturación, así como clasificar individualmente a cada cliente según su comportamiento de compra.



Figura 61: Página Clientes y Geografía del informe desarrollado en Power BI. (Elaboración propia)

En primer lugar, se implementan los principales indicadores KPI y filtros de la página:

- **Tarjeta – N° Clientes.** Muestra el número de clientes únicos presentes en el contexto de filtrado activo. Se implementa mediante la función DAX `DISTINCTCOUNT` sobre el campo `customer_unique_id` de `dim_customer`, garantizando que cada cliente real se contabilice una única vez independientemente del número de pedidos realizados.
- **Tarjeta – Ticket Medio Cliente.** Representa la facturación media por cliente, calculada dividiendo la facturación total entre el número de clientes únicos. Se implementa mediante la función DAX `DIVIDE`, reutilizando las medidas *Facturación por producto* y *N° Clientes*, lo que garantiza que ambas medidas compartan el mismo contexto de filtrado.
- **Filtros – Año y Estado Cliente.** Dos filtros situados en la parte superior de la página que permiten filtrar dinámicamente todas las visualizaciones. El filtro de año actúa sobre el campo `year` de `dim_date`, mientras que el filtro de estado cliente actúa sobre `customer_state` de `dim_customer`, permitiendo acotar el análisis a un periodo concreto o a un estado brasileño específico.

En cuanto a las visualizaciones analíticas, la página se organiza en tres gráficos complementarios que analizan la dimensión geográfica y el comportamiento individual de los clientes.

En primer lugar, se implementa un Treemap situado en la zona superior izquierda de la página, que representa la facturación total agrupada por estado brasileño mediante rectángulos de tamaño proporcional al valor. Se configura con `customer_state` de `dim_customer` como categoría y la medida *Facturación por producto* como valor. Esta visualización permite identificar de forma inmediata la concentración geográfica de los ingresos, destacando São Paulo como el estado dominante.

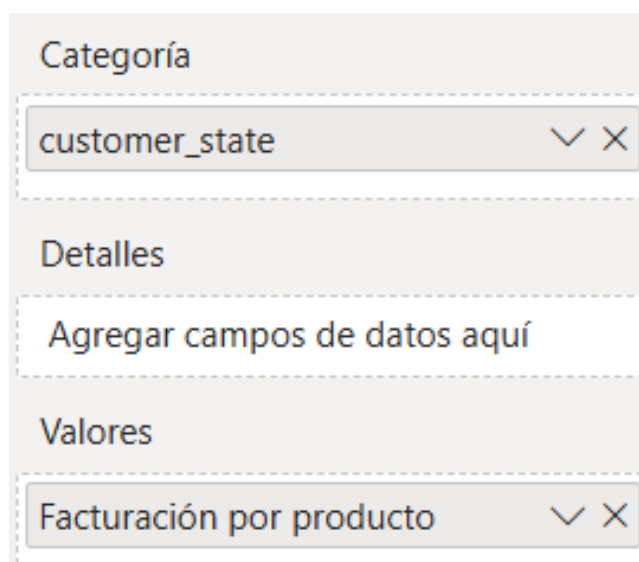


Figura 62: Variables utilizadas en el Treemap de facturación por estado. (Elaboración propia)

En segundo lugar, se desarrolla un gráfico circular situado en la parte superior derecha, que representa la distribución porcentual del número de clientes por estado brasileño. Se configura con `customer_state`

de `dim_customer` como categoría y la medida *Nº Clientes* como valor. Esta visualización complementa al Treemap, ya que permite comparar la distribución de clientes con la facturación: un estado puede concentrar muchos clientes, pero presentar un ticket medio inferior al de otros estados con menor volumen de clientes.



Figura 63: Variables utilizadas en el gráfico circular de distribución de clientes por estado. (Elaboración propia)

Por último, se incorpora una tabla de actividad de compra por cliente, en la que se muestran distintos atributos como el identificador del cliente, el estado, la ciudad, la facturación total, el número de pedidos y el tipo de cliente.

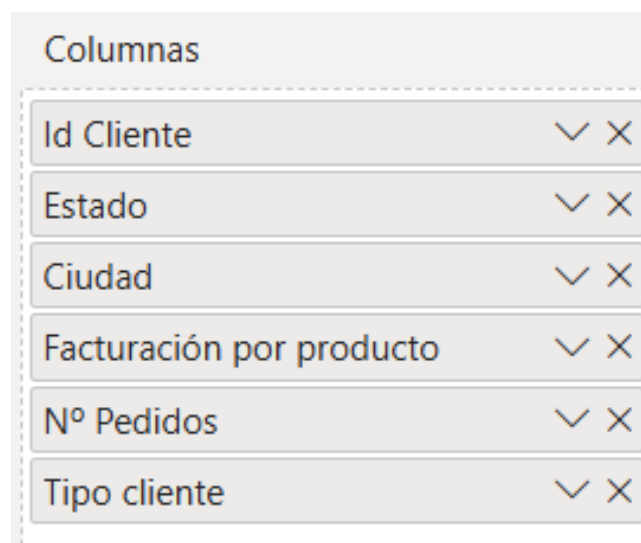


Figura 64: Variables utilizadas en la tabla de actividad de compra por cliente. (Elaboración propia)

Cabe destacar que la columna *Tipo cliente* incorpora un formato condicional con iconos, que clasifica visualmente a cada cliente como único o recurrente según el número de pedidos realizados. La configuración del formato condicional se basa en reglas aplicadas sobre la medida *Cliente Recurrente*, asignando un icono de diamante rojo a los clientes únicos y un icono de círculo verde a los clientes recurrentes.

Iconos - Tipo cliente ×

Estilo de formato: Reglas ▼ Aplicar a: Solo valores ▼

¿En qué campo debemos basar esto? ▼
Tipo cliente ▼

Diseño de los iconos: A la izquierda de los datos ▼ Alineación de los iconos: Superior ▼ Estilo: Personalizado ▼

Reglas ⌵ Inversión del orden de los iconos + Nueva regla

Si el valor es ▼	Único	entonces 🔴 ▼	↑ ↓ ×
Si el valor es ▼	Recurrente	entonces 🟢 ▼	↑ ↓ ×

[Más información sobre el formato condicional](#) Aceptar Cancelar

Figura 65: Formato condicional aplicado a la columna Tipo cliente. (Elaboración propia)

La medida *Cliente Recurrente* utiliza la función DAX `HASONEVALUE` para garantizar que la evaluación se realice en el contexto de un único cliente, devolviendo `BLANK()` cuando el contexto de filtrado incluye múltiples clientes. Cuando el contexto corresponde a un único cliente, compara el número de pedidos con 1 para determinar si el cliente ha comprado más de una vez.

5.4.4. Análisis de pagos

La página de Análisis de pagos tiene como objetivo analizar el comportamiento de los clientes en relación con los métodos de pago utilizados, el uso de financiación a plazos y la evolución temporal de los pagos. Esta página se alimenta directamente de la tabla de hechos `fact_payment`, que tiene granularidad de pago secuencial y permite que un mismo pedido aparezca en múltiples registros.

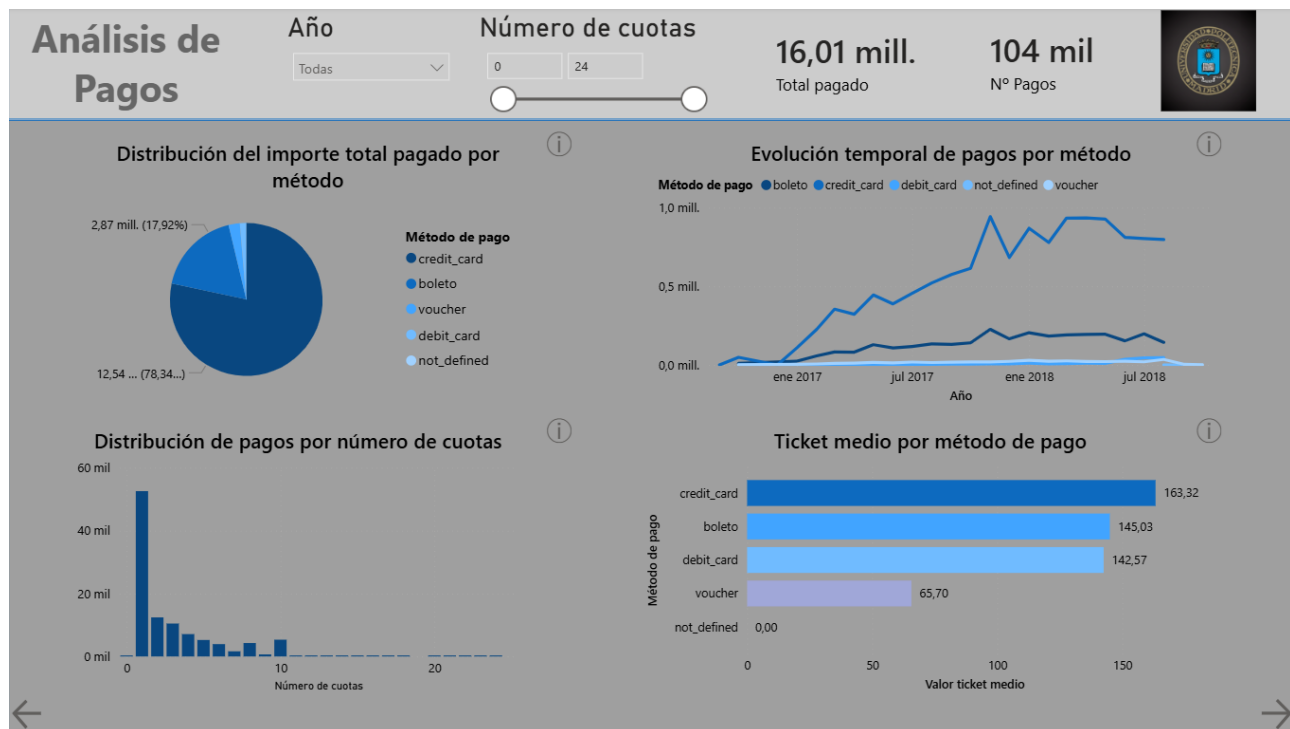


Figura 66: Página Análisis de pagos del informe desarrollado en Power BI. (Elaboración propia)

En primer lugar, se implementan los principales indicadores KPI y filtros de la página:

- **Tarjeta – Total Pagado.** Muestra la suma total del importe pagado en todos los registros de `fact_payment` incluidos en el contexto de filtrado activo. Se implementa mediante la función DAX SUM sobre el atributo `payment_value` de `fact_payment`.
- **Tarjeta – Nº pagos.** Muestra el número total de registros de pago. Se implementa mediante la función DAX COUNT sobre el atributo `order_id` en `fact_payment`. Esta métrica refleja el número de transacciones de pago individuales, no el número de pedidos, por lo que puede ser superior al número de pedidos si algunos se abonaron con múltiples métodos.
- **Filtros – Año y número de cuotas.** La página incorpora dos controles de filtrado. El filtro de año actúa sobre el campo `year` de `dim_date`. El filtro de número de cuotas se implementa como un control deslizante de rango que actúa sobre el campo `payment_installments` de `fact_payment`, permitiendo al usuario acotar el análisis a un rango concreto de cuotas, desde 0 hasta 24.

En cuanto a las visualizaciones analíticas, la página se organiza en cuatro gráficos que analizan la distribución, la evolución temporal y el valor medio de los pagos por método.

En primer lugar, situado en la zona superior izquierda, se implementa un gráfico circular que muestra el peso relativo de cada método de pago sobre el importe total pagado. Se configura con `payment_type` de `fact_payment` como leyenda y la suma de `payment_value` como valor. El dominio de la tarjeta de crédito, que representa el 78,34 % del total, es inmediatamente visible en esta representación.

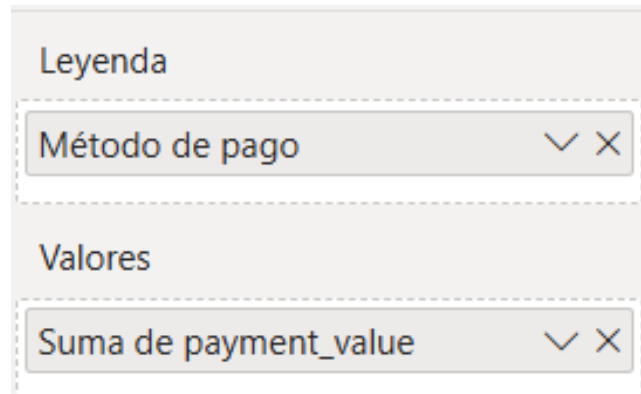


Figura 67: Variables utilizadas en el gráfico de distribución del importe pagado por método. (Elaboración propia)

En segundo lugar, situado en la zona superior derecha, se desarrolla un gráfico de líneas que muestra la evolución mensual del importe pagado dividido por método de pago. El eje X se configura con el campo `full_date` de `dim_date`, con jerarquía reducida a año y mes; el eje Y utiliza la suma de `payment_value`; y la leyenda emplea `payment_type`. Cada método de pago se representa mediante una línea de color diferente, lo que permite analizar simultáneamente la tendencia de crecimiento de cada método a lo largo del periodo.

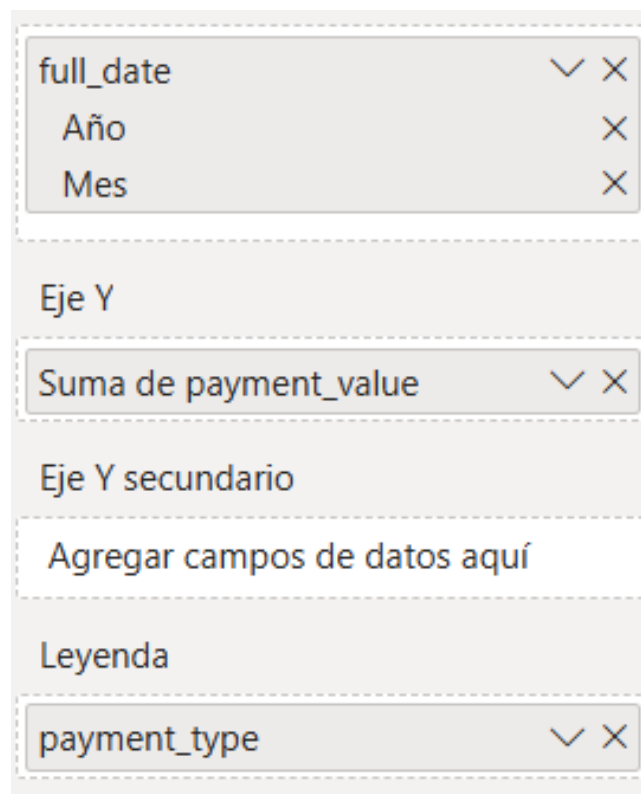
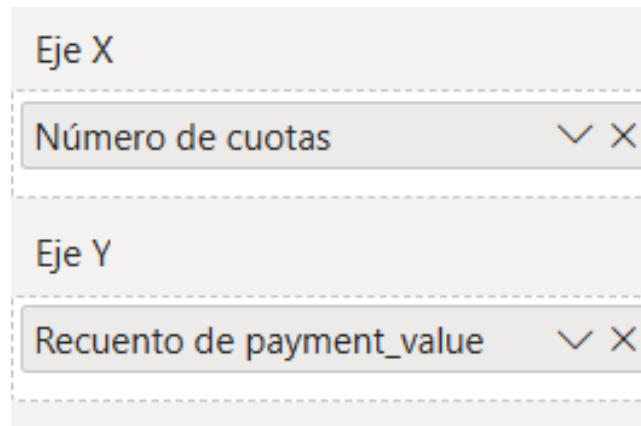


Figura 68: Variables utilizadas en el gráfico de evolución temporal de pagos por método. (Elaboración propia)

En tercer lugar, situado en la zona inferior izquierda, se incorpora un gráfico que muestra la distribu-

ción de frecuencias del número de cuotas utilizado en los pagos. El eje X se configura con el campo `payment_installments` de `fact_payment` y el eje Y con el recuento de `payment_value`. La distribución resultante muestra una concentración elevada en pagos de una sola cuota, con una cola larga hacia los pagos más fraccionados, lo que refleja un comportamiento de compra mayoritariamente al contado o en pocas cuotas.



Eje X

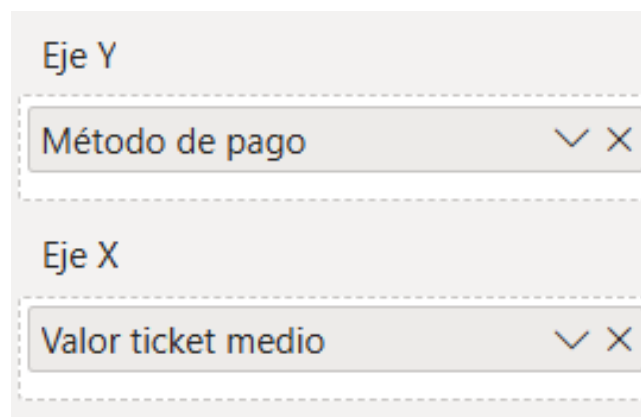
Número de cuotas

Eje Y

Recuento de payment_value

Figura 69: Variables utilizadas en el gráfico de distribución de pagos por número de cuotas. (Elaboración propia)

Por último, situado en la zona inferior derecha, se implementa un gráfico que muestra el importe medio por pago para cada método. El eje Y se configura con `payment_type` y el eje X con el promedio de `payment_value`. Esta visualización permite comparar el gasto medio asociado a cada forma de pago, siendo la tarjeta de crédito la que presenta el ticket más elevado, con un valor de 163,32 euros, seguida del boleto bancario y la tarjeta de débito. Por su parte, el voucher muestra un importe medio sensiblemente inferior, con un valor de 65,70 euros.



Eje Y

Método de pago

Eje X

Valor ticket medio

Figura 70: Variables utilizadas en el gráfico de importe medio por método de pago. (Elaboración propia)

5.4.5. Segmentación de clientes

La página de Segmentación de clientes constituye una vista de análisis avanzado dentro del informe. A diferencia del resto de páginas, que se alimentan directamente del Data Warehouse implementado

en MySQL, esta página utiliza como fuente de datos el archivo `customer_clusters.csv` generado por el script de clustering. Dicho archivo asigna a cada cliente uno de los tres segmentos identificados por el algoritmo K-Means: *Satisfechos*, *Insatisfechos* y *Premium*. El archivo se importa en Power BI como una tabla adicional independiente del modelo dimensional mediante la opción *Obtener datos* desde archivo CSV.

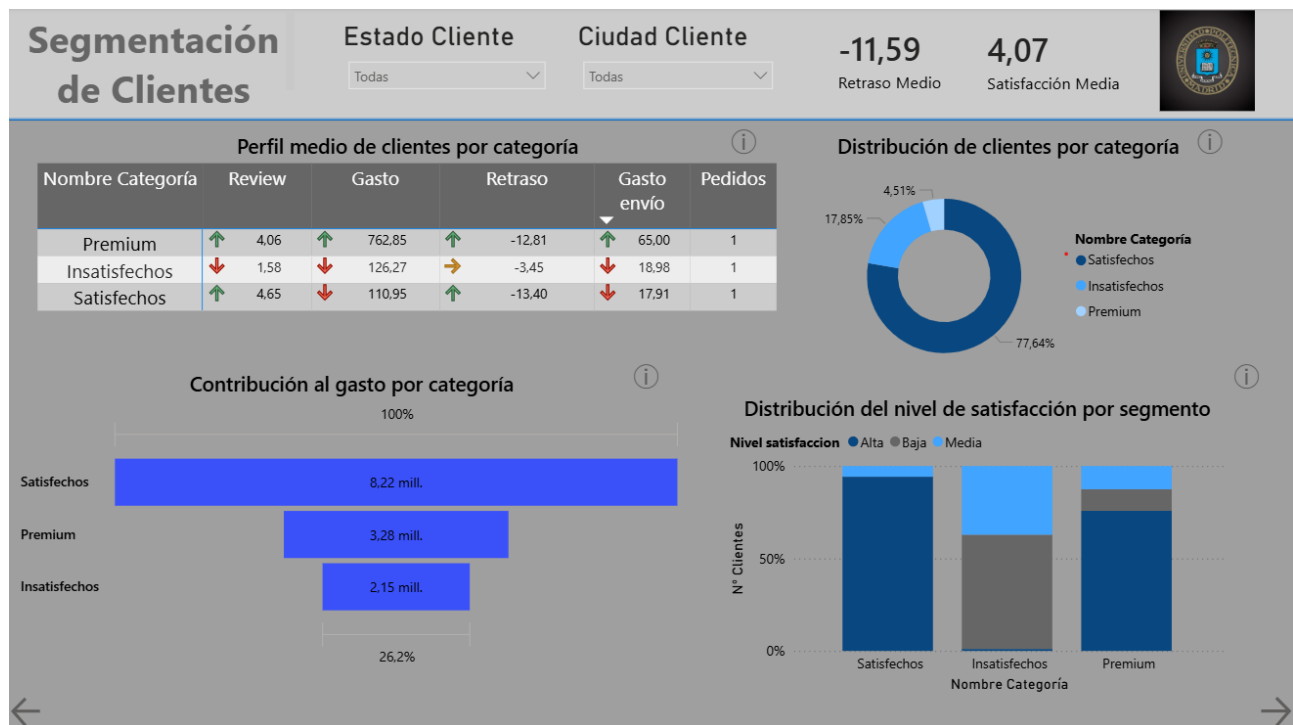


Figura 71: Página Segmentación de clientes del informe desarrollado en Power BI. (Elaboración propia)

Antes de implementar las visualizaciones, se crean dos columnas calculadas directamente sobre la tabla `customer_clusters` en Power BI, con el objetivo de enriquecer los resultados del clustering con etiquetas interpretables desde el punto de vista de negocio.

- **Columna calculada – Nombre Cluster.** Traduce el identificador numérico del clúster asignado por K-Means a su correspondiente etiqueta de negocio mediante la función DAX `SWITCH`. Esta columna constituye la base de todas las visualizaciones de la página, ya que actúa como categoría de segmentación en tablas y gráficos.
- **Columna calculada – Nivel satisfacción.** Clasifica a cada cliente en uno de tres niveles de satisfacción según su puntuación media de reseñas. Se define como *Alta* para valores iguales o superiores a 4, *Media* para puntuaciones entre 2 y 4 y *Baja* para valores inferiores a 2. Su implementación utiliza la función DAX `SWITCH(TRUE())`, evaluando las condiciones de forma secuencial hasta encontrar la primera que se cumple.

Tras esta preparación, se implementan los principales indicadores KPI y filtros de la página:

- **Tarjeta – Retraso Medio.** Muestra el promedio del campo `avg_delay` de la tabla `customer_clusters`.

El valor negativo indica que, en promedio, los pedidos se entregan antes de la fecha estimada. El valor global obtenido para toda la base de clientes es de -11,59 días.

- **Tarjeta – Satisfacción media.** Representa el promedio del campo `avg_review` de `customer_clusters`, mostrando la valoración media global de los clientes. El valor obtenido es de 4,07 sobre 5, lo que refleja un nivel elevado de satisfacción y guarda relación con el retraso medio negativo observado en la tarjeta anterior.
- **Filtros – Estado cliente y ciudad cliente.** Dos filtros situados en la parte superior de la página que permiten segmentar dinámicamente todas las visualizaciones según el estado y la ciudad del cliente. Ambos utilizan los atributos `customer_state` y `customer_city` de `dim_customer`, vinculados a la tabla `customer_clusters` mediante el campo `customer_unique_id`.

En cuanto a las visualizaciones analíticas, la página se organiza en cuatro gráficos que permiten analizar el perfil, la distribución y la contribución económica de cada segmento de clientes.

En primer lugar, situada en la zona superior izquierda, se implementa una tabla que muestra el perfil medio de cada segmento en las cinco variables utilizadas por el clustering. Las filas se configuran con la columna calculada *Nombre Cluster* y los valores con el promedio de los atributos calculados durante el proceso de clustering. La tabla incorpora formato condicional mediante iconos en las columnas de valores, permitiendo identificar visualmente si cada métrica presenta una tendencia positiva o negativa para el segmento correspondiente.

Por un lado, las variables relacionadas con satisfacción, gasto y gasto de envío comparten una misma configuración de formato condicional.

Iconos - Review

Estilo de formato: Reglas | Aplicar a: Solo valores

¿En qué campo debemos basar esto?: Review | Resumen: Promedio

Diseño de los iconos: A la izquierda de los datos | Alineación de los iconos: Superior | Estilo: [Iconos de tendencia]

Reglas: [Inversión del orden de los iconos] [Nueva regla]

Si el valor	>=	0	Porcentaje	y	<	33	Porcentaje	entonces	[Icono rojo hacia abajo]	[Icono verde hacia arriba]	[Icono gris hacia abajo]
Si el valor	>=	33	Porcentaje	y	<	67	Porcentaje	entonces	[Icono amarillo hacia la derecha]	[Icono verde hacia arriba]	[Icono gris hacia abajo]
Si el valor	>=	67	Porcentaje	y	<=	100	Porcentaje	entonces	[Icono verde hacia arriba]	[Icono verde hacia arriba]	[Icono gris hacia abajo]

[Más información sobre el formato condicional](#) [Aceptar] [Cancelar]

Figura 72: Formato condicional aplicado a las variables de satisfacción y gasto. (Elaboración propia)

Por otro lado, la variable de retraso utiliza un formato específico adaptado a valores negativos, ya que en este caso un menor retraso representa una situación favorable.

Iconos - Retraso

Estilo de formato: Reglas (▼) Aplicar a: Solo valores (▼)

¿En qué campo debemos basar esto?: Retraso (▼) Resumen: Promedio (▼)

Diseño de los iconos: A la izquierda de los datos (▼) Alineación de los iconos: Superior (▼) Estilo: Personalizado (▼)

Reglas: TI Inversión del orden de los iconos + Nueva regla

Si el valor	>=	0	Porcentaje	y	<	33	Porcentaje	entonces	↑	↓	×
Si el valor	>=	67	Porcentaje	y	<=	100	Porcentaje	entonces	→	↓	×

[Más información sobre el formato condicional](#) Aceptar Cancelar

Figura 73: Formato condicional aplicado a la variable de retraso medio. (Elaboración propia)

En segundo lugar, situado en la zona superior derecha, se implementa un gráfico de anillos que representa la distribución porcentual del número de clientes entre los distintos segmentos. Se configura con la columna *Nombre Cluster* como leyenda y el recuento de *customer_unique_id* como valor. El gráfico refleja el predominio del segmento *Satisfechos*, que concentra el 77,64 % de los clientes, seguido de *Insatisfechos* con un 17,85 % y *Premium* con un 4,51 %.

Leyenda

Nombre Categoría (▼) (×)

Valores

Recuento de customer_unique_id... (▼) (×)

Figura 74: Variables utilizadas en el gráfico de distribución de segmentos de clientes. (Elaboración propia)

En tercer lugar, situado en la zona inferior izquierda, se desarrolla un gráfico de embudo que representa la facturación total aportada por cada segmento. Se configura con *Nombre Cluster* como categoría y la suma de *total_spent* como valor. A pesar de representar únicamente el 4,51 % de los clientes, el segmento *Premium* aporta 3,28 millones de euros, frente a los 8,22 millones del segmento *Satisfechos*, que concentra la mayor parte de la base de clientes.



Figura 75: Variables utilizadas en el gráfico de facturación por segmento. (Elaboración propia)

Por último, situado en la zona inferior derecha, se implementa un gráfico de columnas 100 % apiladas que muestra la composición interna de cada segmento según el nivel de satisfacción de sus clientes, calculado a partir de la columna *Nivel satisfacción*. El eje X se configura con *Nombre Cluster*, el eje Y con el número de clientes y la leyenda con *Nivel satisfacción*. Las columnas se representan en porcentaje sobre el total de cada segmento, permitiendo comparar la proporción de clientes con satisfacción alta, media y baja independientemente del tamaño absoluto de cada grupo.



Figura 76: Variables utilizadas en el gráfico de composición de satisfacción por segmento. (Elaboración propia)

5.4.6. Predicción de satisfacción

La página de Predicción de satisfacción constituye la vista dedicada al análisis predictivo del informe y permite visualizar e interpretar los resultados de los dos modelos de machine learning desarrollados. Al igual que ocurre con la página de segmentación de clientes, esta sección no se alimenta directamente del Data Warehouse implementado en MySQL, sino de cuatro archivos CSV generados por el script de predicción de reseñas: `predicciones_modelos.csv`, `comparativa_modelos.csv`, `importancia_variables.csv` y `coeficientes_regresion_lineal.csv`. Todos ellos se importan en

Power BI como tablas adicionales independientes del modelo dimensional.

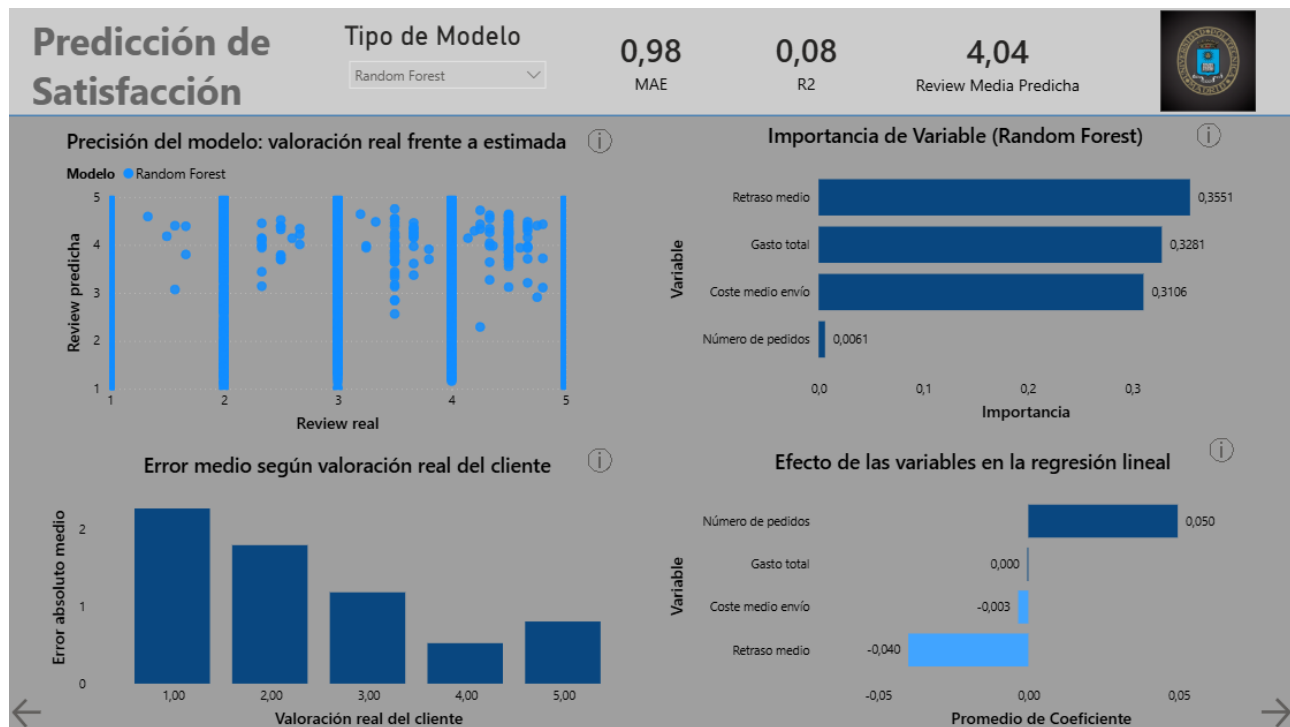


Figura 77: Página Predicción de satisfacción del informe desarrollado en Power BI. (Elaboración propia)

Antes de implementar las visualizaciones, se crean dos columnas calculadas sobre la tabla `predicciones_modelos`, necesarias para el gráfico de error medio.

- **Columna calculada – ReviewRealAgrupada.** Redondea la valoración real de cada registro al entero más cercano para agrupar las predicciones según niveles discretos de puntuación (1, 2, 3, 4 y 5). La función DAX `MAX` garantiza un valor mínimo de 1 en caso de redondeos extremos.
- **Columna calculada – ErrorAbsolutoCol.** Calcula el error absoluto de cada predicción individual como la diferencia en valor absoluto entre la valoración real y la predicha. Esta columna se utiliza posteriormente para calcular el MAE desagregado por nivel de puntuación real.

En primer lugar, se implementan los principales indicadores KPI y filtros de la página:

- **Tarjeta – MAE.** Muestra el error absoluto medio del modelo seleccionado, leído directamente del atributo `MAE` de la tabla `comparativa_modelos`. Los valores obtenidos son 1,0137 para la Regresión Lineal y 0,9813 para el Random Forest.
- **Tarjeta – R^2 .** Representa el coeficiente de determinación del modelo seleccionado, obtenido del atributo `R2` de `comparativa_modelos`. Los valores son 0,0935 para la Regresión Lineal y 0,0771 para el Random Forest.
- **Tarjeta – Review Media Predicha.** Muestra el promedio de las valoraciones predichas por el modelo seleccionado sobre el conjunto de prueba. Se implementa mediante una medida DAX basada en la función `AVERAGE` sobre el campo `Review predicha` de la tabla `predicciones_modelos`.

- **Filtro – Tipo de Modelo.** Segmentador que permite alternar entre los resultados de la Regresión Lineal y el Random Forest, actuando sobre el atributo **Modelo** de la tabla **predicciones_modelos**. Al seleccionar un modelo, todas las métricas y gráficos dependientes se actualizan dinámicamente para reflejar los resultados correspondientes.

En cuanto a las visualizaciones analíticas, la página se organiza en cuatro gráficos orientados a evaluar la precisión del modelo, la distribución del error y la relevancia de las variables explicativas.

En primer lugar, situado en la zona superior izquierda, se implementa un gráfico de dispersión que representa la relación entre la valoración real de cada cliente del conjunto de prueba y la valoración predicha por el modelo. El eje X se configura con el campo **Review real** de **predicciones_modelos**, el eje Y con **Review predicha** y la leyenda con el campo **Modelo**. En un modelo perfecto, todos los puntos se alinearían sobre la diagonal, por lo que la dispersión vertical de los puntos refleja directamente el error de predicción. La concentración de observaciones en los extremos de la escala pone de manifiesto la distribución bimodal de las valoraciones presentes en el dataset.

Valores

Agregar campos de datos aquí

Eje X

Review real

Eje Y

Review predicha

Leyenda

Modelo

Figura 78: Variables utilizadas en el gráfico de dispersión de predicción de satisfacción. (Elaboración propia)

En segundo lugar, situado en la zona superior derecha, se desarrolla un gráfico de barras horizontales que muestra la importancia relativa de cada variable explicativa en el modelo Random Forest. El eje Y se configura con el campo **Variable** de la tabla **importancia_variables** y el eje X con el promedio del campo **Importancia**. Los resultados reflejan que el retraso medio, el gasto total y el coste medio de envío presentan una importancia elevada y similar, mientras que el número de pedidos contribuye de forma prácticamente nula.

The form consists of two sections. The first section, labeled 'Eje Y', contains a dropdown menu with the text 'Variable' and a close button 'X'. The second section, labeled 'Eje X', contains a dropdown menu with the text 'Importancia' and a close button 'X'.

Figura 79: Variables utilizadas en el gráfico de importancia de variables del modelo Random Forest. (Elaboración propia)

En tercer lugar, situado en la zona inferior izquierda, se implementa un gráfico de columnas que representa el error absoluto medio del modelo desagregado por nivel de puntuación real. El eje X utiliza la columna calculada **ReviewRealAgrupada** y el eje Y el promedio de la columna **ErrorAbsolutoCol**. Esta visualización permite identificar para qué niveles de valoración el modelo presenta una mayor o menor precisión. Los resultados muestran que el error es superior en las valoraciones extremas de 1 y 2 estrellas, donde la predicción resulta más compleja debido a su menor frecuencia en el conjunto de entrenamiento.

The form consists of two sections. The first section, labeled 'Eje X', contains a dropdown menu with the text 'ReviewRealAgrupada' and a close button 'X'. The second section, labeled 'Eje Y', contains a dropdown menu with the text 'Promedio de ErrorAbsolutoCol' and a close button 'X'.

Figura 80: Variables utilizadas en el gráfico de error absoluto medio por valoración real. (Elaboración propia)

Por último, situado en la zona inferior derecha, se implementa un gráfico de barras que muestra los coeficientes obtenidos por el modelo de regresión lineal para cada variable explicativa. El eje Y se configura con el campo **Variable** de la tabla **coeficientes_regresion_lineal** y el eje X con el promedio del campo **Coeficiente**. Las barras que se extienden hacia la derecha representan variables con efecto positivo sobre la satisfacción, mientras que las que se desplazan hacia la izquierda representan efectos negativos. El gasto total presenta un coeficiente prácticamente nulo, confirmando que no tiene una influencia lineal significativa sobre la valoración del cliente.

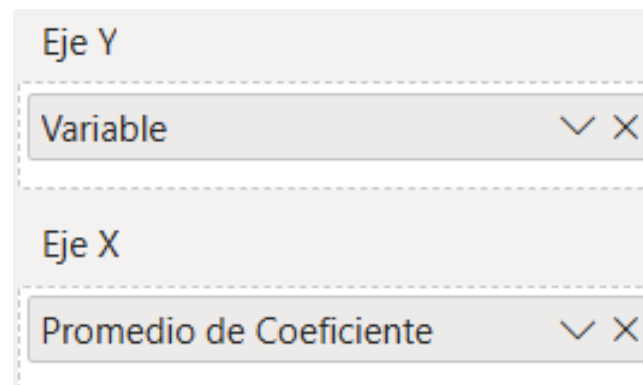


Figura 81: Variables utilizadas en el gráfico de coeficientes de regresión lineal. (Elaboración propia)

5.4.7. Navegación e interactividad

Además de las visualizaciones analíticas, el informe incorpora un conjunto de elementos de navegación e interactividad que mejoran la experiencia del usuario y facilitan la exploración de los datos. Estos elementos se implementan mediante las funcionalidades de botones, marcadores y el panel de selección de Power BI.

La página de Visión General incorpora una barra de navegación superior compuesta por cinco botones, cada uno asociado a una de las páginas principales del informe. Cada botón se configura como un botón de navegación de página en Power BI, de forma que el usuario puede acceder directamente a la sección correspondiente sin necesidad de utilizar los controles de navegación predeterminados de la herramienta.

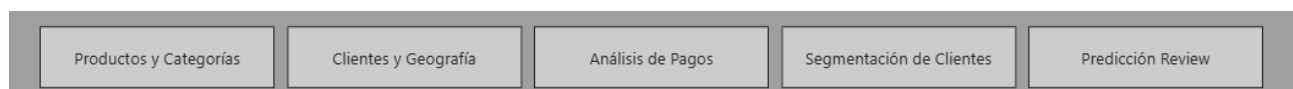


Figura 82: Barra de navegación superior implementada en la página Visión General. (Elaboración propia)

El resto de páginas incorporan dos flechas de navegación situadas en los extremos inferior izquierdo y derecho. La flecha izquierda permite regresar a la página anterior y la flecha derecha avanzar a la siguiente, siguiendo el orden secuencial del informe. Al igual que ocurre con la barra principal, estas flechas se implementan mediante botones de navegación de página.

La combinación de la barra de navegación principal y las flechas de avance y retroceso permite que el usuario pueda desplazarse por el informe tanto de forma directa como secuencial.



Figura 83: Flechas de navegación implementadas en las páginas del informe. (Elaboración propia)

Por otro lado, cada visualización incorpora un botón de información representado mediante un icono situado en la esquina superior derecha del gráfico. Al pulsarlo, aparece un rectángulo redondeado superpuesto sobre la visualización que describe brevemente el contenido y el objetivo analítico del

gráfico correspondiente. Este rectángulo incluye un botón de cierre con el símbolo X que permite ocultarlo y volver a la visualización original.

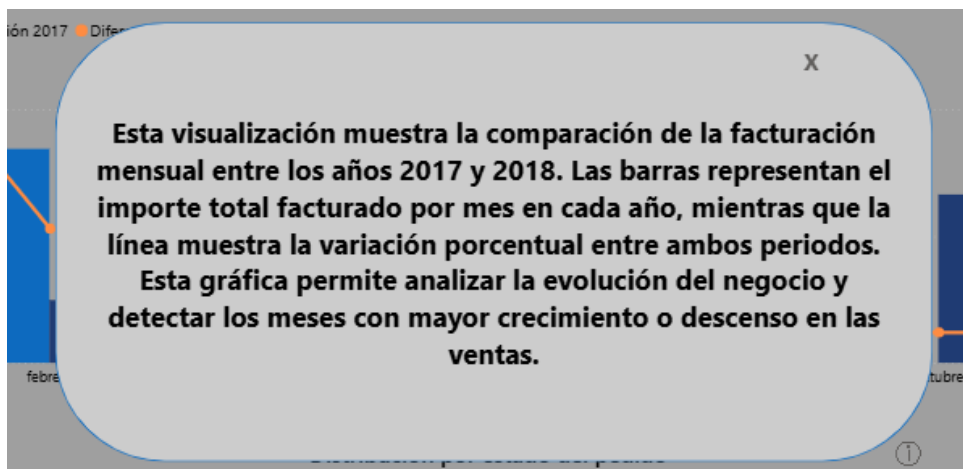


Figura 84: Ejemplo de ventana de información implementada sobre visualización facturación 2018 VS 2017 de la página Visión General. (Elaboración propia)

Este mecanismo se implementa mediante la funcionalidad de marcadores y el panel de selección de Power BI. En primer lugar, se agrupan el rectángulo informativo, el texto descriptivo y el botón de cierre en un único grupo dentro del panel de selección.

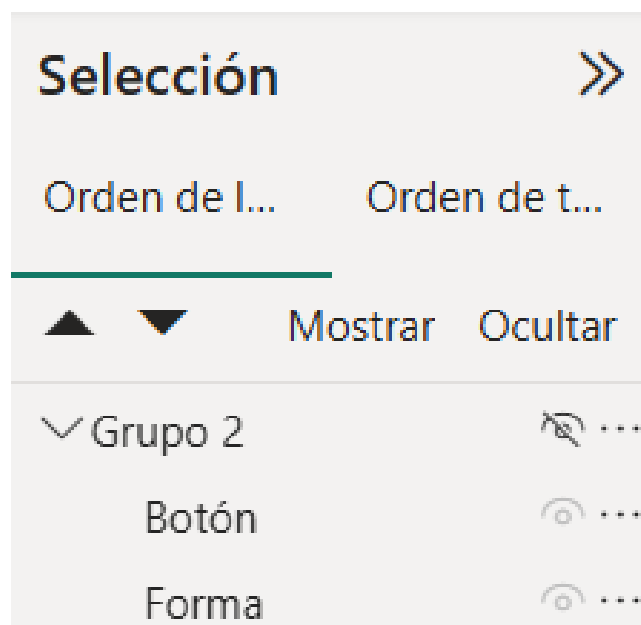


Figura 85: Grupo de elementos creado en el panel de selección. (Elaboración propia)

El panel de selección permite mostrar u ocultar elementos mediante el icono del ojo. Aprovechando esta funcionalidad, se crean dos marcadores distintos: uno denominado *Ocultar*, con el grupo oculto, y otro denominado *Mostrar*, con el grupo visible.

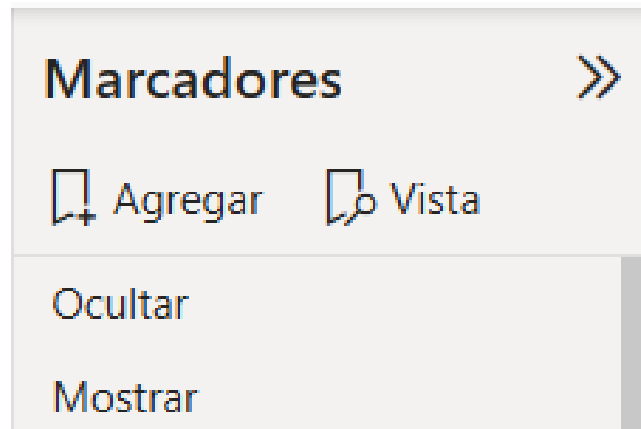


Figura 86: Marcadores utilizados para mostrar y ocultar las ventanas informativas. (Elaboración propia)

Una vez creados los marcadores, el botón de información se configura desde el menú *Acción*, estableciendo el tipo de acción como *Marcador* y asociándolo al marcador *Mostrar*.

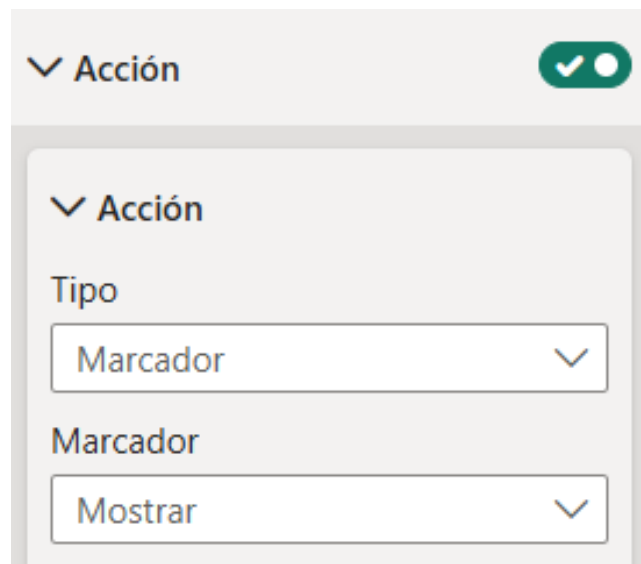
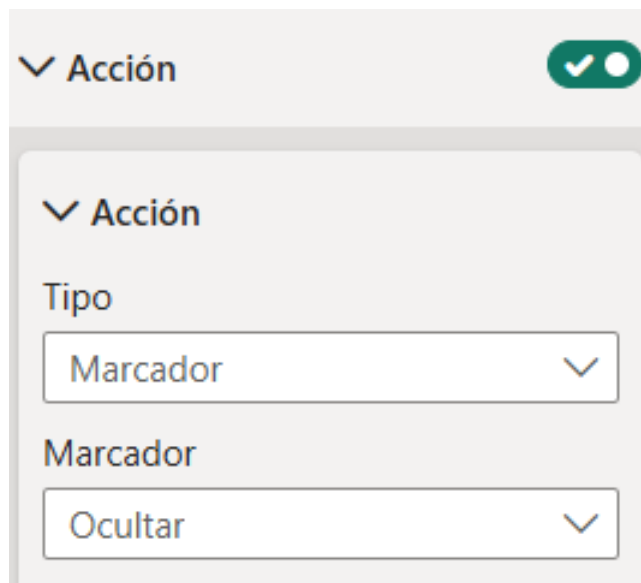


Figura 87: Configuración del botón de información mediante acciones de marcador. (Elaboración propia)

Finalmente, el botón de cierre con el símbolo X se configura de forma análoga, asignándole el marcador *Ocultar* para cerrar la ventana informativa y restaurar la vista original del gráfico.



The image shows a configuration modal for a button action. At the top, there is a header bar with a dropdown arrow and the text 'Acción' on the left, and a green toggle switch with a white checkmark on the right. Below the header, the modal contains a section titled 'Acción' with a dropdown arrow. Under this section, there are two configuration items: 'Tipo' with a dropdown menu showing 'Marcador' and a dropdown arrow, and 'Marcador' with a dropdown menu showing 'Ocultar' and a dropdown arrow.

Figura 88: Configuración del botón de cierre mediante acciones de marcador. (Elaboración propia)

6. Resultados y conclusiones

Tras el desarrollo de toda la implementación del proyecto, en este punto se recogen los principales resultados obtenidos a lo largo del proyecto y las conclusiones que se derivan tanto del sistema desarrollado como del análisis del dataset de Olist.

6.1. Resultados obtenidos

El desarrollo del proyecto ha dado como resultado un sistema de Business Intelligence funcional y completo, capaz de cubrir el ciclo de vida del dato desde su origen hasta su explotación analítica. La siguiente tabla resume los principales componentes desarrollados y los resultados más relevantes obtenidos.

Componente	Resultado
Dataset procesado	7 archivos CSV, aproximadamente 100.000 pedidos correspondientes al periodo 2016–2018.
Data Warehouse	6 tablas (3 dimensiones y 3 hechos) implementadas sobre MySQL.
Proceso ETL	6 scripts desarrollados en Python para la carga automatizada del Data Warehouse.
Informe Power BI	6 páginas interactivas con filtros dinámicos y navegación entre secciones.
Segmentación de clientes	3 segmentos identificados mediante K-Means (Satisfechos, Insatisfechos y Premium).
Modelos predictivos	Regresión lineal ($MAE = 1,0137$; $R^2 = 0,0935$) y Random Forest ($MAE = 0,9813$; $R^2 = 0,0771$).

Cuadro 24: Resumen de los principales resultados obtenidos

Desde el punto de vista analítico, el sistema permite responder a las principales preguntas de negocio definidas en los objetivos del proyecto. Los hallazgos más relevantes obtenidos a partir del análisis del dataset son los siguientes:

6.1.1. Ventas y facturación

- **Crecimiento sostenido.** La facturación de Olist experimentó un crecimiento continuo desde mediados de 2016 hasta principios de 2018, alcanzando su pico máximo en noviembre de 2017. Por otro lado, encontramos una facturación cercana al pico en el primer trimestre de 2018 y, a partir de este punto, se observa una caída, si bien el dataset no cubre el año completo y esta tendencia debe interpretarse con cautela.
- **Categorías más rentables.** Las categorías `health_beauty` y `watches_gift` lideran la facturación con 1,26 y 1,21 millones de euros respectivamente, seguidas de `bed_bath_table` y `sports_leisure`. Estas cuatro categorías concentran una parte significativa de los ingresos totales.

- **Peso del coste de envío.** El coste de envío representa de media el 16,57 % de la facturación total, un porcentaje relevante que justifica su inclusión como variable explicativa en los modelos de satisfacción.

6.1.2. Clientes y geografía

- **Concentración geográfica.** El estado de São Paulo (SP) concentra el 54,39 % de los clientes y la mayor parte de la facturación, con 1,92 millones de euros, seguido a distancia por Río de Janeiro (RJ) y Minas Gerais (MG). Esta concentración refleja la distribución demográfica y económica de Brasil.
- **Perfil de compra.** La gran mayoría de los clientes son compradores únicos que realizan un solo pedido en el periodo analizado. El ticket medio por cliente es de 141,44 euros, con variaciones significativas entre estados.
- **Segmentación.** El análisis de clustering identifica tres perfiles de clientes bien diferenciados. El 77,6 % son clientes satisfechos con gasto moderado, el 17,9 % son clientes insatisfechos que perciben retrasos mayores en sus entregas, y el 4,5 % son clientes Premium con un gasto casi siete veces superior a la media.

6.1.3. Pagos

- **Dominio de la tarjeta de crédito.** El 78,34 % del importe total pagado se abona mediante tarjeta de crédito, con un ticket medio de 163,32 euros. El boleto bancario, método de pago típico en Brasil, representa el 17,92 % restante, con un ticket medio de 145,03 euros.
- **Uso de financiación.** Una proporción relevante de los pagos con tarjeta de crédito se fracciona en cuotas, con una distribución que muestra que la mayoría de los pedidos se pagan en una o dos cuotas, aunque existen casos de hasta 24 cuotas para importes elevados.

6.1.4. Satisfacción del cliente

- **Valoración media elevada.** La puntuación media global de las reseñas es de 4,07 sobre 5, lo que refleja un nivel general de satisfacción alto. Sin embargo, la distribución es bimodal, es decir, existe un grupo significativo de valoraciones extremas de 1 estrella junto a una mayoría de valoraciones máximas de 5 estrellas.
- **El retraso como principal factor.** Tanto el modelo de Regresión Lineal como el Random Forest identifican el retraso en la entrega como el factor operativo de mayor impacto negativo sobre la satisfacción. Los clientes del segmento insatisfechos presentan retrasos medios de -3,45 días, frente a -13,40 días de los satisfechos, confirmando que las entregas más adelantadas respecto a la fecha estimada generan una percepción más positiva.

6.2. Conclusiones

El desarrollo de este proyecto ha permitido extraer las siguientes conclusiones generales:

- **Viabilidad del enfoque metodológico.** La combinación de la metodología Kimball para el diseño del modelo dimensional, Python para el proceso ETL y Power BI para la visualización ha demostrado ser una arquitectura viable, reproducible y mantenible para proyectos de Business Intelligence de escala media.
- **Valor del modelo dimensional.** El Galaxy Schema diseñado ha permitido integrar tres procesos de negocio distintos, las ventas, los pagos y las reseñas, en un modelo coherente que facilita el análisis cruzado entre ellos. La separación entre hechos y dimensiones simplifica significativamente la construcción de las visualizaciones en Power BI.
- **Importancia de la calidad del dato.** El proceso ETL ha puesto de manifiesto la importancia de la limpieza y preparación de los datos, al tratarse de datos reales no preparados para el análisis. Aspectos aparentemente menores, como los errores tipográficos, los espacios invisibles en campos de texto o la correcta gestión de valores nulos, tienen un impacto directo en la integridad del modelo.
- **Límites de los modelos predictivos.** Los modelos de predicción de la satisfacción obtienen métricas de R^2 modestas, lo que evidencia que la satisfacción del cliente es una variable compleja que no puede explicarse completamente mediante los datos operativos disponibles. No obstante, los modelos confirman de forma consistente que el retraso en la entrega es el factor más determinante entre las variables analizadas.
- **Potencial de la segmentación.** El análisis de clustering ha revelado que una minoría del 4,5 % de los clientes concentra un gasto desproporcionadamente elevado, lo que tiene implicaciones directas para estrategias de fidelización y personalización de la oferta.

7. Análisis de impacto

Todo proyecto en el que está involucrada la informática, independientemente de su escala y finalidad, tiene implicaciones que van más allá del ámbito puramente técnico. Este capítulo analiza el impacto potencial del sistema de Business Intelligence desarrollado desde cuatro dimensiones: social, económica, medioambiental y ética. Aunque el proyecto tiene una finalidad académica, la arquitectura y las técnicas aplicadas son extrapolables a entornos reales de producción, lo que justifica un análisis de impacto riguroso y reflexivo.

7.1. Impacto social

El comercio electrónico ha transformado profundamente los patrones de consumo en países emergentes como Brasil, facilitando el acceso a productos y servicios a poblaciones geográficamente dispersas que anteriormente tenían un acceso limitado al comercio minorista tradicional. En este contexto, los sistemas de Business Intelligence aplicados al e-commerce tienen el potencial de generar un impacto social positivo en varios niveles.

En primer lugar, el análisis de la satisfacción del cliente que posibilita este sistema permite a las empresas identificar y corregir los factores que generan experiencias de compra negativas. Los resultados obtenidos en este proyecto confirman que el retraso en la entrega es el principal factor de insatisfacción, lo que proporciona a los operadores logísticos y a los vendedores información concreta y accionable para mejorar su servicio. Una mejora en la calidad del servicio beneficia directamente al consumidor final, especialmente en regiones con infraestructuras logísticas menos desarrolladas.

En segundo lugar, la segmentación de clientes desarrollada permite detectar grupos con comportamientos y necesidades diferenciadas. La identificación del segmento de clientes insatisfechos constituye el primer paso para implementar medidas correctoras dirigidas específicamente a este colectivo, como la mejora de las estimaciones de entrega o la gestión proactiva de incidencias.

Por último, desde una perspectiva más amplia, la democratización de las herramientas de análisis de datos mediante soluciones como Power BI permite que pequeñas y medianas empresas del sector del e-commerce puedan tomar decisiones basadas en datos sin necesidad de realizar grandes inversiones en infraestructura tecnológica, contribuyendo así a reducir la brecha competitiva entre grandes y pequeños operadores del mercado.

7.2. Impacto económico

El impacto económico de un sistema de Business Intelligence se materializa principalmente a través de la mejora en la toma de decisiones empresariales. En el caso de este proyecto, los análisis implementados tienen implicaciones económicas directas e indirectas para los distintos actores del ecosistema del comercio electrónico.

Desde el punto de vista de los vendedores, el sistema permite identificar las categorías de producto con mayor rentabilidad y aquellas con un mayor potencial de crecimiento. El análisis de la evolución temporal de la facturación facilita la planificación de la demanda y la gestión del inventario, reduciendo los costes asociados tanto a la sobreproducción como a la rotura de stock. Asimismo, el análisis del

coste de envío proporciona información relevante para la negociación con operadores logísticos y la optimización de los procesos de distribución.

Para la plataforma, la identificación del segmento Premium tiene un valor estratégico considerable. Retener a este 4,5 % de clientes de alto valor mediante programas de fidelización específicos puede generar un impacto desproporcionadamente elevado sobre los ingresos totales. De forma análoga, reducir la proporción de clientes insatisfechos mediante mejoras operativas en la cadena logística se traduce directamente en una menor tasa de abandono y en un incremento del valor del ciclo de vida del cliente. Desde una perspectiva más amplia, la metodología aplicada en este proyecto, basada en herramientas de código abierto como Python y `scikit-learn`, combinadas con MySQL y Power BI, demuestra que es posible construir un sistema de Business Intelligence de calidad profesional con una inversión tecnológica reducida. Esto tiene implicaciones económicas positivas para pequeñas y medianas empresas que no disponen del presupuesto necesario para implantar soluciones empresariales de Business Intelligence de alto coste.

7.3. Impacto medioambiental

El impacto medioambiental directo del proyecto desarrollado es reducido. El desarrollo y la ejecución del sistema se realizan sobre infraestructura local, sin recurrir a servicios cloud de alta demanda energética. El volumen de datos procesado, alrededor de 100.000 pedidos, es manejable en un entorno de escritorio sin necesidad de infraestructuras de computación de alto rendimiento, lo que mantiene el consumo energético asociado dentro de niveles ordinarios.

El entrenamiento de los modelos de machine learning implementados tiene un coste computacional bajo y no requiere el uso de unidades de procesamiento gráfico ni de clústeres de computación distribuida. En este sentido, el impacto medioambiental de los modelos desarrollados es significativamente inferior al de modelos de aprendizaje profundo o grandes modelos de lenguaje, cuyo entrenamiento conlleva un consumo energético considerable.

No obstante, si el sistema se desplegara en un entorno de producción real con cargas de datos periódicas y acceso multiusuario, su migración a una infraestructura cloud introduciría un consumo energético adicional que debería considerarse durante el diseño de la arquitectura. En ese escenario, la elección de proveedores cloud con compromisos de neutralidad de carbono y la optimización de los procesos ETL para reducir el tiempo de cómputo serían medidas adecuadas para minimizar el impacto medioambiental del sistema.

Desde una perspectiva indirecta, los sistemas de Business Intelligence aplicados a la cadena logística del comercio electrónico pueden contribuir a reducir el impacto medioambiental del sector. La optimización de las rutas de entrega, la reducción de los tiempos de almacenamiento mediante una mejor previsión de la demanda y la disminución de las devoluciones derivadas de la insatisfacción del cliente tienen implicaciones positivas sobre las emisiones asociadas al transporte y la gestión de residuos.

7.4. Impacto ético

El tratamiento de datos de clientes en sistemas de análisis plantea consideraciones éticas que deben abordarse de forma explícita, independientemente de la finalidad del proyecto. En el caso del presente

trabajo, el riesgo ético es limitado debido a la naturaleza pública y anonimizada del dataset utilizado, pero su análisis sigue siendo pertinente tanto por rigor académico como por las implicaciones que tendría una implementación real del sistema.

En cuanto a la privacidad de los datos, el Brazilian E-Commerce Public Dataset de Olist fue publicado por la propia empresa tras un proceso de anonimización que elimina cualquier referencia que permita identificar a clientes o vendedores individuales. Los identificadores de cliente presentes en el dataset son cadenas hash sin correspondencia con datos personales reales. En consecuencia, el proyecto no trata datos personales en el sentido del Reglamento General de Protección de Datos (RGPD) y no genera riesgos de privacidad para los individuos representados en los datos.

Respecto al uso de los modelos de machine learning desarrollados, es importante destacar que su finalidad es exclusivamente analítica y académica. Los modelos de predicción de la satisfacción del cliente y de segmentación no se utilizan para tomar decisiones automatizadas que afecten a las personas, como la denegación de servicios, la modificación de precios o la exclusión de clientes, sino para generar conocimiento sobre patrones de comportamiento agregados. Esta distinción entre análisis descriptivo y sistemas de decisión automatizada resulta fundamental desde el punto de vista ético.

No obstante, si el sistema se implementara en un entorno real, sería necesario considerar algunos riesgos éticos adicionales. La segmentación de clientes, por ejemplo, podría utilizarse de forma discriminatoria si se empleara para ofrecer condiciones de servicio diferentes a distintos grupos de clientes sin una justificación objetiva y transparente. Asimismo, los modelos predictivos podrían perpetuar sesgos presentes en los datos históricos si el dataset de entrenamiento no fuera representativo de toda la base de clientes. Estas consideraciones deberían abordarse mediante mecanismos de auditoría de los modelos, transparencia en los criterios de segmentación y supervisión humana de las decisiones derivadas del análisis.

En conjunto, el proyecto presenta un perfil ético adecuado para su finalidad académica. Las limitaciones identificadas no comprometen la validez del trabajo realizado, pero sí ponen de manifiesto aspectos que deberían abordarse antes de cualquier despliegue en producción con datos reales de clientes.

8. Líneas futuras

El sistema desarrollado constituye una base sólida sobre la que es posible realizar múltiples ampliaciones. A continuación, se describen las líneas de trabajo futuro más relevantes, organizadas por áreas de mejora.

8.1. Ampliación del modelo de datos

- **Incorporación de `dim_seller`.** El dataset de Olist incluye información sobre los vendedores, específicamente en el archivo `olist_sellers_dataset.csv`, que fue descartada del proyecto por criterio propio, ya que el análisis quería enfocarse más hacia el ciclo de vida de los usuarios, para facilitar la toma de decisiones para personas de negocio. Su incorporación permitiría analizar el rendimiento por vendedor, identificar los que generan mayor satisfacción o más retrasos, y enriquecer el modelo con una dimensión de análisis adicional.
- **Incorporación de datos de geolocalización.** El archivo `olist_geolocation_dataset.csv`, descartado por su elevado volumen y falta de contenido novedoso para el análisis, contiene coordenadas geográficas a nivel de código postal. Su integración permitiría análisis geoespaciales más precisos que los actualmente disponibles a nivel de estado, como mapas de calor de densidad de pedidos o análisis de tiempos de entrega por zona geográfica.
- **Implementación de dimensiones de cambio lento (SCD).** El modelo actual implementa una carga completa en cada ejecución del ETL. En un entorno productivo con actualizaciones incrementales sería necesario implementar dimensiones de cambio lento, conocidas en inglés como Slowly Changing Dimensions, para gestionar correctamente los cambios en los atributos de clientes y productos a lo largo del tiempo.

8.2. Mejora del proceso ETL

- **Implementación de un área de staging.** Aunque su ausencia está justificada en el contexto del proyecto, un entorno productivo con cargas incrementales se beneficiaría de un área de staging persistida en base de datos que facilitaría la auditoría, la recuperación ante fallos y la sincronización con múltiples fuentes de datos.
- **Automatización y orquestación.** El proceso ETL actual requiere la ejecución manual de cada script en el orden correcto. La integración de una herramienta de orquestación como Apache Airflow permitiría programar las cargas, gestionar las dependencias entre scripts de forma automática y monitorizar el estado del pipeline.
- **Gestión de cargas incrementales.** El modelo de recarga completa o full load actual es adecuado para un dataset histórico o estático, pero ineficiente para fuentes de datos en producción. La implementación de cargas incrementales basadas en marcas de tiempo reduciría significativamente el tiempo de ejecución del ETL.

8.3. Mejora de los modelos analíticos

- **Incorporación de nuevas variables explicativas.** Los modelos predictivos de satisfacción obtienen R^2 modestos con las variables operativas disponibles. La incorporación de variables de producto como categoría y número de fotos, o de variables derivadas como el tiempo de aprobación del pago, podría mejorar la capacidad explicativa de los modelos.
- **Experimentación con otros algoritmos.** Además de la regresión lineal y el Random Forest, técnicas como Gradient Boosting, redes neuronales o modelos de series temporales podrían ofrecer mejores resultados en la predicción de la satisfacción o en el análisis de la evolución temporal de las ventas.
- **Optimización del número de clústeres.** El valor $k = 3$ del algoritmo K-Means se eligió por criterios de interpretabilidad del negocio. Un análisis más riguroso mediante el método del codo o la métrica de la silueta podría determinar el número óptimo de clústeres desde un punto de vista estadístico.
- **Modelo de churn y propensión a la recompra.** Dado que la mayoría de los clientes realizan un único pedido, el desarrollo de modelos de propensión a la recompra o de predicción del valor de ciclo de vida del cliente aportaría información de alto valor para estrategias de retención y fidelización.

8.4. Mejora de la capa de visualización

- **Actualización automática del informe.** La conexión actual de Power BI en modo Import requiere una actualización manual del informe tras cada ejecución del ETL. La publicación del informe en Power BI con actualización programada permitiría mantener los datos siempre actualizados sin intervención manual.
- **Incorporación de alertas y métricas de seguimiento.** La plataforma Power BI permite configurar alertas automáticas cuando un KPI supera o cae por debajo de un umbral definido. Esta funcionalidad añadiría capacidad de monitorización proactiva al sistema, complementando el análisis exploratorio actual.
- **Desarrollo de una vista móvil.** Power BI da acceso a un editor de diseño para dispositivos móviles que permite crear vistas optimizadas para pantallas pequeñas. Su implementación mejoraría la accesibilidad del informe para usuarios que necesiten consultar los datos fuera de su puesto de trabajo.

Referencias

- [1] Statista. «Retail e-commerce sales worldwide from 2014 to 2027. »dirección: <https://www.statista.com/statistics/379046/worldwide-retail-e-commerce-sales/>
- [2] InovaLabs. «Big Data para PYMES. »dirección: <https://inovalabs.es/es/big-data-para-pymes/>
- [3] Piwik PRO. «Raw Data vs Processed Data. »dirección: <https://piwik.pro/glossary/raw-data-vs-processed-data/>
- [4] Shopify. «Shopify Reports and Analytics. »dirección: <https://help.shopify.com/en/manual/reports-and-analytics>
- [5] Google. «Looker Studio. »dirección: <https://cloud.google.com/looker-studio>
- [6] Microsoft. «¿Qué es Microsoft Fabric? »Dirección: <https://learn.microsoft.com/es-es/fabric/fundamentals/microsoft-fabric-overview>
- [7] Hiberus. «Business Intelligence. »dirección: <https://www.hiberus.com/crecemos-contigo/business-intelligence/>
- [8] IBM. «¿Qué es Big Data? »Dirección: <https://www.ibm.com/es-es/think/topics/big-data>
- [9] IntechOpen. «Business Intelligence and Analytics: From Big Data to Big Impact. »dirección: <https://www.intechopen.com/chapters/76332>
- [10] A. Kumar y S. Gupta, «Generative AI and Augmented Business Intelligence: Transforming Data Analytics,» *IEEE Access*, 2024, Accessed: 2026-06-04. DOI: 10.1109/ACCESS.2024.3351234 dirección: <https://ieeexplore.ieee.org/abstract/document/10392077>
- [11] Dataprix. «BI Usability: evolución y tendencia. »dirección: <https://www.dataprix.com/articulo/business-intelligence/bi-usability-evolucion-y-tendencia>
- [12] Gravitar. «Tipos de analítica de datos. »dirección: <https://gravitar.biz/bi/tipos-analitica/>
- [13] Instituto Tecnológico Europeo. «¿Qué es un dashboard en Power BI? Todo lo que necesitas saber. »dirección: <https://institutotecnologicoeuropeo.com/que-es-un-dashboard-en-power-bi-todo-lo-que-necesitas-saber/>
- [14] Trusted Shops. «¿Qué son los KPI y para qué sirven? »Dirección: <https://business.trustedshops.es/blog/que-son-kpi/>
- [15] Bismart. «Cómo transformar datos en insights de negocio. »dirección: <https://blog.bismart.com/transformar-datos-en-insights>
- [16] M. Golfarelli y S. Rizzi, *Data Warehouse Design: Modern Principles and Methodologies*. McGraw-Hill, 2009.
- [17] OVHcloud. «What is Data Warehousing? »Dirección: <https://www.ovhcloud.com/es-es/learn/what-is-data-warehousing/>

- [18] KeepCoding. «Componentes de un Data Warehouse. »dirección: <https://keepcoding.io/blog/componentes-data-warehouse/>
- [19] TAKTIC. «Cómo se aplica el Business Intelligence. »dirección: <https://taktic.es/blog/como-se-aplica-el-business-intelligence/>
- [20] R. Kimball y M. Ross, *The Data Warehouse Toolkit*, 3.^a ed. Wiley, 2013.
- [21] Gnosis IT. «Modelado dimensional: la base del análisis de datos.» Accessed: 2026-06-04. dirección: <https://gnosis-it.com/blog/modelado-dimensional-base-analisis-datos>
- [22] Holistics. «Kimball's Dimensional Data Modeling. »dirección: <https://www.holistics.io/books/setup-analytics/kimball-s-dimensional-data-modeling/>
- [23] Microsoft. «¿Qué es Power BI? »Dirección: <https://learn.microsoft.com/es-es/power-bi/fundamentals/power-bi-overview>
- [24] IEBS Business School. «Microsoft Power BI: analítica y usabilidad. »dirección: <https://www.iebschool.com/hub/microsoft-power-bi-analitica-usabilidad/>
- [25] Immune Technology Institute. «¿Qué es Tableau y para qué sirve? »Dirección: <https://immune.institute/blog/tableau-que-es/>
- [26] Wikimedia Commons. «Tableau Logo. »dirección: https://commons.wikimedia.org/wiki/File:Tableau_Logo.png
- [27] Qlik. «Diferencias entre QlikView y Qlik Sense. »dirección: https://help.qlik.com/es-ES/qlikview/September2025/Content/QV_HelpSites/Difference-qlikview-qliksense.htm
- [28] Wikimedia Commons. «Qlik Logo. »dirección: https://commons.wikimedia.org/wiki/File:Qlik_Logo.svg
- [29] beServices. «Looker, una plataforma de inteligencia empresarial. »dirección: <https://blog.beservices.es/blog/looker-una-plataforma-de-inteligencia-empresarial>
- [30] Tekne DataLabs. «Herramienta Looker. »dirección: <https://teknedatalabs.com/es/herramienta-looker/>
- [31] Amazon Web Services. «¿Qué es Python? »Dirección: <https://aws.amazon.com/es/what-is/python/>
- [32] Logo.wine. «Python Programming Language Logo. »dirección: [https://download.logo.wine/logo/Python_\(programming_language\)/Python_\(programming_language\)-Logo.wine.png](https://download.logo.wine/logo/Python_(programming_language)/Python_(programming_language)-Logo.wine.png)
- [33] OpenWebinars. «Introducción al lenguaje R. »dirección: <https://openwebinars.net/blog/introduccion-lenguaje-r/>
- [34] Logo.wine. «R Programming Language Logo. »dirección: [https://download.logo.wine/logo/R_\(programming_language\)/R_\(programming_language\)-Logo.wine.png](https://download.logo.wine/logo/R_(programming_language)/R_(programming_language)-Logo.wine.png)
- [35] Liora. «Todo sobre D3.js. »dirección: <https://liora.io/es/todo-sobre-d3-js>
- [36] Dribbble. «D3.js Logo. »dirección: <https://dribbble.com/shots/2456036-D3-js-Logo>
- [37] Amazon Web Services. «¿Qué es SQL? »Dirección: <https://aws.amazon.com/es/what-is/sql/>

- [38] Intuji. «What is MySQL? Definition, Comparison with SQL, Benefits and Features Explained. »dirección: <https://intuji.com/what-is-mysql-definition-comparison-with-sql-benefits-features-explained/>
- [39] ISDI Foundation. «El origen y evolución de la Ciencia de Datos (Data Science). »dirección: <https://isdfundacion.org/2021/07/02/el-origen-y-evolucion-de-la-ciencia-de-datos-data-science/>
- [40] Insightsoftware. «Comparing Descriptive, Predictive, Prescriptive, and Diagnostic Analytics. »dirección: <https://insightsoftware.com/es/blog/comparing-descriptive-predictive-prescriptive-and-diagnostic-analytics/>
- [41] Immune Technology Institute. «Profesiones que trabajan con datos. »dirección: <https://immune.institute/blog/profesiones-que-trabajan-con-datos/>
- [42] Santander Open Academy. «Fundamentos de Power BI,» visitado 5 de mayo de 2026. dirección: <https://lms.santanderopenacademy.com/courses/408>
- [43] Olist. «Brazilian E-Commerce Public Dataset by Olist,» visitado 5 de mayo de 2026. dirección: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- [44] Zeus Vision. «Diseño para dashboards: más que una herramienta, una experiencia visual. »dirección: https://zeus.vision/business-intelligence/disenio-para-dashboards-mas-que-una-herramienta-una-experiencia-visual/#1_Jerarquia_Visual
- [45] Bismart. «10 trucos para usar menos color en visualización de datos. »dirección: <https://blog.bismart.com/10-trucos-menos-color-visualizacion-de-datos>
- [46] Microsoft. «Power BI Desktop. »dirección: <https://powerbi.microsoft.com/es-es/desktop/>